



Universidad Loyola

Grado en Ingeniería Informática y Tecnologías Virtuales

TRABAJO FIN DE GRADO

Aprendizaje por refuerzo para la ciberseguridad: un defensor basado en flujos para decisiones binarias de permitir/bloquear

Autor: Javier Rivero Iglesias

Tutor: Alfonso Carlos Martínez Estudillo

Julio de 2026

Agradecimientos

Estas líneas cierran una etapa que no habría sido posible recorrer solo, y es de justicia dedicárselas a quienes la han hecho más llevadera.

A mis padres, Juan y Esmeralda, por su apoyo incondicional, por sostenerme en los momentos de duda y por aguantarme (que no es poco) durante todos estos años de estudio. Todo lo bueno que haya en este trabajo empezó, sin duda, en casa.

A mi hermano, Carlos, por soportarme en el día a día, por las conversaciones a deshoras y por recordarme, cuando más falta hacía, que siempre hay vida más allá de una pantalla.

A mi tutor, Alfonso Carlos, por guiarme durante todo este tiempo con paciencia y criterio, por sus consejos en cada reunión y por la confianza depositada en este proyecto desde el primer día.

A todos ellos, gracias de corazón: este trabajo también es vuestro.

Resumen

Este Trabajo Fin de Grado diseña y evalúa un agente autónomo de ciberseguridad que decide, flujo a flujo, si el tráfico de una red debe permitirse o bloquearse. El problema se formula como una tarea de decisión sensible al coste mediante aprendizaje por refuerzo: un agente QRDQN (*Quantile Regression Deep Q-Network*) observa un vector canónico de 152 dimensiones (76 características de flujo y 76 indicadores de presencia) y recibe recompensas asimétricas que penalizan más dejar pasar un ataque que bloquear tráfico legítimo. Con factor de descuento nulo, la formulación equivale a un bandido contextual, decisión que se declara y justifica explícitamente.

El agente se entrena sobre el conjunto CICIDS2017 con una partición aleatoria estratificada 80/20 y se somete a una escalera de validación de exigencia creciente: evaluación directa, reentrenamiento con etiquetas barajadas, partición temporal por días, cuantificación de duplicados entre particiones, comparación con una línea base de Random Forest bajo el mismo protocolo y una segunda fase de inferencia *offline* sobre tráfico capturado en un laboratorio doméstico cerrado.

En la partición aleatoria el agente alcanza un F1 de ataque de 0.984, con un recall de 0.995 y una tasa de falsos positivos del 0.66 %; no obstante, se cuantifica que el 24.86 % de las filas de prueba está duplicado en el entrenamiento, por lo que estas cifras se interpretan como una cota optimista. Bajo la partición por

días, el recall de ataque desciende a 0.530 (medido con una red reducida de comprobación), frente al 0.081 del Random Forest, que domina en distribución pero se desploma ante el cambio temporal. La Fase 2 alcanza un F1 de 0.984 con una tasa de bloqueo del 25.2 %, con validez externa limitada. Se concluye que la generalización dura, y no el rendimiento sobre el benchmark cómodo, es la señal creíble de utilidad defensiva.

Palabras clave: aprendizaje por refuerzo; detección de intrusiones en red; CICIDS2017; generalización; QRDQN.

Índice general

Agradecimientos	i
Resumen	ii
Índice de figuras	xii
Índice de tablas	xiv
Índice de algoritmos	xvi
Glosario de acrónimos y términos	xvii
1. Introducción	1
1.1. Motivación	1
1.2. Definición del problema	3
1.2.1. Desafíos metodológicos	3
1.2.2. Especificación verificable del proyecto	4
1.3. Enfoque propuesto	6
1.4. Contribución de la tesis	7
1.5. Alcance y limitaciones	8
1.6. Estructura del documento	9

2. Objetivos, alcance y planificación	11
2.1. Objetivo general	11
2.2. Objetivos contractuales	12
2.3. Objetivos específicos	12
2.3.1. Representación del tráfico basada en flujos (OBJ-01) . .	14
2.3.2. Preprocesamiento canónico y esquema de características (OBJ-02)	15
2.3.3. Entorno de aprendizaje por refuerzo (OBJ-03)	16
2.3.4. Agente de decisión binaria basado en QRDQN (OBJ-04)	17
2.3.5. Comparación con línea base supervisada (OBJ-05)	18
2.3.6. Evaluación con control de fugas (OBJ-06)	19
2.3.7. Análisis de escala de entrenamiento y eficiencia de datos (OBJ-07)	20
2.3.8. Validación complementaria con tráfico de laboratorio (OBJ-08)	21
2.4. Alcance del trabajo	23
2.5. No objetivos	24
2.6. Resultados esperados	25
2.7. Restricciones	26
2.8. Recursos	26
2.9. Planificación temporal	29
3. Estado del arte	31
3.1. Introducción y alcance del capítulo	31
3.2. Detección de intrusiones y monitorización de seguridad en red .	32
3.3. Representación del tráfico de red	35
3.4. Conjuntos de datos públicos para la detección de intrusiones en red	38
3.5. CICIDS2017 como benchmark interno principal	40
3.6. Aprendizaje supervisado y aprendizaje profundo para NIDS . . .	42

3.7. Fundamentos del aprendizaje por refuerzo	45
3.8. Q-Learning profundo y aprendizaje por refuerzo distribucional .	47
3.9. Aprendizaje por refuerzo para la detección de intrusiones y la defensa cibernética	49
3.10. Clasificación como RL y decisiones de seguridad binarias	52
3.11. Riesgos metodológicos en NIDS basado en ML/DL/RL	53
3.12. Protocolos de evaluación, métricas y reproducibilidad	55
3.13. Eficiencia de datos y evaluación a escala de entrenamiento . . .	58
3.14. Brecha de investigación y posicionamiento de esta tesis	59
4. Diseño del sistema	62
4.1. Visión general metodológica	62
4.2. Fuente de datos y representación de la entrada	64
4.2.1. CSVs de flujos de CICIDS2017	64
4.2.2. Mapeo de etiquetas binarias	65
4.2.3. Representación tabular a nivel de flujo	65
4.3. Limpieza de datos y preprocesamiento	66
4.3.1. Normalización de columnas y coerción numérica	66
4.3.2. Valores inválidos, valores ausentes e imputación	67
4.3.3. Exclusión de campos susceptibles de fuga	67
4.4. Esquema canónico de características	68
4.4.1. Características canónicas de flujo	68
4.4.2. Máscara de presencia/ausencia	70
4.4.3. Vector de observación final	71
4.5. Formulación del aprendizaje por refuerzo	71
4.5.1. Espacio de observación	73
4.5.2. Espacio de acciones	73
4.5.3. Función de recompensa	74
4.5.4. Estructura del episodio	76
4.5.5. Estado del entorno y protocolo de transición	76

4.5.6.	Limitaciones de la formulación conjunto-de-datos-como-entorno	79
4.6.	Agente QRDQN	79
4.6.1.	Papel algorítmico	79
4.6.2.	Función de pérdida de regresión cuantílica	80
4.6.3.	Factor de descuento $\gamma = 0$: formulación de un solo paso	81
4.6.4.	Estrategia de exploración	82
4.6.5.	Configuración del entrenamiento	83
4.6.6.	Procedencia de los hiperparámetros	85
4.6.7.	Persistencia del modelo y el preprocesamiento	85
4.7.	Marco de implementación	87
4.7.1.	Pila de procesamiento de datos y aprendizaje supervisado	87
4.7.2.	Gestión de artefactos	88
4.7.3.	Controles de reproducibilidad	88
5.	Protocolo experimental	89
5.1.	Fases experimentales	89
5.1.1.	Fase 1: benchmark interno de CICIDS2017	89
5.1.2.	Fase 2: inferencia offline sobre tráfico de laboratorio	91
5.2.	Separación entrenamiento-prueba y escalado	92
5.2.1.	Partición aleatoria estratificada	92
5.2.2.	Partición por CSV/día	93
5.2.3.	Partición leave-one-CSV-out	93
5.2.4.	Estandarización ajustada sobre entrenamiento	94
5.3.	Línea base supervisada	95
5.3.1.	Línea base con Random Forest	95
5.3.2.	Comparación bajo el mismo protocolo	95
5.3.3.	Desbalanceo de clases y ponderación sensible al coste	96
5.4.	Protocolo de escala de entrenamiento y eficiencia de datos	96
5.4.1.	Protocolo de curva de aprendizaje	97

5.4.2.	Niveles de presupuesto y comparación	97
5.5.	Salidas de evaluación y reproducibilidad	99
5.5.1.	Métricas	99
5.5.2.	Escalera de validación	99
5.5.3.	Artefactos de ejecución	102
5.6.	Pipeline de inferencia offline en la Fase 2	103
5.6.1.	Mapeo canónico para flujos de laboratorio	103
5.6.2.	Diagnósticos de cambio de distribución	106
5.6.3.	Métricas de verdad cuando están disponibles	107
5.7.	Limitaciones metodológicas	107
5.7.1.	Entorno estático y toma de decisiones secuencial	107
5.7.2.	Mapeo de etiquetas binarias y detalle por familia de ataque	108
5.7.3.	Fragilidad del benchmark y generalización	108
5.7.4.	Algoritmo único y alcance de la reproducibilidad	109
6.	Resultados	110
6.1.	Condiciones generales y trazabilidad	110
6.2.	Experimento 1: ejecución MAIN sobre la partición aleatoria fija	111
6.2.1.	Condiciones de ejecución	111
6.2.2.	Dinámica de entrenamiento	112
6.2.3.	Resultados en prueba	112
6.2.4.	Discusión breve	113
6.3.	Intervalos de confianza bootstrap	114
6.4.	Escalera de validación: comprobaciones A, B y C	115
6.5.	Análisis de duplicados y fuga en la partición aleatoria	116
6.6.	Línea base de Random Forest bajo el mismo protocolo	117
6.7.	Inferencia de la Fase 2 sobre tráfico de laboratorio	120
6.8.	Síntesis y cobertura de objetivos	121
7.	Discusión	124
7.1.	Lectura cuantitativa del rendimiento principal	124

7.2. Partición aleatoria frente a generalización por día	125
7.3. QRDQN frente a Random Forest	126
7.4. Posicionamiento frente al estado del arte	127
7.5. Lectura multiobjetivo de seguridad e impacto	128
7.6. Amenazas a la validez	128
8. Limitaciones	130
8.1. Modelo y semilla únicos	130
8.2. Fuga en la partición aleatoria	131
8.3. Comparación con la línea base	132
8.4. Validez externa de la Fase 2	133
8.5. Experimentos planificados pero no ejecutados	133
9. Consideraciones éticas y legales	135
9.1. Uso dual y alcance estrictamente defensivo	135
9.2. Privacidad y tratamiento de los datos	136
9.3. Riesgo operativo y asimetría de errores	137
9.4. Reproducibilidad como práctica responsable	138
10. Conclusiones	140
10.1. Objetivos contractuales	140
10.2. Síntesis de los objetivos específicos	142
10.3. Reflexión del autor	143
10.4. Líneas futuras	144
10.4.1. Corto plazo: completar la evidencia diseñada	144
10.4.2. Medio plazo: ampliar la comparación y la validación externa	144
10.4.3. Largo plazo: tratar incertidumbre y transición operacional	145
Bibliografía	146
A. Manual de reproducibilidad	162
A.1. Obtención del repositorio y de los datos	163

A.1.1.	Clonado y Git LFS	163
A.1.2.	Condiciones y procedencia de CICIDS2017	164
A.2.	Instalación del entorno	164
A.2.1.	Entorno local con uv	164
A.2.2.	Entorno RunPod mediante venv	165
A.2.3.	Comprobación mínima del entorno	166
A.3.	Entrenamiento canónico	166
A.3.1.	Smoke test del perfil MAIN	166
A.3.2.	Entrenamiento MAIN exacto	167
A.4.	Verificación del split fijo y del escalador	168
A.5.	Análisis y validaciones reproducibles	169
A.5.1.	Intervalos bootstrap	169
A.5.2.	Análisis de duplicados	170
A.5.3.	Línea base Random Forest	170
A.5.4.	Checks A, B y C con el código actual	171
A.6.	Inferencia MAIN de la Fase 2	172
A.7.	Experimentos diferidos con GPU	173
A.7.1.	Leave-one-CSV-out de QRDQN	173
A.7.2.	Escalera de tamaños con test fijo	174
A.7.3.	Estudio multisevilla del pipeline	175
B.	Entorno hardware y software	177
B.1.	Hardware efectivamente empleado	177
B.2.	Runtime registrado en MAIN	178
B.3.	Contratos de dependencias	179
B.4.	Entorno de construcción de la memoria	180
C.	Esquema canónico completo	182
D.	Catálogo de artefactos	190
D.1.	Convenciones de lectura	190

D.2. Proveniencia de la entrada de Fase 2	195
D.3. Elementos excluidos del catálogo oficial	195

Índice de figuras

2.1. Diagrama de Gantt del proyecto	30
4.1. Pipeline general del sistema	64
4.2. Construcción del vector de observación	73
4.3. Interacción agente-entorno	75
4.4. Arquitectura QRDQN de la ejecución MAIN	80
5.1. Composición visual por día de CICIDS2017	91
5.2. Balance de clases en la partición aleatoria fija	92
5.3. Curvas de aprendizaje pendientes	98
5.4. Escalera de validación del proyecto	101
5.5. Pipeline de inferencia offline de la Fase 2	106
6.1. Curvas de entrenamiento de MAIN	112
6.2. Matriz de confusión de MAIN	113
6.3. Intervalos bootstrap de MAIN	115
6.4. Matriz de confusión de la Comprobación C	116
6.5. Duplicados y fuga entre entrenamiento y prueba	117
6.6. QRDQN frente a Random Forest	119
6.7. Matriz de confusión de Random Forest por día	120
6.8. Resultados de la Fase 2	121

8.1. Validación externa futura pendiente	134
--	-----

Índice de tablas

1.1. Especificación verificable del proyecto	5
2.1. Objetivo contractual C1	12
2.2. Objetivo contractual C2	13
2.3. Objetivo contractual C3	13
2.4. Objetivo contractual C4	14
2.5. Matriz de trazabilidad de objetivos	23
2.6. Catálogo de restricciones del proyecto	27
2.7. Recursos hardware y software del proyecto	28
2.8. Hitos y fases del proyecto	29
3.1. Comparación de conjuntos de datos públicos de NIDS	39
3.2. Trabajos de RL para NIDS	50
4.1. Agrupación lógica de las 76 características canónicas	69
4.2. Matriz de recompensa del entorno del defensor binario	74
4.3. Lectura multiobjetivo de la función de recompensa	76
4.4. Hiperparámetros de la ejecución oficial MAIN	86
5.1. Composición por día de CICIDS2017	90
5.2. Inventario de ejecuciones oficiales	104

6.1. Métricas del modelo MAIN (partición aleatoria fija)	113
6.2. Intervalos de confianza bootstrap del modelo MAIN	114
6.3. QRDQN frente a Random Forest bajo el mismo protocolo	118
6.4. Comparación leave-one-CSV-out pendiente	119
6.5. Cobertura de objetivos en el capítulo de resultados	123
8.1. Tabla de varianza entre semillas pendiente	131
8.2. Comparación ampliada de líneas base pendiente	132
B.1. Hardware empleado en el proyecto	178
B.2. Runtime de la ejecución principal	179
B.3. Paquetes registrados en la ejecución principal	179
B.4. Contratos de dependencias del proyecto	180
B.5. Entorno de construcción de la memoria	181
C.1. Esquema canónico completo	183
D.1. Catálogo de artefactos de entrenamiento, validación y Fase 2 . .	191

Índice de algoritmos

1.	Mapeo al esquema canónico y construcción de la observación . .	72
2.	Paso de decisión del entorno	78
3.	Entrenamiento del defensor QRDQN	84
4.	Inferencia offline robusta de la Fase 2	105

Glosario de acrónimos y términos

Acrónimos

ACK	<i>Acknowledgement</i> , flag TCP que confirma recepción de datos.
CIC	Canadian Institute for Cybersecurity (Universidad de Nuevo Brunswick), creador del conjunto de datos CICIDS2017.
CICIDS2017	Conjunto de datos de referencia para la detección de intrusiones en red, publicado por el CIC en 2017.
CPU	Unidad central de procesamiento (<i>Central Processing Unit</i>).
CSE	Communications Security Establishment, organismo coautor del conjunto CSE-CIC-IDS2018.
CSV	Valores separados por comas (<i>comma-separated values</i>), formato de los ficheros tabulares usados por el proyecto.
DDoS	Denegación de servicio distribuida (<i>Distributed Denial of Service</i>).
DDPG	<i>Deep Deterministic Policy Gradient</i> , algoritmo actor-crítico citado como antecedente de RL continuo.
DL	Aprendizaje profundo (<i>Deep Learning</i>).

DQN	<i>Deep Q-Network</i> , algoritmo de aprendizaje por refuerzo profundo basado en valores.
DRL	Aprendizaje por refuerzo profundo (<i>Deep Reinforcement Learning</i>).
ECE	<i>Explicit Congestion Notification Echo</i> , flag TCP incluido entre las estadísticas de flujo.
F1	Media armónica de la precisión y el recall.
FIN	Flag TCP que indica cierre ordenado de una conexión.
FN / FNR	Falso negativo (ataque no bloqueado) / tasa de falsos negativos.
FP / FPR	Falso positivo (tráfico benigno bloqueado) / tasa de falsos positivos.
GPU	Unidad de procesamiento gráfico (<i>Graphics Processing Unit</i>).
HIDS	Sistema de detección de intrusiones en host (<i>Host Intrusion Detection System</i>).
HTTP	<i>Hypertext Transfer Protocol</i> , protocolo de aplicación citado como ejemplo de tráfico de red.
IC	Intervalo de confianza.
ID	Identificador; en esta tesis se evita como característica cuando puede introducir fuga de información.
IDS	Sistema de detección de intrusiones (<i>Intrusion Detection System</i>).
IoT	Internet de las cosas (<i>Internet of Things</i>).
IP	Protocolo de Internet; las direcciones IP se excluyen de las características del modelo por riesgo de fuga.
IPFIX	<i>IP Flow Information Export</i> , estándar de exportación de registros de flujo.

IPS	Sistema de prevención de intrusiones (<i>Intrusion Prevention System</i>).
JSON	<i>JavaScript Object Notation</i> , formato usado para configuraciones, métricas y manifiestos de artefactos.
KDD	KDD Cup 1999, conjunto de datos histórico de detección de intrusiones.
MCC	Coficiente de correlación de Matthews (<i>Matthews Correlation Coefficient</i>).
ML	Aprendizaje automático (<i>Machine Learning</i>).
MLP	Perceptrón multicapa (<i>Multilayer Perceptron</i>).
NIDS	Sistema de detección de intrusiones en red (<i>Network Intrusion Detection System</i>).
NIST	National Institute of Standards and Technology, organismo de referencia en guías técnicas de seguridad.
NSL-KDD	Revisión depurada del conjunto KDD Cup 1999.
POMDP	Proceso de decisión de Markov parcialmente observable (<i>Partially Observable Markov Decision Process</i>).
PSH	<i>Push</i> , flag TCP incluido entre las estadísticas de flujo.
QRDQN	<i>Quantile Regression Deep Q-Network</i> , variante distribucional de DQN empleada en este trabajo.
RF	<i>Random Forest</i> , línea base supervisada de comparación.
RL	Aprendizaje por refuerzo (<i>Reinforcement Learning</i>).
RST	<i>Reset</i> , flag TCP que indica reinicio abrupto de una conexión.
RUN_ID	Identificador único de ejecución usado para localizar configuración, métricas y artefactos reproducibles.
SB3	Stable-Baselines3, biblioteca de algoritmos de RL; en el proyecto se usa junto con <code>sb3-contrib</code> .
SDN	Redes definidas por software (<i>Software-Defined Networking</i>).

SHA-256	Función hash criptográfica usada para verificar identidad de artefactos o particiones.
SIEM	Gestión de eventos e información de seguridad (<i>Security Information and Event Management</i>).
SOAR	Orquestación, automatización y respuesta de seguridad (<i>Security Orchestration, Automation and Response</i>).
SYN	Flag TCP usado en el establecimiento de conexión.
TCP	Protocolo de control de transmisión (<i>Transmission Control Protocol</i>).
TFG	Trabajo Fin de Grado.
TLS	Seguridad de la capa de transporte (<i>Transport Layer Security</i>).
TN / TP	Verdadero negativo / verdadero positivo.
UNSW-NB15	Conjunto de datos de detección de intrusiones de la Universidad de Nueva Gales del Sur (2015).
URG	<i>Urgent</i> , flag TCP incluido entre las estadísticas de flujo.

Términos

Bandido	contextual	Formulación de decisión de un solo paso en la que la acción no modifica los estados futuros. En esta tesis describe la consecuencia de usar $\gamma = 0$ sobre un conjunto de datos estático.
Benchmark		Conjunto de datos y protocolo usados como referencia de evaluación. CICIDS2017 funciona aquí como benchmark interno, no como prueba de rendimiento en producción.

Bootstrap	Técnica de remuestreo usada para estimar intervalos de confianza sobre el conjunto de prueba fijo. En esta memoria cuantifica precisión de muestreo, no varianza entre semillas de entrenamiento.
Buffer de repetición	Memoria de transiciones usada por DQN/QRDQN para muestrear minibatches durante el entrenamiento y reducir correlaciones inmediatas entre pasos.
Cambio de dominio	Diferencia entre la distribución usada para entrenar el modelo y la distribución sobre la que se aplica después; por ejemplo, CICIDS2017 frente a tráfico de laboratorio.
Despliegue activo	Integración operativa que modifica tráfico real, por ejemplo bloqueando paquetes o flujos. Este trabajo no implementa despliegue activo.
Escalador	Transformación de normalización ajustada sobre entrenamiento, aquí <code>StandardScaler</code> ; debe reutilizarse en inferencia sin recalcularse sobre datos de prueba o laboratorio.
F1	Métrica que combina precisión y recall mediante su media armónica; resulta útil cuando hay desbalanceo, pero no sustituye a la lectura de falsos positivos y falsos negativos.
Falso negativo	Error en el que un ataque se clasifica como benigno o se permite. En la tesis es el error más penalizado por la recompensa.
Falso positivo	Error en el que tráfico benigno se clasifica como ataque o se bloquea. Afecta a disponibilidad y carga operativa.

Flujo	Resumen tabular de una comunicación de red, normalmente bidireccional, descrita por estadísticas agregadas de paquetes, bytes, tiempos y flags.
Inferencia offline	Aplicación del modelo sobre ficheros ya capturados y almacenados, sin actuar sobre la red en vivo.
Leave-one-CSV-out	Protocolo de validación que retiene un CSV completo como prueba y entrena con los restantes, tratando cada fichero como una unidad de dominio.
Línea base	Modelo comparativo usado para contextualizar los resultados del método principal; en esta tesis, Random Forest es la línea base supervisada principal.
Máscara de presencia	Vector binario que indica si cada característica canónica procede de una columna fuente disponible o ha sido imputada por ausencia de columna.
Partición aleatoria	División entrenamiento/prueba por filas, normalmente estratificada. Es útil como comprobación de aprendizaje, pero puede sobreestimar generalización cuando hay duplicados o dependencia entre filas.
Partición por día	División que separa ficheros completos de CICIDS2017 según día o escenario de captura. Evalúa generalización de forma más estricta que la partición aleatoria.
Pipeline	Secuencia reproducible de pasos de procesamiento, entrenamiento, evaluación o inferencia. En esta tesis incluye carga, limpieza, mapeo canónico, escalado, modelo y artefactos de salida.
Precisión	Proporción de predicciones positivas que son correctas. Para la clase ataque, mide cuántos bloqueos o alertas corresponden realmente a ataques.

Recall	Proporción de ejemplos reales de una clase que el modelo detecta. Para la clase ataque, mide qué fracción de ataques se bloquea o identifica.
Recorte por percentiles	Limitación de valores crudos a rangos observados en entrenamiento, por ejemplo entre los percentiles 0.5 y 99.5. Sirve para acotar valores extremos en inferencia.
Recorte por z-score	Limitación de valores escalados a un umbral absoluto de desviaciones estándar. En Fase 2 ayuda a controlar valores fuera de distribución después del escalado.
Red objetivo	Copia estabilizadora de la red de valores usada en DQN/QRDQN para calcular objetivos temporales. Con $\gamma = 0$ no contribuye al objetivo de esta tesis.

Introducción

Este capítulo sitúa el problema de investigación, delimita una especificación verificable para el prototipo y adelanta las contribuciones, el alcance y la organización de la memoria. La intención es distinguir desde el inicio entre la evidencia que puede obtenerse mediante experimentación offline y cualquier afirmación de preparación para un despliegue operativo.

1.1 Motivación

Las infraestructuras de red modernas generan volúmenes de tráfico que superan con creces lo que los analistas humanos pueden inspeccionar manualmente. Una única red empresarial puede producir millones de registros de flujo al día, y los despliegues a gran escala agregan flujos procedentes de miles de endpoints, servicios en la nube y enlaces entre centros de datos (Scarfone et al. [2007](#)). Distinguir la comunicación legítima de la actividad maliciosa dentro de este volumen es uno de los desafíos operativos centrales en la seguridad de redes. Las respuestas tradicionales se apoyan en sistemas de detección de intrusiones en red que aplican coincidencia de firmas, umbrales estadísticos o reglas específicas de protocolo para señalar actividad sospechosa. Estos enfoques

son efectivos frente a patrones de amenaza conocidos, pero requieren actualizaciones manuales constantes de reglas, resultan en gran medida ineficaces frente a variantes novedosas de ataque y generan volúmenes de alertas que habitualmente desbordan a los equipos de seguridad.

El aprendizaje automático ofrece una perspectiva complementaria. En lugar de codificar el conocimiento experto como reglas estáticas, los modelos de ML pueden aprender patrones estadísticos a partir de datos de tráfico etiquetados y generalizar hacia instancias de ataque no vistas previamente. A medida que el campo ha madurado, las representaciones a nivel de flujo se han convertido en un sustrato práctico para este aprendizaje: cada conexión de red se resume como un conjunto fijo de características estadísticas que cubren duración, conteos de paquetes, totales de bytes, tiempos entre llegadas, distribuciones de flags TCP y métricas basadas en tasas (Sperotto et al. 2010; Umer et al. 2017). Esta reducción hace abordable el aprendizaje a gran escala y evita las complicaciones legales y técnicas de la inspección profunda de paquetes, que resulta cada vez más ineficaz de todos modos a medida que crece la fracción de tráfico cifrado en internet.

La limitación clave del aprendizaje supervisado estándar en este contexto es que trata la detección de intrusiones como un problema de clasificación simétrico, minimizando el error de clasificación global sin tener en cuenta las consecuencias operativas de los distintos tipos de error (Axelsson 2000). En la práctica, esas consecuencias son profundamente asimétricas. Un falso negativo permite a un actor de amenazas avanzar sin ser detectado y puede conducir a exfiltración de datos, despliegue de ransomware o interrupción del servicio. Un falso positivo genera una alerta o un bloqueo de tráfico que consume tiempo del analista, pero rara vez causa daños duraderos. El aprendizaje por refuerzo proporciona un marco natural para codificar esta asimetría directamente en el objetivo de aprendizaje: al diseñar una función de recompensa que penaliza los ataques no detectados más severamente que las falsas alarmas, un agente de RL puede ser guiado hacia políticas que reflejen las prioridades de riesgo

operativo real en lugar de la exactitud agregada (Sutton et al. 2018; Lee et al. 2002). Explorar cómo un algoritmo de RL distribucional como QRDQN navega estos compromisos sobre datos a nivel de flujo es la motivación central de esta tesis (Dabney et al. 2018).

1.2 Definición del problema

1.2.1 Desafíos metodológicos

Aplicar el aprendizaje por refuerzo a la detección de intrusiones en red plantea varios desafíos metodológicos que van más allá de sustituir un algoritmo de RL por un clasificador estándar. El primer desafío es la calidad de los datos. Los benchmarks públicos de NIDS como CICIDS2017 son ampliamente utilizados, pero contienen artefactos conocidos de su pipeline de extracción de flujos, entre ellos ruido en las etiquetas, características duplicadas, fallos de implementación de CICFlowMeter y campos estadísticos que codifican inadvertidamente el contexto del día de captura (Ring et al. 2019; Engelen et al. 2021; Lanvin et al. 2023). Los modelos entrenados sin auditar estos artefactos pueden aprender a reconocer firmas de extracción o identificadores específicos de escenario en lugar de comportamiento de ataque genuino, produciendo puntuaciones infladas en el benchmark que no se transfieren a ningún entorno nuevo.

El segundo desafío es la fuga de información. Muchos estudios publicados de NIDS evalúan los modelos utilizando particiones aleatorias por filas sobre todo el tráfico disponible, lo que puede situar flujos altamente correlacionados a ambos lados del límite entrenamiento-prueba. Más allá de las filas duplicadas, la fuga puede introducirse a través de direcciones IP, marcas temporales absolutas, identificadores del flujo (*Flow IDs*) y números de puerto que son específicos de un escenario de ataque programado en lugar de indicativos del comportamiento

malicioso en general (Arp et al. 2020; Kaufman et al. 2011). Los pasos de preprocesamiento como el escalado de características, aplicados globalmente antes de la partición, pueden contaminar adicionalmente la evaluación al permitir que la información distribucional de la partición de prueba influya en la normalización del entrenamiento. Un estudio riguroso no puede limitarse a elegir un algoritmo de alto rendimiento y reportar su exactitud sobre un conjunto de datos público dividido aleatoriamente.

El tercer desafío es la comparación justa contra un baseline de un método estándar. Los métodos de DRL pueden ser costosos de ajustar, sensibles a los hiperparámetros y más lentos en converger que los modelos supervisados tabulares. Random Forest y los ensamblados de árboles con *gradient boosting* son líneas base sólidas para la detección de intrusiones a nivel de flujo y pueden alcanzar puntuaciones casi perfectas en CICIDS2017 bajo condiciones de evaluación favorables (H. Liu et al. 2019; Apruzzese et al. 2022). Un defensor basado en RL no puede evaluarse de manera significativa sin una comparación bajo el mismo protocolo frente a dichas líneas base. La pregunta no es si QRDQN alcanza alta exactitud en una sola ejecución, sino si su formulación sensible al coste ofrece alguna ventaja empírica sobre un modelo supervisado bien ajustado bajo esquemas de características, protocolos de partición y métricas de evaluación equivalentes.

1.2.2 Especificación verificable del proyecto

Como adaptación al contexto de investigación de la especificación propia de un proyecto de desarrollo de software (PDS), esta tesis formula el trabajo como un sistema experimental verificable y no como un producto de bloqueo listo para operar. La Tabla 1.1 fija el problema que se aborda, los límites que preservan la seguridad metodológica y la evidencia exigida para considerar cubierto cada compromiso.

Especificación verificable del prototipo experimental	
Necesidad	Construir y evaluar un defensor offline que clasifique flujos de red como PERMIT o BLOCK, haciendo explícita la asimetría entre permitir un ataque y bloquear tráfico legítimo.
Entradas	CSV de flujos de CICIDS2017 y, para la comprobación complementaria, flujos extraídos de tráfico de laboratorio; todos se adaptan al contrato canónico y se procesan sin usar identificadores ni otros campos propensos a fuga.
Proceso	Limpieza y escalado ajustados solo con entrenamiento, representación de $76 + 76 = 152$ dimensiones, entrenamiento QRDQN, comparación con Random Forest y escalera de validación con artefactos persistidos.
Salidas	Predicciones binarias experimentales, métricas por clase y artefactos de ejecución trazables; BLOCK permanece como salida de inferencia offline y no acciona un cortafuegos ni una red real.
Verificación	Los compromisos C1–C4, especificados en las Tablas 2.1 a 2.4, requieren código auditable, ejecuciones comprometidas y resultados interpretados dentro de las restricciones declaradas.
Límite de la afirmación	El rendimiento sobre CICIDS2017 y sobre una captura concreta de laboratorio constituye evidencia experimental acotada; no certifica generalización a producción ni bloqueo activo seguro.

Tabla 1.1: Especificación verificable del prototipo experimental, adaptada al contexto de un proyecto de desarrollo de software. Fuente: elaboración propia.

1.3 Enfoque propuesto

Esta tesis formula la tarea defensiva como un problema de aprendizaje por refuerzo offline con el conjunto de datos como entorno (Levine et al. 2020). La elección del entorno offline es deliberada: desplegar un agente de RL que deba interactuar con una red en vivo para aprender generaría riesgos de seguridad inaceptables, ya que un agente en exploración tomaría inevitablemente acciones subóptimas durante el entrenamiento. Al tratar un conjunto de datos etiquetado estático como el entorno, el agente puede entrenarse y evaluarse sin ninguna exposición a una red en vivo. El tráfico de red se procesa en un esquema canónico de flujo de 76 características fijo, independiente del formato nativo de cualquier conjunto de datos específico. Este contrato canónico separa la nomenclatura de columnas propia de la extracción del espacio de observación del modelo y hace posible procesar tanto flujos de CICIDS2017 como tráfico capturado en un laboratorio doméstico cerrado a través del mismo pipeline sin modificación alguna.

Para gestionar datos ausentes o heterogéneos al transitar entre conjuntos de datos y herramientas de captura, se añade una máscara de presencia/ausencia de 76 valores al vector de características, produciendo una observación de 152 dimensiones para el agente. Cada posición en la máscara registra si la característica canónica correspondiente estaba presente y era válida (codificada como 1) o estaba ausente y fue reemplazada por un valor por defecto (codificada como 0). Esto hace que los huecos en los datos sean visibles tanto para el modelo como para el evaluador, evitando el problema de la imputación silenciosa en el que una red aprende sobre valores de sustitución sin ninguna indicación de que la medición subyacente no estaba disponible. Al pasar de CICIDS2017 al tráfico capturado en laboratorio, distintos pipelines de extracción pueden producir conjuntos de columnas diferentes, y la máscara de presencia/ausencia garantiza que cualquier discrepancia de ese tipo se codifique en la observación en lugar

de quedar oculta a la inspección.

La tarea de decisión se mapea a un espacio de acciones binario: **PERMIT** para el tráfico clasificado como benigno, y **BLOCK** para el tráfico clasificado como ataque. Un agente *Quantile Regression Deep Q-Network* (Dabney et al. 2018), implementado a través de la biblioteca **sb3-contrib** (Stable-Baselines3 Contributors 2023), se entrena dentro de un entorno compatible con Gymnasium (Farama Foundation 2023). El entorno aplica una función de recompensa asimétrica que penaliza los falsos negativos más severamente que los falsos positivos, codificando directamente las prioridades de riesgo operativo descritas anteriormente (Cárdenas et al. 2006). El conjunto de datos CICIDS2017 (Sharafaldin et al. 2018; Canadian Institute for Cybersecurity 2017) sirve como benchmark interno primario, procesado a través de un pipeline de carga curado que excluye los campos propensos a fuga de información antes de que comience cualquier entrenamiento del modelo. El rendimiento se evalúa frente a una línea base de Random Forest bajo el mismo esquema de características y protocolo de partición (Engelen et al. 2021). Un segundo nivel de validación extiende la evaluación ejecutando inferencia offline sobre características de flujo extraídas de tráfico capturado por el operador en un laboratorio doméstico cerrado y no adversarial, ofreciendo una indicación parcial y de validez externa limitada del comportamiento del sistema ante un cambio de dominio.

1.4 Contribución de la tesis

Las contribuciones se expresan mediante los cuatro compromisos contractuales del Capítulo 2. Esta formulación permite separar el producto construido, la evidencia de evaluación y los límites de cada conclusión, en lugar de presentar el rendimiento de una sola partición como una demostración global.

C1 — Pipeline experimental reproducible con contrato de datos canónico. La primera contribución es un pipeline offline de extremo a extremo

que fija una representación de $76 + 76 = 152$ dimensiones y conserva, para cada ejecución, los artefactos necesarios para reconstruir la lectura de sus resultados. La Tabla 2.1 especifica este compromiso y la Tabla 5.2 enlaza las ejecuciones oficiales con sus configuraciones, métricas y salidas persistidas.

C2 — Defensor RL y escalera de validación con control de fugas.

La segunda contribución es la implementación de QRDQN como defensor de decisiones binarias y su evaluación mediante partición aleatoria, comprobaciones A/B/C y análisis de duplicados. La Tabla 2.2 hace explícito el peldaño *leave-one-CSV-out* de QRDQN que permanece pendiente de GPU; la cobertura empírica disponible se sintetiza en la Tabla 6.5.

C3 — Decisión sensible al coste con lectura multiobjetivo. La tercera contribución convierte la asimetría entre ataques permitidos y tráfico legítimo bloqueado en un elemento auditable del diseño, no en una interpretación posterior de la exactitud. La Tabla 2.3 enlaza esta contribución con la matriz de recompensa y su lectura de seguridad e impacto operativo (Tabla 4.3).

C4 — Comparación honesta y transferencia offline acotada. La cuarta contribución compara QRDQN y Random Forest con la misma representación y métricas en las particiones aleatoria fija y por día disponibles; en esta última, QRDQN corresponde a la Comprobación C con una red proxy, no con los pesos MAIN. La comparación *leave-one-CSV-out* de QRDQN sigue pendiente de GPU. La Tabla 2.4, la Tabla 6.3 y la Figura 6.8 aportan la evidencia; no obstante, la transferencia se formula con validez externa limitada y sin atribuir a BLOCK ningún efecto operativo.

1.5 Alcance y limitaciones

El alcance de esta investigación se limita estrictamente a la evaluación offline, tabular y a nivel de flujo. Las salidas PERMIT y BLOCK son clasificaciones binarias experimentales sobre registros de flujo estáticos; el sistema no realiza bloqueo

en vivo de cortafuegos, descarte de paquetes, manipulación de `iptables` ni prevención de intrusiones en línea. El entorno no simula un adversario activo cuyas acciones posteriores dependan de las decisiones del defensor, lo que significa que el sistema no opera como un plano de control de toma de decisiones secuenciales completo. Estas restricciones son decisiones de diseño deliberadas que hacen que la evaluación sea abordable, reproducible y segura. No son omisiones que deban subsanarse en iteraciones futuras de este mismo prototipo.

CICIDS2017 se utiliza como benchmark interno primario para la experimentación controlada. Los resultados sobre este conjunto de datos deben interpretarse como evidencia sobre el comportamiento bajo un benchmark específico y bien caracterizado, no como prueba de generalización a redes de producción arbitrarias. El tráfico de laboratorio, generado por el operador en un entorno doméstico cerrado y no adversarial, se utiliza como nivel de evaluación secundario para examinar el cambio de dominio; dada su procedencia, su validez externa es limitada y, además, también es inferencia offline y no despliegue en vivo. La tesis ocupa una posición metodológica entre la evaluación pura sobre benchmark y la validación de despliegue completo, apuntando a la reproducibilidad e interpretabilidad del marco experimental más que a la operatividad en producción.

1.6 Estructura del documento

Tras esta introducción, el Capítulo 2 fija los objetivos contractuales y específicos, el alcance, las restricciones y la planificación. El Capítulo 3 revisa el estado del arte de los NIDS basados en flujos, los conjuntos de datos públicos, el aprendizaje automático y el aprendizaje por refuerzo aplicados a detección de intrusiones, junto con sus riesgos metodológicos. El Capítulo 4 presenta el diseño del sistema: datos, limpieza anti-fuga, contrato canónico, formulación del entorno y agente QRDQN. El Capítulo 5 define el protocolo experimental,

las particiones, las métricas, la línea base y la inferencia offline de la Fase 2.

El Capítulo 6 reúne los resultados trazables de MAIN, los controles de validación, el análisis de duplicados, la comparación con Random Forest y la captura de laboratorio. El Capítulo 7 los discute desde la generalización, la seguridad y el impacto operativo; el Capítulo 8 expone sus limitaciones; y el Capítulo 9 delimita las implicaciones éticas y de uso responsable. Finalmente, el Capítulo 10 recoge las conclusiones y las líneas futuras. La bibliografía reúne las fuentes citadas. Los anexos se reservan para el manual de reproducibilidad (A), el entorno hardware/software (B), el esquema canónico completo (C) y el catálogo de artefactos por ejecución (D).

Objetivos, alcance y planificación

2.1 Objetivo general

El objetivo general de esta tesis es diseñar, implementar y evaluar rigurosamente un defensor de ciberseguridad basado en aprendizaje por refuerzo, de carácter offline, que toma decisiones binarias sensibles al coste (PERMIT o BLOCK) sobre observaciones de flujos de red estandarizados. Este objetivo se persigue mediante un *pipeline* experimental controlado que opera íntegramente sobre datos de flujo etiquetados y estáticos, priorizando el rigor metodológico y la reproducibilidad sobre cualquier pretensión de preparación para el despliegue.

El enfoque se fundamenta en la observación de que la detección de intrusiones implica costes inherentemente asimétricos: permitir un ataque conlleva consecuencias que son, en general, considerablemente más graves que las de generar una falsa alarma. Una formulación basada en recompensas hace explícita esta asimetría en el objetivo de aprendizaje, en lugar de dejarla implícita en un ajuste posterior de umbrales. La pregunta central que guía el trabajo es si un algoritmo de RL distribucional como QRDQN puede configurarse, entrenarse y evaluarse de forma que demuestre un comportamiento sensible al coste y medible sobre un benchmark público bien estudiado, manteniéndose a la vez

comparable con sólidas líneas base supervisadas bajo las mismas condiciones controladas.

2.2 Objetivos contractuales

El objetivo general se concreta en cuatro objetivos contractuales, C1 a C4, que constituyen el compromiso verificable de esta tesis. Siguiendo el estilo de especificación de un proyecto de desarrollo de software, cada uno se formula con una descripción cerrada, un tipo, un criterio de verificación anclado en artefactos concretos y los capítulos en los que se materializa (Tablas 2.1 a 2.4). La cobertura efectiva de estos compromisos se retoma en la síntesis de resultados y en las conclusiones.

C1 — Pipeline experimental reproducible con contrato de datos canónico	
Descripción	Construir un pipeline offline de extremo a extremo (ingesta de CICIDS2017, limpieza anti-fuga, esquema canónico de $76 + 76 = 152$ dimensiones, entrenamiento y evaluación) cuyas cifras sean reproducibles a partir de los artefactos persistidos por ejecución: RUN_ID, configuración, métricas, escalador y percentiles de entrenamiento.
Tipo	Producto (infraestructura experimental).
Verificación	Inventario de ejecuciones oficiales (Tabla 5.2); Algoritmos 3 y 1 contrastados con el código fuente; pruebas automatizadas del repositorio.
Capítulos	4, 5 y 6; anexos A y D.

Tabla 2.1: Especificación del objetivo contractual C1. Fuente: elaboración propia.

2.3 Objetivos específicos

C2 — Defensor RL evaluado mediante una escalera de validación con control de fugas

Descripción	Entrenar un agente QRDQN distribucional sobre la representación canónica —con $\gamma = 0$, es decir, una formulación de bandido contextual sensible al coste— y evaluarlo mediante una escalera de validación progresivamente más estricta (partición aleatoria estratificada, comprobaciones A/B/C, análisis de duplicados), haciendo explícito qué modo de fallo detecta cada peldaño.
Tipo	Producto y evaluación.
Verificación	Ejecución MAIN y comprobaciones A/B/C con artefactos comprometidos; análisis de duplicados de la partición fija (Tabla 5.2). El peldaño <i>leave-one-CSV-out</i> de QRDQN queda pendiente de ejecución en GPU.
Capítulos	4, 5 y 6.

Tabla 2.2: Especificación del objetivo contractual C2. Fuente: elaboración propia.

C3 — Decisión sensible al coste con lectura multiobjetivo

Descripción	Codificar en la función de recompensa la asimetría de costes del dominio con una lectura multiobjetivo explícita entre <i>seguridad</i> —minimizar ataques admitidos; el falso negativo, con recompensa -5.0 , es el coste dominante— e <i>impacto operativo</i> —minimizar el bloqueo de tráfico legítimo; falso positivo con recompensa -2.0 —, y analizar los resultados por clase bajo esa lectura.
Tipo	Diseño y evaluación.
Verificación	Tablas 4.2 y 4.3 contrastadas con <code>src/rl_defender_env.py</code> ; tasas de falsos positivos y falsos negativos reportadas por clase en el capítulo de resultados y leídas en clave multiobjetivo en la discusión.
Capítulos	4, 6 y 7.

Tabla 2.3: Especificación del objetivo contractual C3. Fuente: elaboración propia.

C4 — Comparación honesta con línea base y transferencia a tráfico externo	
Descripción	Comparar QRDQN con una línea base Random Forest bajo el mismo protocolo (misma representación canónica, mismas particiones y mismas métricas) y comprobar la transferencia del pipeline mediante la inferencia offline de la Fase 2 sobre tráfico de laboratorio generado por el operador, declarando su validez externa limitada.
Tipo	Evaluación.
Verificación	Barrido Random Forest con artefactos por partición y ejecución oficial de la Fase 2 (Tabla 5.2); resultados comparativos y de transferencia en el capítulo de resultados.
Capítulos	5, 6, 7 y 8.

Tabla 2.4: Especificación del objetivo contractual C4. Fuente: elaboración propia.

2.3.1 Representación del tráfico basada en flujos (OBJ-01)

El primer objetivo específico es extraer y normalizar datos de tráfico de red en registros tabulares de flujos, garantizando que el formato de entrada sea compatible tanto con benchmarks públicos como CICIDS2017 como con capturas de laboratorio offline. Los flujos de red son resúmenes bidireccionales de intercambios de paquetes que comparten una quintupla común, y su estructura tabular compacta y de anchura fija los hace muy adecuados como entradas para modelos de aprendizaje automático (Sperotto et al. 2010; Umer et al. 2017). Alcanzar este objetivo requiere auditar las características disponibles en los ficheros CSV generados por CICFlowMeter, identificar qué campos contienen señal estadística legítima y cuáles representan artefactos de extracción o posibles vectores de fuga de información.

Este objetivo implica también definir un contrato de características estable que tienda un puente entre las publicaciones académicas en formato CSV y las exportaciones brutas de CICFlowMeter. Un contrato estable significa que el conjunto de características utilizado como entrada al modelo no varía entre

experimentos, que los campos excluidos por razones de fuga se excluyen de forma consistente, y que cualquier persona que lea el código fuente puede verificar exactamente qué medidas llegan al modelo. Esta estabilidad es un requisito previo para todos los objetivos posteriores, ya que todos los demás componentes del pipeline dependen de un espacio de observación bien definido y auditable.

Criterio de verificación (OBJ-01). El contrato de características es auditable en el código (`src/canonical_schema.py`) y se aplica de forma idéntica a los CSV de CICIDS2017 y a los flujos de laboratorio de la Fase 2; el capítulo de diseño documenta el mapeo, los campos excluidos y su justificación (Algoritmo 1).

2.3.2 Preprocesamiento canónico y esquema de características (OBJ-02)

El segundo objetivo específico es definir y hacer cumplir un esquema canónico estricto de 76 características junto con una máscara de presencia/ausencia de 76 valores, lo que da lugar a un vector de observación estable de 152 dimensiones. Las 76 características canónicas se extraen de estadísticas de estilo CICFlowMeter que cubren la duración del flujo, conteos de paquetes y bytes, distribuciones de tiempos de llegada, conteos de flags TCP, estadísticas de longitud de paquetes, resúmenes de periodos activos e inactivos, y métricas de tasa de flujo. Los campos que podrían exponer información de identidad específica del conjunto de datos, como las direcciones IP de origen y destino, las marcas temporales absolutas, los *Flow ID* y los números de puerto que funcionan como proxies directos del tipo de ataque, se excluyen explícitamente a nivel de esquema, en lugar de filtrarse en pasos de preprocesamiento ad hoc.

La máscara de presencia/ausencia de 76 valores registra si cada valor canónico fue medido directamente y está presente (codificado como 1) o ausente y sustituido por un marcador de posición por defecto (codificado como 0). Este

diseño hace visibles las lagunas de datos tanto para el modelo como para el evaluador, evitando una situación en la que la red se entrena silenciosamente sobre ceros imputados sin ninguna indicación de que la medición subyacente no estaba disponible. Al pasar de CICIDS2017 a tráfico capturado en laboratorio, distintos pipelines de extracción pueden producir conjuntos de columnas diferentes; la máscara de presencia/ausencia garantiza que cualquier discrepancia de este tipo quede codificada en la observación en lugar de quedar oculta, de modo que el lector pueda distinguir un cero medido de uno imputado. Juntos, el esquema canónico y la máscara constituyen el contrato de observación del que depende cada componente posterior.

Criterio de verificación (OBJ-02). Todas las ejecuciones oficiales operan sobre el vector de 152 dimensiones (la configuración de la ejecución `MAIN` registra `n_features` igual a 152), y la semántica exacta de la máscara —presencia de la columna fuente, constante e igual a 1 sobre CICIDS2017 nativo— queda documentada en el capítulo de diseño del sistema.

2.3.3 Entorno de aprendizaje por refuerzo (OBJ-03)

El tercer objetivo específico es construir una interfaz conjunto-de-datos-como-entorno compatible con Gymnasium (Farama Foundation 2023) que presente los registros de flujo al agente como una secuencia de observaciones. El entorno envuelve un conjunto de datos etiquetado estático de modo que cada llamada a `step()` devuelve un registro de flujo como observación, junto con la recompensa calculada a partir de la acción anterior del agente y la etiqueta verdadera del último registro. Esta formulación permite que los bucles de entrenamiento de RL estándar, desarrollados para entornos interactivos, se apliquen directamente a conjuntos de datos de flujo offline sin modificar el algoritmo.

El entorno debe implementar una función de recompensa asimétrica y sensible al coste que penalice los falsos negativos más que los falsos positivos,

reflejando los perfiles de riesgo en ciberseguridad en los que los ataques no detectados son operacionalmente más perjudiciales que las falsas alarmas (Lee et al. 2002; Cárdenas et al. 2006). Los pesos de recompensa específicos se tratan como parámetros experimentales configurables en lugar de constantes universales fijas, ya que la relación de costes adecuada depende del modelo de amenaza y del contexto operativo del escenario de despliegue. El entorno debe también admitir reinicios reproducibles, gestión de límites de episodios y particiones del conjunto de datos configurables, de modo que la misma interfaz pueda utilizarse tanto para el entrenamiento como para la evaluación bajo distintos modos de partición.

Criterio de verificación (OBJ-03). El entorno implementado (`src/rl_defender_env.py`; Algoritmo 2) expone la interfaz Gymnasium con la matriz de recompensa asimétrica registrada en el artefacto de configuración de cada ejecución (Tabla 4.2) y con evaluación determinista sin barajado.

2.3.4 Agente de decisión binaria basado en QRDQN (OBJ-04)

El cuarto objetivo específico es configurar y entrenar un agente Quantile Regression Deep Q-Network (Dabney et al. 2018), aprovechando su enfoque distribucional para la estimación del valor, para navegar el espacio de acciones binario PERMIT/BLOCK a partir de las observaciones canónicas de 152 dimensiones. QRDQN extiende el DQN estándar estimando la distribución completa de los retornos esperados en lugar de solo la media, utilizando un conjunto de cuantiles aprendidos y una pérdida de regresión cuantílica. Esta representación distribucional resulta de interés en un contexto sensible al coste porque proporciona información más rica sobre la dispersión y la forma de los posibles resultados, no solo el retorno promedio, y en principio permite que la política sea más conservadora ante los peores casos.

La implementación práctica utiliza el módulo QRDQN de la biblioteca `sb3-contrib` (Stable-Baselines3 Contributors 2023), que proporciona una implementación estable y probada del algoritmo con arquitectura de red configurable, buffer de repetición, esquema de exploración e hiperparámetros de entrenamiento. El objetivo incluye seleccionar una configuración razonable por defecto, validar que el bucle de entrenamiento converge en el benchmark interno, y persistir todos los artefactos del modelo entrenado junto con la configuración y el estado de preprocesamiento utilizados para producirlos. La reproducibilidad exige que un modelo almacenado pueda cargarse y evaluarse de forma idéntica en una fecha posterior sin necesidad de reconstruir el pipeline de entrenamiento completo desde cero.

Criterio de verificación (OBJ-04). La ejecución oficial MAIN completó el entrenamiento con los hiperparámetros de la Tabla 4.4 y persiste modelo, escalador, configuración y métricas recuperables de forma independiente (Tabla 5.2).

2.3.5 Comparación con línea base supervisada (OBJ-05)

El quinto objetivo específico es implementar una línea base supervisada robusta, utilizando principalmente Random Forest (H. Liu et al. 2019), para ofrecer una evaluación comparativa justa, con el mismo protocolo, del valor empírico del agente QRDQN. Random Forest se elige como línea base principal porque está bien establecido para la clasificación tabular, gestiona interacciones no lineales entre características sin un preprocesamiento extenso, es robusto ante características irrelevantes y tiende a obtener buenos resultados en benchmarks de NIDS basados en flujos (Apruzzese et al. 2022). Utilizar el mismo esquema de características, el mismo protocolo de partición y las mismas métricas de evaluación tanto para el agente de RL como para la línea base es esencial; cualquier discrepancia en el preprocesamiento, la disponibilidad de características o las condiciones de evaluación comprometería la validez de la

comparación.

Esta comparación cumple varios propósitos. En primer lugar, determina si la formulación QRDQN ofrece alguna ventaja empírica sobre un clasificador supervisado bien ajustado en condiciones controladas. En segundo lugar, proporciona una comprobación de cordura sobre la dificultad de la tarea en el benchmark, ya que un Random Forest que alcanza puntuaciones muy altas contextualiza lo que QRDQN logra. En tercer lugar, fundamenta cualquier discusión sobre el diseño de la función de recompensa sensible al coste en evidencia concreta. Si ambos modelos se evalúan utilizando las mismas métricas ponderadas por coste, la comparación revela si la función de recompensa de RL conduce realmente al agente hacia un punto de operación diferente al de un modelo supervisado entrenado con los mismos datos, o si ambos enfoques convergen a un comportamiento similar.

Criterio de verificación (OBJ-05). La línea base Random Forest se entrenó y evaluó con la misma representación canónica, las mismas particiones y las mismas métricas que QRDQN, con ponderación de clases balanceada, y sus artefactos están comprometidos para las tres particiones consideradas (Tabla 5.2).

2.3.6 Evaluación con control de fugas (OBJ-06)

El sexto objetivo específico es diseñar pipelines de preprocesamiento y protocolos de validación que impidan estrictamente la fuga de información en todos los niveles de evaluación. Esto requiere la exclusión programática de campos identificativos, como las direcciones IP de origen y destino, las marcas temporales absolutas, los Flow ID y los proxies directos de puertos, en la fase de carga de datos antes de que comience cualquier entrenamiento de modelos (Arp et al. 2020; Kaufman et al. 2011). Estos campos se excluyen a nivel de esquema mediante el contrato canónico de características, en lugar de depender de un filtrado ad hoc que podría aplicarse de forma inconsistente en distintas

ejecuciones o versiones del conjunto de datos.

Las transformaciones de escalado deben ajustarse exclusivamente sobre la partición de entrenamiento y luego aplicarse a las particiones de validación y prueba utilizando únicamente los parámetros aprendidos de los datos de entrenamiento. Ajustar un escalador sobre el conjunto de datos completo antes de la partición permite que la información distribucional del conjunto de prueba influya en la normalización de las características de entrenamiento, una forma sutil de fuga de información que puede inflar el rendimiento reportado en conjuntos de datos con una estructura temporal o de escenario marcada. El escalador ajustado se persiste como parte del artefacto de la ejecución para que pueda recargarse y aplicarse de forma idéntica durante la inferencia de la Fase 2 sobre tráfico de laboratorio, garantizando que se utilice la misma normalización para flujos fuera de distribución sin que ninguna información sobre su distribución llegue al estado del escalador.

Criterio de verificación (OBJ-06). La exclusión de identificadores se aplica a nivel de esquema, el escalado se ajusta exclusivamente sobre entrenamiento, y las comprobaciones A (predicción directa), B (etiquetas barajadas) y C (partición por día), junto con el análisis de duplicados de la partición fija, están ejecutadas y comprometidas como artefactos (Tabla 5.2).

2.3.7 Análisis de escala de entrenamiento y eficiencia de datos (OBJ-07)

El séptimo objetivo específico es evaluar el rendimiento tanto del agente de RL como de las líneas base supervisadas bajo distintas cantidades de datos de entrenamiento, identificando las características de eficiencia de datos de la formulación propuesta y evaluando con qué rapidez converge QRDQN en comparación con las alternativas supervisadas. Los métodos de RL profundo pueden ser muy demandantes en cuanto a muestras, y su dinámica de aprendizaje difiere sustancialmente de la de los métodos de árboles de decisión en

conjunto. Comprender cuánto tráfico etiquetado requiere cada enfoque para alcanzar un rendimiento útil es prácticamente relevante: el tráfico de ataque etiquetado es costoso de recopilar, y los escenarios de despliegue reales pueden no tener acceso a los grandes conjuntos de datos equilibrados disponibles en los benchmarks académicos (Ring et al. 2019).

Los experimentos de escala de entrenamiento utilizan subconjuntos de entrenamiento progresivamente más grandes extraídos de la misma partición, midiendo cómo cambian la precisión, el recall, el F1, la tasa de falsos positivos y la tasa de falsos negativos a medida que crece el presupuesto. La comparación entre QRDQN y Random Forest en cada nivel de presupuesto revela si la formulación de RL tiene un requisito mayor de muestras, si converge más lentamente, o si alcanza una meseta de rendimiento final diferente. Estos resultados se interpretan como evidencia interna sobre esta combinación específica de formulación y benchmark, no como afirmaciones generales sobre la eficiencia de datos de RL en todos los contextos de NIDS.

Criterio de verificación (OBJ-07). El protocolo y su instrumentación están diseñados e implementados (presupuestos anidados mediante límites de filas de entrenamiento); el objetivo se considerará alcanzado cuando los artefactos de la curva de aprendizaje estén comprometidos en el repositorio. Su ejecución está pendiente de disponibilidad de GPU y esta memoria no reporta ninguna cifra de escala de entrenamiento.

2.3.8 Validación complementaria con tráfico de laboratorio (OBJ-08)

El octavo objetivo específico es construir un pipeline de inferencia offline robusto capaz de ingerir tráfico capturado por el operador en un entorno de laboratorio doméstico cerrado y no adversarial, y producir predicciones a partir de un modelo entrenado en CICIDS2017. Este nivel de validación ofrece una indicación parcial, de validez externa limitada, de si el esquema canónico de

características y el modelo entrenado se comportan de forma razonable sobre flujos extraídos de un entorno de red distinto del benchmark, potencialmente utilizando una versión distinta de CICFlowMeter o una configuración de captura diferente. El cambio de dominio entre la distribución de entrenamiento y la distribución de inferencia es un reto fundamental para los modelos de NIDS y rara vez se evalúa en estudios que solo evalúan sobre un único conjunto de datos público (Apruzzese et al. 2022; Layeghy et al. 2023).

El pipeline de inferencia aplica recorte opcional por percentiles y z-score para gestionar los valores de características brutas fuera de distribución que caen fuera del rango observado durante el entrenamiento en CICIDS2017. El escalador ajustado de la Fase 1 se carga y aplica sin reajuste, preservando la integridad de la separación entre entrenamiento y prueba. La máscara de presencia/ausencia en el vector de observación codifica explícitamente cualquier característica que no haya podido calcularse a partir de la captura de laboratorio, haciendo visible el caso de datos incompletos en la entrada de predicción en lugar de imputarlo silenciosamente. El resultado esperado de este objetivo es un conjunto de artefactos de predicción y métricas que permitan comparar directamente los resultados del tráfico de laboratorio con los de la validación de la Fase 1, proporcionando evidencia concreta sobre cómo varía el rendimiento bajo un cambio de dominio.

Criterio de verificación (OBJ-08). La ejecución oficial de la Fase 2 (P2v2 sobre el modelo MAIN) procesó la captura de laboratorio con el escalador persistido y produjo predicciones, métricas y diagnósticos comprometidos (Tabla 5.2; Algoritmo 4), manteniendo el marco declarado de validez externa limitada.

La Tabla 2.5 resume la trazabilidad entre los objetivos específicos, los capítulos donde se desarrollan y la evidencia que respalda el estado de cada uno; los identificadores completos de las ejecuciones citadas figuran en la Tabla 5.2. El único objetivo sin evidencia de ejecución es OBJ-07, cuyo protocolo está diseñado e implementado pero pendiente de disponibilidad de GPU.

Objetivo	Capítulos	Evidencia (ejecución o artefacto)	Estado
OBJ-01	4	Contrato de características en código (<code>canonical_schema.py</code>); Algoritmo 1	Completado
OBJ-02	4	Configuración de MAIN (<code>n_features = 152</code>); semántica de la máscara documentada	Completado
OBJ-03	4	<code>rl_defender_env.py</code> ; Algoritmo 2; Tablas 4.2 y 4.3	Completado
OBJ-04	4 y 6	Ejecución MAIN; Tabla 4.4	Completado
OBJ-05	5 y 6	Barrido Random Forest sobre las tres particiones	Completado
OBJ-06	5 y 6	Comprobaciones A/B/C; análisis de duplicados; partición fija con semilla 42	Completado (<i>leave-one-CSV-out</i> de QRDQN pendiente de GPU)
OBJ-07	5	Protocolo e instrumentación implementados; sin artefactos de ejecución	Pendiente de GPU
OBJ-08	5 y 6	Ejecución P2v2 sobre el modelo MAIN (Fase 2)	Completado

Tabla 2.5: Matriz de trazabilidad entre objetivos específicos, capítulos, evidencia y estado. Los RUN_ID completos figuran en la Tabla 5.2. Fuente: elaboración propia.

2.4 Alcance del trabajo

El trabajo está delimitado por los requisitos de una evaluación offline de un pipeline de aprendizaje automático tabular. Comprende la ingesta de registros de flujo en formato CSV tanto de CICIDS2017 como de capturas de laboratorio, la aplicación de escalado estándar y recorte opcional de valores atípicos, la ejecución de bucles de entrenamiento mediante el framework Stable Baselines3 (`sb3-contrib`), y la generación de artefactos de ejecución estructurados y reproducibles que persisten modelos, métricas, escaladores y ficheros de configuración junto a cada ejecución de entrenamiento. La experimentación interna se realiza íntegramente sobre el conjunto de datos CICIDS2017, utilizando particiones explícitas entre entrenamiento y prueba y una escalera de validación multietapa para evaluar el comportamiento del modelo bajo condiciones progresivamente más estrictas antes de probarlo con tráfico capturado externamente.

El alcance incluye explícitamente la escalera de validación multietapa descrita en los objetivos de evaluación. Esta escalera cubre particiones aleatorias estratificadas para comprobaciones básicas de aprendizaje, una comprobación de etiquetas barajadas para verificar que el rendimiento colapsa cuando las etiquetas no contienen señal real, una partición por día o por CSV para comprobar la generalización dentro de CICIDS2017, una evaluación *leave-one-CSV-out* para comprobar la sensibilidad a ficheros de captura específicos, y la inferencia offline sobre flujos capturados por el operador en un laboratorio doméstico cerrado. Cada nivel aporta evidencia diferenciada y forma parte del alcance experimental acordado.

2.5 No objetivos

Esta tesis excluye explícitamente el desarrollo de monitorización de red en línea o la interceptación de paquetes en tiempo real. El sistema no se integrará con cortafuegos, reglas de `iptables`, controladores de Software-Defined Networking (SDN), ni sistemas de prevención de intrusiones (IPS) activos. Estas exclusiones existen porque el despliegue en producción requeriría un diseño de sistema sustancialmente diferente, una validación de seguridad y una metodología de pruebas operacionales que quedan fuera del alcance de un estudio académico controlado. La tesis está diseñada como un prototipo experimental, no como un producto operativo destinado a su uso en producción.

La tesis tampoco pretende resolver el problema más amplio de las operaciones cibernéticas adversariales multiagente o la interrupción de ataques con dependencia de secuencias, donde el estado del entorno cambia dinámicamente en respuesta a las acciones del defensor. En la formulación actual, cada registro de flujo se clasifica de forma independiente, y el conjunto de datos no reacciona a las decisiones del agente. Esta es una limitación conocida del enfoque *conjunto-de-datos-como-entorno*, e implica que el sistema no puede modelar la

adaptación adversarial, la persistencia del atacante ni las cadenas de ataques multietapa que abarcan muchas decisiones individuales de flujo. Por último, el objetivo no es demostrar que QRDQN represente el estado del arte definitivo para todas las tareas de NIDS, sino evaluar su comportamiento bajo un marco experimental cuidadosamente controlado, libre de fugas y reproducible, y compararlo honestamente con líneas base supervisadas.

2.6 Resultados esperados

El resultado esperado de esta tesis es un pipeline de inferencia offline completamente funcional, documentado y reproducible, respaldado por un análisis empírico detallado. El proyecto producirá artefactos persistentes para todas las ejecuciones de entrenamiento y validación, incluyendo ficheros de modelo entrenado, escaladores ajustados, informes de métricas, salidas de validación y registros de configuración que especifican completamente las condiciones bajo las cuales se produjo cada resultado. Esta disciplina en la gestión de artefactos forma parte de la contribución metodológica: un resultado que no puede reproducirse a partir de su rastro de artefactos proporciona una evidencia más débil que uno que sí puede.

En cuanto al contenido empírico, el proyecto producirá mediciones de precisión, recall, F1, tasa de falsos positivos y tasa de falsos negativos tanto para el agente QRDQN como para la línea base de Random Forest en todos los niveles de validación. Los experimentos de escala de entrenamiento producirán curvas de aprendizaje que caracterizarán cómo evoluciona el rendimiento de cada modelo a medida que crece el presupuesto de entrenamiento. Los resultados de inferencia de la Fase 2 caracterizarán cómo varía el rendimiento cuando el modelo se enfrenta a tráfico capturado por el operador en un laboratorio doméstico cerrado y no adversarial, en lugar de flujos de CICIDS2017 reservados para la prueba. En conjunto, estos resultados cuantificarán los límites de

generalización de los modelos de RL entrenados en benchmarks cuando se someten a particiones temporales estrictas, evaluación *leave-one-CSV-out* y tráfico de laboratorio doméstico generado por el operador, y proporcionarán una evaluación empírica honesta de si la formulación de RL sensible al coste ofrece ventajas medibles sobre una línea base supervisada bien equiparable.

2.7 Restricciones

El desarrollo del proyecto está condicionado por un conjunto de restricciones que delimitan qué evidencia puede producirse y cómo debe interpretarse. La Tabla 2.6 las cataloga en dos categorías: *factores de dato*, impuestos por las fuentes de datos disponibles, y *factores estratégicos*, decisiones de alcance adoptadas para mantener el proyecto abordable, reproducible y seguro. Ninguna de estas restricciones se oculta en los capítulos posteriores: cada una reaparece, cuantificada cuando es posible, en los resultados y en las limitaciones.

2.8 Recursos

Para el desarrollo y la ejecución del proyecto se han empleado dos entornos de cómputo diferenciados. El entorno remoto de RunPod produjo la ejecución principal MAIN sobre CICIDS2017; el ordenador portátil local se utilizó únicamente para desarrollar el código, ejecutar pruebas y construir la memoria, no para entrenar modelos. La Tabla 2.7 resume los recursos y mantiene separadas la confirmación del autor, la evidencia registrada por la ejecución y los contratos de instalación. El inventario completo se presenta en el Anexo B.

El entrenamiento principal (MAIN_qrdqn_cicids2017_canonical_full_random_20260609_193655) se ejecutó íntegramente sobre el dispositivo cuda del entorno remoto. El portátil local quedó fuera del entrenamiento de modelos;

ID	Restricción	Implicación para el proyecto
<i>Factores de dato</i>		
RES-D1	CICIDS2017 presenta problemas documentados: flujos duplicados, inconsistencias de etiquetado y sensibilidad a la partición.	Los resultados <i>in-distribution</i> se leen como cota optimista; la escalera de validación y el análisis de duplicados son obligatorios.
RES-D2	El mapeo binario colapsa todas las familias de ataque en una única clase BLOCK.	El análisis de errores por familia queda fuera de la ruta principal de RL y exigiría etiquetas de familia preservadas.
RES-D3	El tráfico de la Fase 2 procede de un único laboratorio doméstico cerrado, generado por el operador y no adversarial.	Validez externa limitada: la Fase 2 se interpreta como comprobación de transferencia, no como validación de producción.
RES-D4	No se dispone de tráfico de producción etiquetado de forma independiente.	Las afirmaciones de generalización se restringen al benchmark y al laboratorio.
<i>Factores estratégicos</i>		
RES-E1	Evaluación exclusivamente offline; sin bloqueo en vivo ni integración con cortafuegos, SDN o IPS.	El sistema es un instrumento de medición experimental; un despliegue exigiría diseño y validación adicionales.
RES-E2	Presupuesto de cómputo GPU puntual (instancia en la nube reservada para el entrenamiento principal).	Los experimentos de escala de entrenamiento, el <i>leave-one-CSV-out</i> de QRDQN y el estudio multi-semilla quedan diferidos, pendientes de GPU.
RES-E3	Una única semilla en la ejecución principal.	Los intervalos bootstrap cuantifican precisión de muestreo, no varianza entre entrenamientos.
RES-E4	Un único algoritmo de RL (QRDQN) y una única línea base supervisada (Random Forest).	Las conclusiones algorítmicas se limitan a esta pareja bajo el protocolo común.

Tabla 2.6: Catálogo de restricciones del proyecto: factores de dato y factores estratégicos. Fuente: elaboración propia.

Recurso	Especificación versión	/	Uso en el proyecto	Procedencia
<i>Hardware</i>				
CPU y memoria remotas	AMD EPYC 9005; 128 GB de RAM		Entorno RunPod que produjo el entrenamiento MAIN	Confirmación del autor
GPU remota	NVIDIA GeForce RTX 3090 Ti; 24 GB de VRAM		Entrenamiento principal QRDQN (3 000 000 de pasos) sobre CICIDS2017	Confirmación del autor; <code>environment.json</code> corrobora el modelo
Ordenador portátil local	No caracterizado como entorno de entrenamiento		Desarrollo, pruebas y construcción de la memoria; ningún entrenamiento de modelos	Uso confirmado por el autor
<i>Software</i>				
Runtime de MAIN	Linux 6.8.0-110-generic, glibc 2.39; Python 3.12.11; CUDA 13.0; cuDNN 9.20.0		Entorno observado durante entrenamiento y evaluación de la ejecución principal	<code>environment.json</code> de MAIN
Bibliotecas de MAIN	<code>torch</code> 2.12.1+cu130; NumPy 2.4.6; pandas 3.0.3; <code>scikit-learn</code> 1.9.0; <code>Gymnasium</code> 1.2.3; SB3 y sb3-contrib 2.8.0; <code>joblib</code> 1.5.3		Pipeline de datos, entorno RL, entrenamiento y persistencia	<code>environment.json</code> de MAIN
Contratos de instalación	<code>requirements.txt</code> , <code>requirements-runpod-cu130.txt</code> y <code>pyproject.toml</code>		Reproducción local, GPU, desarrollo y ajuste de hiperparámetros	Manifiestos del repositorio
Construcción de la memoria	Windows; TeX Live 2025; <code>latexmk</code> 4.87; <code>biber</code> 2.21		Producción del documento; fuera del runtime del modelo	Tracker de mejora de la memoria

Tabla 2.7: Recursos efectivamente utilizados y procedencia de cada dato. Las versiones observadas proceden de `environment.json`; los manifiestos expresan contratos de instalación, no observaciones de ejecución.

su función se limitó al desarrollo, las pruebas y la construcción documental.

2.9 Planificación temporal

La Tabla 2.8 recoge los hitos contractuales y las fases efectivas del proyecto, y la Figura 2.1 las representa como diagrama de Gantt. Las fechas están ancladas a dos fuentes verificables: el calendario contractual del TFG (firma, reunión de inicio y entregas parciales) y la evidencia del propio repositorio (marcas temporales de los artefactos de ejecución y del historial de desarrollo). No se planifica hacia el futuro: el cronograma documenta lo efectivamente ocurrido.

Hito o fase	Fecha(s)
Firma del contrato	11 de noviembre de 2025
Reunión de inicio	18 de noviembre de 2025
Entregas parciales 1–5	15-dic-2025, 2-feb, 17-mar, 20-may y 15-jun de 2026
Desarrollo e implementación	11 de diciembre de 2025 – 9 de junio de 2026
Redacción de la memoria	12 de diciembre de 2025 – 6 de julio de 2026
Experimentos preliminares A/B/C	12 – 23 de febrero de 2026
Fase 2: captura e inferencia en laboratorio	23 de febrero – 10 de junio de 2026
Entrenamiento del modelo MAIN	9 de junio de 2026
Auditoría de validación y línea base RF	27 – 28 de junio de 2026

Tabla 2.8: Hitos contractuales y fases efectivas del proyecto. Fuente: elaboración propia a partir del calendario contractual del TFG y de las marcas temporales de los artefactos del repositorio.

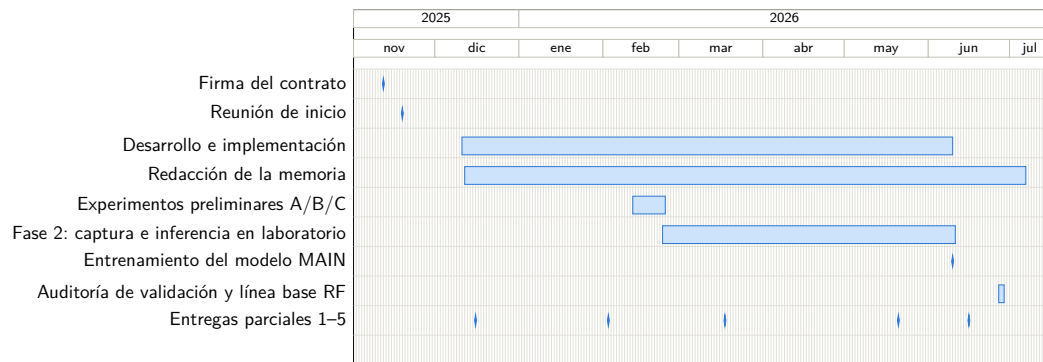


Figura 2.1: Planificación temporal del proyecto como diagrama de Gantt: hitos contractuales, desarrollo, redacción, experimentos y fases de validación. Fuente: elaboración propia a partir del calendario contractual del TFG y de las marcas temporales de los artefactos del repositorio.

Estado del arte

3.1 Introducción y alcance del capítulo

Este capítulo mapea el trasfondo técnico de un defensor de ciberseguridad basado en aprendizaje por refuerzo que opera sobre observaciones de red a nivel de flujo. El alcance es deliberadamente más amplio que una comparación de clasificadores binarios. La tesis combina detección de intrusiones en red, representación de características de flujo, conjuntos de datos de referencia públicos, líneas base de aprendizaje supervisado, una formulación de conjunto de datos como entorno al estilo Gymnasium, QRDQN, preprocesamiento con control de fugas, comprobaciones de validación más estrictas, experimentos de eficiencia de datos, y validación planificada o preferida sobre tráfico capturado por el operador en un laboratorio doméstico cerrado y no adversarial.

El capítulo sigue por tanto un embudo. Primero revisa la detección de intrusiones en red y la representación del tráfico, luego sitúa a CICIDS2017 entre los conjuntos de datos públicos, después discute las líneas base de aprendizaje supervisado y aprendizaje profundo, y finalmente enmarca la formulación de RL tanto dentro de la teoría del aprendizaje por refuerzo como en trabajos previos de RL/DRL para detección de intrusiones y defensa cibernética. Las últimas

secciones se centran en los riesgos metodológicos, los protocolos de evaluación, la evidencia sobre escala de entrenamiento y la brecha de investigación que aborda esta tesis.

La afirmación central es cautelosa. El aprendizaje por refuerzo para la detección de intrusiones ya existe, y los benchmarks públicos de NIDS son ampliamente utilizados (W. Yang et al. 2026; Kheddar et al. 2025; Gueriani et al. 2024). Esta tesis no pretende introducir el primer IDS basado en RL, demostrar que QRDQN es el estado del arte para NIDS, ni demostrar una prevención en línea lista para producción. Estudia un pipeline de decisión reproducible, offline, a nivel de flujo, con acciones PERMIT/BLOCK, y evalúa cómo se comporta dicha formulación bajo benchmarks internos controlados, comparación con líneas base y validación progresivamente más estricta.

La revisión también separa tres niveles que con frecuencia se confunden. El primero es la monitorización: recopilar paquetes, flujos, registros o eventos derivados y decidir si son sospechosos. El segundo es la predicción: entrenar un modelo que asigne representaciones de tráfico a etiquetas benignas o maliciosas. El tercero es el control: tomar acciones que modifiquen el estado futuro de la red. Esta tesis trabaja principalmente en los dos primeros niveles. Utiliza el lenguaje de PERMIT y BLOCK porque el entorno expone una decisión de seguridad binaria, pero la implementación actual no es un plano de control en línea. Esa distinción condiciona todas las afirmaciones posteriores.

3.2 Detección de intrusiones y monitorización de seguridad en red

Los sistemas de detección de intrusiones monitorizan la actividad de hosts o redes en busca de evidencias de ataques, usos indebidos o violaciones de políticas (Scarfone et al. 2007). Un sistema de detección de intrusiones en host

observa endpoints, eventos del sistema operativo, registros, procesos, cambios en la integridad de archivos o llamadas al sistema. Un sistema de detección de intrusiones en red observa el tráfico que atraviesa enlaces, taps, puertos de mirror de switches, brokers de paquetes, interfaces de red virtual o colectores de flujos. Ambas perspectivas son complementarias: el HIDS puede ver el contexto local de procesos y archivos que los sensores de red no pueden, mientras que el NIDS puede inspeccionar movimientos laterales, escaneos, tráfico de mando y control, y patrones de comunicación agregados sin instalar un agente en cada host.

También es necesario distinguir el IDS de los sistemas de prevención de intrusiones. Un IDS detecta y notifica principalmente actividad sospechosa. Su salida puede ser una alerta, un evento de registro, un registro de flujo enriquecido, un ticket, un evento SIEM o un artefacto forense. Un IPS se posiciona en línea y puede bloquear, descartar, modificar, limitar la tasa, reiniciar o redirigir el tráfico (Scarfone et al. 2007). En la práctica, los productos y arquitecturas frecuentemente difuminan esta frontera porque un motor de detección puede alimentar reglas de firewall, acciones EDR, playbooks SOAR o actualizaciones de política SDN. Para la evaluación, sin embargo, la distinción importa. Un detector puede tolerar un perfil de falsos positivos diferente al de un bloqueador en línea, dado que las alertas falsas principalmente afectan a la carga de trabajo del analista, mientras que los bloqueos falsos pueden interrumpir tráfico legítimo de negocio.

Las taxonomías clásicas de IDS distinguen enfoques basados en firmas, basados en anomalías, basados en especificaciones e híbridos (Scarfone et al. 2007). La detección basada en firmas coteja patrones maliciosos conocidos: secuencias de bytes, predicados de reglas, condiciones de protocolo, cadenas de comandos, rastros de exploits o combinaciones de comportamiento conocido. Su fortaleza es la precisión y la explicabilidad ante amenazas conocidas. Su debilidad es la dependencia del mantenimiento de reglas y la escasa cobertura de ataques nuevos, comportamiento polimórfico, cargas útiles cifradas y variaciones

específicas del entorno.

La detección basada en anomalías modela el comportamiento esperado y señala las desviaciones (Sommer et al. 2010; Ring et al. 2019). Esta es la familia más naturalmente asociada con el NIDS basado en ML, pero la palabra anomalía no debe interpretarse como sinónimo automático de aprendizaje no supervisado. Muchos estudios públicos de NIDS son clasificadores supervisados entrenados sobre flujos etiquetados como benignos y de ataque, incluso cuando la motivación más amplia es la detección de anomalías. El reto operativo es que el tráfico benigno es amplio y no estacionario, la prevalencia de ataques es baja, las etiquetas son incompletas, y los cambios en la actividad de negocio pueden parecer anómalos sin ser maliciosos.

La detección basada en especificaciones codifica el comportamiento permitido o esperado, por ejemplo máquinas de estado de protocolo, secuencias de comandos permitidas o flujos de trabajo de control industrial restringidos. Puede ser potente cuando el sistema monitorizado tiene un comportamiento estrecho y estable, pero requiere conocimiento del dominio y mantenimiento. Los sistemas híbridos combinan estas ideas, como reglas de firmas para exploits conocidos, puntuaciones estadísticas de anomalías para comportamiento de flujo inusual, y telemetría de host para confirmación en el endpoint. Los programas modernos de monitorización raramente dependen de un único paradigma de detección.

La generación de alertas y la respuesta activa forman un eje separado. Un sistema puede limitarse a producir alertas para su posterior clasificación, puede enriquecer las alertas con evidencia contextual, puede recomendar pasos de respuesta o puede desencadenar contención activa. Esta tesis se sitúa deliberadamente en el extremo conservador de ese eje. Estudia un modelo de decisión offline similar a un NIDS sobre filas de flujo extraídas. El modelo emite predicciones PERMIT/BLOCK durante los experimentos, pero no descarta paquetes, no reescribe políticas de firewall, no actualiza reglas SDN, no pone en cuarentena hosts ni opera en un bucle de retroalimentación en vivo. La

bibliografía de NIDS es, por tanto, relevante, pero las afirmaciones sobre IPS en producción quedan fuera del alcance soportado.

3.3 Representación del tráfico de red

El tráfico de red puede representarse como paquetes, bytes de carga útil, sesiones, conexiones o flujos. Las representaciones a nivel de paquete preservan la temporización, las cabeceras, los tamaños, las direcciones, los flags y, en ocasiones, parte de la carga útil por paquete. Son adecuadas para el análisis de protocolos a grano fino y el modelado de secuencias, pero resultan costosas de almacenar y procesar en redes de alta velocidad. Las representaciones a nivel de carga útil retienen el contenido de la aplicación y pueden identificar exploits o firmas de malware invisibles en los metadatos. Su utilidad está limitada por el cifrado, las restricciones de privacidad, las restricciones legales y el coste de CPU.

Las representaciones a nivel de sesión o conexión agrupan paquetes pertenecientes al mismo intercambio, habitualmente utilizando la quintupla de IP de origen, IP de destino, puerto de origen, puerto de destino y protocolo. Pueden incluir el estado de la conexión, el comportamiento del handshake, metadatos de la aplicación o atributos derivados de TLS/HTTP. Las representaciones a nivel de flujo son más compactas: agregan paquetes en una ventana temporal o registro similar a una conexión, y resumen el comportamiento mediante duración, bytes, paquetes, direccionalidad, tiempos entre llegadas, estadísticas de longitud de paquetes, conteos de flags TCP y periodos activos o inactivos (Sperotto et al. 2010; Umer et al. 2017).

Los registros de estilo NetFlow e IPFIX son el punto de referencia operativo para la monitorización de flujos escalable. Routers, switches, sondas y colectores exportan registros de flujo compactos que pueden retenerse durante largos periodos y procesarse a escala. Estos registros son atractivos para el NIDS

porque reducen el volumen y evitan la inspección de la carga útil. También tienen limitaciones porque los despliegues exportan a menudo solo un subconjunto de los campos posibles, pueden muestrear el tráfico y pueden omitir estadísticas más ricas utilizadas en conjuntos de datos académicos. Esto crea una brecha entre la telemetría de flujos en producción y los CSV de benchmark con abundantes características.

Las características al estilo CICFlowMeter se sitúan en el lado más rico y orientado a la investigación de esa brecha. CICIDS2017 proporciona PCAPs y CSV de flujos bidireccionales generados, con características como la duración del flujo, conteos de paquetes y bytes en sentido directo e inverso, estadísticas de tiempos entre llegadas, estadísticas de longitud de paquetes, conteos de flags TCP, agregados de longitud de cabecera, tasas de flujo y temporización activa/inactiva (Sharafaldin et al. 2018; Habibi Lashkari et al. 2017; Canadian Institute for Cybersecurity 2017). Estas características son convenientes para ML tabular porque cada flujo se convierte en una fila de anchura fija, pero dependen de la reconstrucción y el etiquetado offline de PCAPs, así como de las decisiones de extracción de características (Farrukh et al. 2022).

La idoneidad de la representación a nivel de flujo es, por tanto, condicional. Es adecuada para esta tesis porque el defensor implementado consume vectores numéricos de anchura fija, porque CICIDS2017 ya está disponible en formato de flujo, y porque la ruta prevista en Fase 2 también extrae CSV de flujos antes de la inferencia. Está limitada porque se pierden la semántica de la carga útil, el ordenamiento exacto de paquetes, el contenido de la aplicación y algunas dependencias temporales de múltiples flujos. Los ataques cuya evidencia es principalmente a nivel de contenido, a nivel de host o de comportamiento a largo plazo pueden ser solo indirectamente visibles para un clasificador de flujos (Sperotto et al. 2010; Umer et al. 2017).

La disponibilidad de características es otra limitación. Una característica calculada por CICFlowMeter a partir de PCAPs completos puede no estar disponible en un exportador NetFlow/IPFIX en producción, o puede calcularse

de manera diferente. Sarhan et al. abordan este problema mapeando conjuntos heterogéneos de características de NIDS hacia una representación estándar alineada con NetFlow para mejorar la generalizabilidad y la explicabilidad (Sarhan et al. 2021). Esta tesis no adopta el conjunto exacto de características estándar de Sarhan, pero sigue la misma motivación metodológica: definir un contrato de características estable, documentar qué está presente o ausente, y evitar tratar un esquema CSV específico de un benchmark como si fuera universal.

El proyecto actual utiliza un esquema canónico de flujo con 76 valores de características y una máscara de presencia/ausencia de 76 valores, produciendo observaciones de 152 dimensiones. La máscara registra si cada valor canónico estaba presente y era válido, en lugar de ocultar silenciosamente los campos ausentes o imputados. Esto importa al pasar de CICIDS2017 a tráfico de laboratorio porque diferentes pipelines de extracción pueden no producir exactamente las mismas columnas. La máscara no hace que los datos faltantes sean inofensivos, y no resuelve el cambio de dominio. Hace auditable la entrada del modelo: un lector puede distinguir un cero medido de un marcador de posición imputado.

La representación a nivel de flujo también modifica el perfil de privacidad y de fugas. La eliminación de las cargas útiles puede reducir la exposición de privacidad, pero los metadatos aún pueden identificar usuarios, servicios, horarios y escenarios. Las direcciones IP, las marcas temporales absolutas, los Flow ID, la identidad de los endpoints, la estructura del día de captura y los puertos usados como proxies de escenario pueden filtrar etiquetas en lugar de representar comportamiento de ataque (Arp et al. 2020; Kaufman et al. 2011). La tesis trata por tanto la representación y la selección de características como decisiones de diseño relevantes para la seguridad, no solo como detalles de preprocesamiento.

3.4 Conjuntos de datos públicos para la detección de intrusiones en red

Los conjuntos de datos públicos de NIDS permiten la experimentación compartida, la reproducibilidad y la comparación entre trabajos (Ring et al. 2019; Thakkar et al. 2020; Goldschmidt et al. 2025). También condicionan las conclusiones. Un conjunto de datos refleja su generador de tráfico, su topología, su proceso de captura, su extractor de características, sus etiquetas, sus scripts de ataque y su formato de distribución. El rendimiento sobre un benchmark público es, por tanto, evidencia sobre una tarea controlada, no evidencia directa de preparación para el despliegue (Apruzzese et al. 2022; Layeghy et al. 2023).

KDDCup99 y NSL-KDD son históricamente importantes porque dieron forma a la evaluación temprana de IDS (KDD Cup 1999; Tavallae et al. 2009). KDDCup99 contiene registros de conexión derivados del tráfico de la era DARPA, con un esquema de 41 características y categorías de ataque como DoS, Probe, R2L y U2R. NSL-KDD redujo algunos problemas de redundancia y facilitó el uso del benchmark, pero mantuvo el mismo origen básico y lógica de características. Estos conjuntos son útiles para comprender la historia de la evaluación de IDS, pero evidencia débil para la defensa moderna basada en flujos: su tráfico, comportamiento de ataques y esquema son antiguos (Tavallae et al. 2009; Ring et al. 2019; Özgür et al. 2016). En este repositorio, NSL-KDD es contexto histórico, no la ruta final del defensor actual.

UNSW-NB15 es un conjunto de datos de cyber-range más reciente, generado con tráfico benigno y malicioso contemporáneo, capturas sin procesar y características de ingeniería (Moustafa y Slay 2015; Moustafa y Slay 2016). Sigue siendo útil porque amplió el campo más allá de los benchmarks al estilo KDD e incluye categorías de ataque más ricas. CICIDS2017 y CSE-CIC-IDS2018 son conjuntos de datos de la familia CIC ampliamente utilizados para la investigación de NIDS basada en flujos; ambos proporcionan escenarios de ataque

generados en laboratorio y características de flujo derivadas de capturas de paquetes (Sharafaldin et al. 2018; Communications Security Establishment et al. 2018). Bot-IoT y ToN_IoT amplían el panorama hacia IoT e IIoT, donde el comportamiento de los dispositivos, la telemetría y los patrones de ataque difieren de las redes empresariales (Koroniotis et al. 2019; Moustafa, Ahmed et al. 2020; Alsaedi et al. 2020).

Conjunto de datos	Papel como benchmark	Representación	Principales fortalezas	Principales advertencias
KDDCup99	Benchmark histórico heredado	Registros de conexión con 41 características artesanales	Muy ampliamente utilizado; línea base sencilla para la bibliografía clásica de IDS (KDD Cup 1999)	Tráfico obsoleto; redundancia; débil como aproximación al NIDS moderno (Ring et al. 2019)
NSL-KDD	Benchmark heredado depurado	Mismo esquema de conexión al estilo KDD	Reduce algunos problemas de registros duplicados; útil para comparación histórica (Tavallaei et al. 2009)	Sigue heredando las suposiciones antiguas de DARPA/KDD y el diseño de características
UNSW-NB15	Conjunto de datos moderno de laboratorio/cyber range	PCAPs, registros y bulares de ingeniería	Taxonomía de ataques más contemporánea y características más ricas (Moustafa y Slay 2015)	Entorno controlado; no constituye evidencia directa de producción
CICIDS2017	Benchmark interno principal de la tesis	PCAPs y CSV de flujos CICFlowMeter	PCAPs públicos; múltiples días de ataque; ricas características de flujo (Sharafaldin et al. 2018)	Problemas conocidos de etiquetado, extracción, duplicación y sensibilidad a la partición (Engelen et al. 2021; Lanvin et al. 2023)
CSE-CIC-IDS2018	Benchmark de mayor tamaño de la familia CIC	PCAPs y flujos al estilo CICFlowMeter	Colaboración más amplia CIC/CSE; comparación moderna útil (Communications Security Establishment et al. 2018)	Advertencias similares sobre generación en laboratorio y artefactos de flujo (L. Liu et al. 2022)
Bot-IoT	Benchmark de IoT/botnets	PCAPs, flujos al estilo Argus, exportaciones CSV	Escenarios de botnet, escaneo, DoS/DDoS y exfiltración orientados a IoT (Koroniotis et al. 2019)	Fuerte desbalanceo y artefactos de topología específicos del conjunto de datos
ToN_IoT	Benchmark de telemetría IoT/IIoT	Flujos de red, registros de host y telemetría de sensores	Perspectiva multimodal de IoT/IIoT (Alsaedi et al. 2020)	Entorno de nicho; los artefactos de identificadores y escenarios siguen siendo posibles

Tabla 3.1: Comparación compacta de los conjuntos de datos públicos de NIDS relevantes para la tesis.

La Tabla 3.1 no establece un rango. Muestra que la elección del conjunto de datos debe alinearse con la solidez de las afirmaciones. Los benchmarks

históricos sitúan el campo; los conjuntos de datos modernos de laboratorio permiten comparación controlada; los de IoT prueban suposiciones de tráfico diferentes; y el tráfico de laboratorio externo o similar al de producción aporta evidencia de generalización más sólida. Ninguno de estos conjuntos de datos debe tratarse como un sustituto completo de todas las redes reales. Su valor es máximo con preprocesamiento transparente, líneas base sólidas y protocolos que prueben el cambio de distribución, no solo el rendimiento sobre distribuciones aleatorias emparejadas.

3.5 CICIDS2017 como benchmark interno principal

CICIDS2017 es el benchmark interno principal de esta tesis porque proporciona tráfico benigno y de ataque etiquetado, PCAPs sin procesar y CSV de flujos generados con características al estilo CICFlowMeter (Sharafaldin et al. 2018; Habibi Lashkari et al. 2017; Canadian Institute for Cybersecurity 2017). Fue diseñado para mejorar conjuntos de datos de IDS anteriores mediante perfiles de tráfico contemporáneos, actividad benigna realista y múltiples escenarios de ataque (Sharafaldin et al. 2018; Sharafaldin et al. 2019). Estas propiedades lo hacen adecuado para la experimentación controlada con modelos de NIDS a nivel de flujo.

Su estructura también es atractiva para la tesis. El conjunto abarca múltiples días de captura y familias de ataque, y está disponible tanto como capturas de paquetes sin procesar como en CSV generados (Oyelakin et al. 2023). Los PCAPs están más cerca de la evidencia de medición original. Los CSV facilitan el ML porque CICFlowMeter ya reconstruyó los flujos bidireccionales y calculó características estadísticas. Esa comodidad también es un riesgo: al trabajar solo con CSV resumidos, se puede dejar de comprobar cómo se generaron los

flujos, cómo se asignaron las etiquetas y si las filas mantienen la independencia entre las particiones de entrenamiento y prueba.

CICFlowMeter reconstruye flujos de trazas de paquetes mediante claves de endpoint, direccionalidad, lógica de timeout y cálculos de características. Pequeñas diferencias en los parámetros de extracción y en la integridad de los flujos pueden cambiar filas, duraciones, conteos y etiquetas (Pekar et al. 2024). Engelen et al. muestran problemas prácticos en la reconstrucción de flujos y la consistencia de CICIDS2017 (Engelen et al. 2021). Lanvin et al. reportan errores que afectan al rendimiento de detección y que correcciones aparentemente pequeñas pueden producir diferencias métricas significativas (Lanvin et al. 2023; Lanvin et al. 2022). Liu et al. analizan la prevalencia de errores en CIC-IDS-2017 y CSE-CIC-IDS-2018, mientras Rosay et al. ofrecen un análisis más amplio y una discusión orientada a la reconstrucción (L. Liu et al. 2022; Rosay et al. 2022). La crítica de Dube es especialmente relevante para la disciplina en las afirmaciones, porque cuestiona las interpretaciones habituales de puntuaciones elevadas sobre los datos resumidos de CIC-IDS 2017 (Dube 2024).

Estas críticas no hacen inútil a CICIDS2017; hacen más identificable el uso irresponsable. Los patrones comunes incluyen tratar los CSV pregenerados como verdad incuestionable, usar particiones aleatorias por filas de todos los días, conservar identificadores o marcas temporales que exponen el contexto del escenario, informar solo de la exactitud, ignorar el desbalanceo de clases y no documentar si los datos son originales, corregidos, reconstruidos o curados. Las variantes corregidas o reconstruidas pueden ser valiosas, pero no son intercambiables con la distribución original de CSV. Boukhamla y Coronel Gavero aplican PCA a CICIDS2017 y evalúan esa versión reducida (Boukhamla et al. 2021). Ese caso ilustra por qué una tesis debe nombrar qué variante utiliza, el preprocesamiento aplicado y evitar mezclar resultados de diferentes variantes como si procedieran de un único conjunto de datos.

El repositorio actual responde a estas preocupaciones mediante una copia

curada y rastreada, una copia de referencia sin rastrear y limpieza a nivel de cargador en `src/load_cicids2017.py`. El adaptador realiza coerción numérica, tratamiento de valores no válidos, mapeo canónico y exclusión anti-fugas. También ajusta el escalado sobre la partición de entrenamiento y aplica la transformación ajustada a prueba, en vez de ajustar sobre todos los datos. El mapeo canónico en `src/canonical_schema.py` excluye IPs, marcas temporales, Flow ID y puertos específicos de las características canónicas.

El código de evaluación admite particiones aleatorias, por CSV/día y validación *leave-one-CSV-out*. El rendimiento con partición aleatoria sirve para comprobar que modelo e implementación aprenden el benchmark interno. Las particiones por día/archivo más difíciles, las comprobaciones de etiquetas barajadas, la validación *leave-one-CSV-out* y la inferencia offline sobre flujos de laboratorio prueban modos de fallo distintos. Una puntuación alta en el escenario más sencillo no basta para una afirmación de despliegue. En esta tesis, CICIDS2017 debe leerse como benchmark interno principal, no como sustituto de la validación con tráfico capturado por el operador en un laboratorio doméstico cerrado y no adversarial.

3.6 Aprendizaje supervisado y aprendizaje profundo para NIDS

La mayor parte del NIDS basado en flujos se formula como aprendizaje supervisado: cada registro de tráfico se mapea a una etiqueta benigna, maliciosa o de familia de ataque (H. Liu et al. 2019; Ahmad et al. 2021). Los métodos clásicos incluyen árboles de decisión, Random Forests, *SVMs*, *k-NN*, regresión logística, *Naive Bayes* y conjuntos de árboles con *gradient boosting*. Difieren en sus suposiciones, coste operativo y requisitos de memoria, entrenamiento e inferencia. La regresión logística es simple e interpretable y ofrece una referencia

lineal, pero está limitada ante interacciones no lineales. *Naive Bayes* es rápido y sirve como línea base débil, aunque sus supuestos de independencia son irrealistas para estadísticas de flujo correlacionadas. k-NN puede capturar vecindades locales, pero escala mal con tablas grandes y es sensible al escalado de características. Las SVMs pueden ser potentes en conjuntos pequeños, pero resultan costosas con tablas de flujos grandes y exigen escalado cuidadoso de características y selección del kernel.

Los modelos basados en árboles son especialmente importantes para NIDS tabular. Los árboles de decisión y Random Forests manejan interacciones no lineales, escalas heterogéneas, variables irrelevantes y distribuciones mixtas (H. Liu et al. 2019; Apruzzese et al. 2022). Las familias con *gradient boosting*, como *XGBoost*, *LightGBM* y *CatBoost*, son a menudo aprendices tabulares potentes, pero este capítulo no necesita afirmaciones separadas sobre cada implementación si la tesis no las evalúa. Rodríguez et al. comparan técnicas de ML sobre flujos de CICIDS2017, incluidos flujos obtenidos desde PCAP con Zeek; sus resultados siguen siendo evidencia intra-benchmark, no de despliegue (Rodríguez et al. 2022). El punto metodológico importante es que un clasificador de flujos basado en RL debe compararse con líneas base supervisadas tabulares competentes, no solo con métodos débiles u obsoletos.

Esto es directamente relevante para la tesis. Si cada observación es una fila de flujo etiquetada y la recompensa deriva de la corrección de la clasificación, un clasificador supervisado es la línea base natural. Un agente QRDQN no puede evaluarse únicamente contra su curva de recompensa, pérdida de entrenamiento o una línea base de clase mayoritaria; debe compararse con métodos supervisados sólidos bajo el mismo esquema de características, protocolo de partición, preprocesamiento y métricas. El repositorio ya incluye una línea base de Random Forest alineada con las particiones de QRDQN y artefactos por barrido. La comparación debe limitarse a métricas respaldadas por esos artefactos y al mismo protocolo, en vez de inferir superioridad general.

El aprendizaje profundo amplía el espacio de diseño mediante MLPs, CNNs,

RNNs, LSTMs, autoencoders, mecanismos de atención, modelos al estilo transformer y enfoques basados en grafos (Xiao et al. 2021; Asadullah et al. 2023; Sayeeduddin et al. 2022; Nguyen et al. 2022). Los MLPs tratan las características de flujo como vectores numéricos genéricos; las CNNs se utilizan cuando bytes de paquetes, matrices de características o patrones locales se codifican en forma estructurada. Las RNNs y LSTMs se orientan a secuencias temporales de paquetes, flujos o eventos. Los autoencoders apoyan frecuentemente la detección de anomalías al aprender representaciones comprimidas del comportamiento benigno y señalar errores de reconstrucción. Los modelos al estilo transformer y las redes neuronales en grafo son direcciones más recientes para representaciones de tráfico secuenciales y estructurales.

Estas arquitecturas pueden alcanzar puntuaciones de benchmark muy elevadas, sobre todo en conjuntos públicos, pero las revisiones identifican debilidades recurrentes: preprocesamiento poco claro, particiones aleatorias favorables, análisis insuficiente del desbalanceo, validación externa limitada y reproducibilidad insuficiente (H. Liu et al. 2019; Ring et al. 2019; Arp et al. 2020). Un modelo profundo puede sobreajustarse a los artefactos del benchmark igual que un modelo superficial. La familia de modelos por sí sola no resuelve fugas, desbalanceo de clases, disponibilidad de características ni cambio de dominio.

La estandarización de características es otra lección importante del aprendizaje supervisado. Diferentes conjuntos de datos y herramientas exponen conjuntos solapados, pero no idénticos, lo que dificulta la comparación entre conjuntos. Sarhan et al. propusieron un mapeo de características alineado con NetFlow para mejorar la generalizabilidad y la explicabilidad en NIDS (Sarhan et al. 2021). La tesis no adopta ese estándar exacto, pero sí su motivación: un esquema canónico fijo hace auditable la interfaz del modelo y permite comparar las entradas de flujo de CICIDS2017 y de laboratorio a través de un único contrato de preprocesamiento. La máscara de presencia/ausencia extiende esta idea al hacer visibles las características ausentes al modelo y al evaluador.

3.7 Fundamentos del aprendizaje por refuerzo

El aprendizaje por refuerzo estudia cómo un agente selecciona acciones en un entorno para maximizar el retorno (Sutton et al. 2018). Sus conceptos centrales son los estados u observaciones, las acciones, las recompensas, las políticas, las funciones de valor y las transiciones. En un proceso de decisión de Markov, el estado actual resume la información necesaria para la toma de decisiones futura, y las acciones influyen tanto en la recompensa inmediata como en los estados posteriores (Sutton et al. 2018). Esta propiedad secuencial es lo que distingue el RL de la clasificación supervisada ordinaria.

Una política especifica cómo el agente elige acciones a partir de las observaciones. Una función de valor estima el retorno esperado desde un estado, mientras que una función de valor-acción estima el retorno esperado tras tomar una acción particular en un estado. Las recompensas definen el objetivo de la tarea, y el retorno agrega las recompensas futuras, generalmente con descuento. Estas definiciones importan porque cambiar la función de recompensa cambia aquello para lo que el agente está optimizado. En un contexto de seguridad, una recompensa que penaliza más los falsos negativos que los falsos positivos codifica una preferencia operativa concreta; no es un hecho neutral sobre todas las redes.

La distinción entre tareas episódicas y continuas también es relevante. Las tareas episódicas tienen límites y estados terminales, como un episodio de juego o un escenario simulado de respuesta a incidentes. Las tareas continuas modelan la operación continua, como la monitorización continua de la red. Una formulación estática de conjunto de datos como entorno impone a menudo episodios artificiales: un episodio puede ser un número fijo de filas muestreadas, un recorrido por un conjunto de datos o un archivo. Eso es aceptable para la experimentación, pero es más débil que un entorno donde la acción del defensor modifica el estado siguiente.

El Q-learning clásico es un método de diferencia temporal model-free y off-policy que aprende valores de acción sin requerir un modelo de la dinámica del entorno (Watkins et al. 1992). Los métodos model-free aprenden directamente de la interacción o de transiciones registradas, mientras que los métodos basados en modelo aprenden o utilizan un modelo de transición para la planificación. Los métodos on-policy aprenden sobre la política que está generando el comportamiento actual; los métodos off-policy pueden aprender sobre una política objetivo mientras el comportamiento es generado de manera diferente. DQN y QRDQN heredan el enfoque basado en valores y off-policy. La repetición de experiencias, introducida anteriormente como mecanismo para reutilizar experiencias pasadas, también se volvió central en los sistemas de Q-learning profundo posteriores (L.-J. Lin 1992).

El RL tabular no es apropiado cuando las observaciones son vectores numéricos de alta dimensión. Una tabla de búsqueda sobre todas las posibles combinaciones de características de flujo es inviable, y pequeños cambios numéricos podrían crear efectivamente estados nuevos. La aproximación de funciones es, por tanto, necesaria. El DRL utiliza redes neuronales para aproximar políticas o funciones de valor sobre espacios de observación grandes. En esta tesis, la observación es un vector de 152 dimensiones: 76 valores canónicos de flujo más 76 valores de máscara de presencia/ausencia. Eso justifica un algoritmo de la familia DQN más que un algoritmo de RL tabular, pero no justifica por sí solo el RL frente al aprendizaje supervisado.

La ciberseguridad puede contener problemas de RL secuenciales genuinos. Un defensor puede aislar un host, parchear un servicio, cambiar el enrutamiento, desplegar una contramedida o elegir dónde inspeccionar a continuación, y esas acciones pueden afectar a las observaciones posteriores y a las oportunidades del atacante. Sin embargo, un conjunto de datos estático de flujos etiquetados es un escenario más débil. Si se muestrea una fila de flujo, se clasifica y va seguida de otra fila que no es causalmente afectada por la acción anterior, el problema se parece más a una clasificación sensible al coste o a un proceso de

decisión contextual que a una defensa cibernética autónoma completa (Levine et al. 2020; Fujimoto et al. 2019). La tesis debe mantener esta distinción de forma explícita.

3.8 Q-Learning profundo y aprendizaje por refuerzo distribucional

El Q-Learning Profundo sustituye una función de valor-acción tabular por una red neuronal, haciendo aplicable el RL basado en valores a observaciones de alta dimensión (Mnih et al. 2015). DQN popularizó dos mecanismos estabilizadores: la memoria de repetición y una red objetivo separada (Mnih et al. 2015). La memoria de repetición rompe las correlaciones a corto plazo muestreando transiciones pasadas para el entrenamiento y mejora la reutilización de datos. Una red objetivo estabiliza los objetivos de bootstrapping manteniendo el cálculo del objetivo separado de los pesos de la red online, que cambian rápidamente.

Variantes posteriores de DQN abordaron debilidades conocidas. Double DQN reduce el sesgo de sobreestimación desacoplando la selección de acciones de la evaluación del objetivo (Hasselt et al. 2016). Dueling DQN separa la estimación del valor del estado y la ventaja de la acción, lo que puede ayudar cuando muchas acciones tienen valores similares en un estado (Wang et al. 2016). Prioritized Experience Replay muestrea transiciones según la señal de aprendizaje en lugar de hacerlo de forma uniforme (Schaul et al. 2016). Rainbow combinó varias mejoras en un único agente de la familia DQN, incluyendo double Q-learning, redes dueling, repetición priorizada, retornos de múltiples pasos, aprendizaje distribucional y exploración con ruido (Hessel et al. 2018).

El aprendizaje por refuerzo distribucional cambia el objeto que se estima. En lugar de estimar únicamente el retorno esperado, aproxima la distribución de

los retornos posibles (Bellemare et al. 2017). C51 representa la distribución del retorno sobre un soporte categórico fijo. QRDQN usa regresión cuantílica para aproximar los cuantiles del retorno, evitando la misma suposición de soporte fijo (Dabney et al. 2018). QRDQN no es, por tanto, un paradigma separado de DQN; es un miembro distribucional y basado en valores de la familia DQN.

La motivación de seguridad es intuitiva, pero debe formularse con cuidado. Los falsos positivos y los falsos negativos tienen consecuencias asimétricas, y los errores raros de alto coste pueden importar más de lo que sugiere una puntuación media. Una estimación distribucional del valor puede ser útil para estudiar la estructura del retorno bajo recompensas asimétricas. Puede revelar si una decisión tiene una distribución de retornos estrecha o amplia bajo el modelo aprendido. Sin embargo, el RL distribucional no proporciona automáticamente incertidumbre calibrada, seguridad operativa, comportamiento sensible al riesgo ni superioridad sobre el DQN escalar. Esas propiedades requerirían reglas de decisión explícitas, calibración, objetivos de riesgo en la cola o criterios de evaluación más allá de simplemente entrenar un agente distribucional (Bellemare et al. 2017; Dabney et al. 2018).

En esta tesis, QRDQN es una elección algorítmica exploratoria. Es adecuado para un espacio de acciones binario discreto y observaciones de flujo de dimensión moderadamente alta, y está disponible a través de la implementación de SB3-Contrib utilizada por el proyecto (Stable-Baselines3 Contributors 2023; Farama Foundation 2023). La elección algorítmica es coherente: dos acciones, observaciones numéricas, aprendizaje de valores off-policy, repetición y un objetivo distribucional. La afirmación evidencial sigue siendo empírica. QRDQN no debe describirse como probadamente superior para NIDS a menos que las comparaciones con el mismo protocolo frente a DQN, Random Forest y otras líneas base apoyen esa afirmación.

3.9 Aprendizaje por refuerzo para la detección de intrusiones y la defensa cibernética

El RL y el DRL para la detección de intrusiones ya forman una bibliografía activa (W. Yang et al. 2026; Kheddar et al. 2025; Gueriani et al. 2024; Adawadkar et al. 2022). Los trabajos previos incluyen clasificadores de intrusiones al estilo DQN, enfoques actor-crítico, formulaciones de entorno adversarial, agentes de selección de características, sistemas orientados a SDN o IoT, y simulaciones de defensa cibernética autónoma. El campo es heterogéneo, por lo que los trabajos no deben tratarse como directamente comparables a menos que su formulación de tarea, conjuntos de datos, acciones, recompensas y protocolos de validación estén alineados.

Una rama está próxima a esta tesis: la detección de intrusiones con conjunto de datos como entorno. Lopez-Martin et al. reformularon la detección supervisada de intrusiones como un problema de RL profundo tratando los registros etiquetados como estados, las predicciones de clase como acciones y la recompensa como función de la corrección (Lopez-Martin, Carro et al. 2020). Su trabajo posterior extendió una formulación relacionada de RL offline a varios conjuntos de datos, incluyendo CICIDS2017 y UNSW-NB15 (Lopez-Martin, Sanchez-Esguevillas et al. 2021). Ren et al. utilizaron DRL para la selección de características y la clasificación en CSE-CIC-IDS2018 (Ren et al. 2022). Estos trabajos muestran que las formulaciones de RL para IDS tienen precedentes.

Otros trabajos de RL orientados a IDS exploran DQN, Double DQN, DQN adversarial, actor-crítico, SAC, DDPG y diseños al estilo Rainbow (Caminero et al. 2019; Han et al. 2020; Hsu et al. 2023; Alavizadeh et al. 2021; Hossain et al. 2025). Los detalles varían ampliamente. Algunos trabajos utilizan clasificación binaria, otros etiquetas de ataque multiclase, selección de características, control de umbral o acciones específicas de SDN. Muchos siguen evaluándose sobre conjuntos de datos públicos estáticos. Eso los convierte en antecedentes

relevantes para la tesis, pero también limita la comparabilidad directa. Un trabajo que usa NSL-KDD con particiones aleatorias y recompensas de corrección no es evidencia de que un defensor QRDQN offline generalice de CICIDS2017 a tráfico de laboratorio.

Una segunda rama estudia la defensa cibernética genuinamente secuencial. Los enfoques POMDP y de grafo de ataque modelan acciones defensivas bajo incertidumbre (Hu et al. 2020). CybORG proporciona un entorno al estilo gym para agentes cibernéticos autónomos (Standen et al. 2021). Los trabajos más amplios de defensa cibernética autónoma discuten agentes que seleccionan contramedidas en redes simuladas o emuladas (Palmer et al. 2023; Center for Security and Emerging Technology et al. 2023). Estos escenarios son conceptualmente diferentes de esta tesis porque las acciones del defensor pueden alterar el estado futuro. Representan una forma más fuerte de problema de RL que la clasificación estática por filas.

Tabla 3.2: Síntesis de trabajos de RL/DRL para NIDS y límites de comparabilidad con esta tesis. La relación indicada se infiere de cada fuente y de la formulación QRDQN offline sobre flujos de CICIDS2017 adoptada en este trabajo.

Fuente	Formulación de RL	Datos / ámbito	Límite de evaluación	Relación / diferencia con esta tesis
Malik y Singh Saini (2023)	RL; el algoritmo no es verificable desde el registro primario público.	NIDS genérico; El registro no expone datos no verificados cables.	El registro no expone pro-tocolo ni métricas comparables.	Antecedente temático, no comparación directa con QRDQN/CICIDS2017.
Malik y Saini (2023)	Técnicas de RL; algoritmo no verificable desde el registro primario público.	NIDS genérico; El registro no expone datos no verificados cables.	El registro no expone pro-tocolo ni métricas comparables.	Antecedente genérico, sin comparabilidad empírica directa.
Sanusi (2023)	DRL para IDS binario.	NSL-KDD; la tarea multiclase se reduce a binaria.	la Tesis offline sobre benchmark; no evidencia de despliegue.	Antecedente de IDS binario con DRL, pero datos e implementación distintos.

Continúa en la página siguiente

Tabla 3.2 (continuación)

Fuente	Formulación de RL	Datos / ámbito	Límite de evaluación	Relación / diferencia con esta tesis
B. Yang et al. (2022)	DQN y gra- diente de política con flujo; entrenamiento adversarial.	IDS a nivel de paquete y sin evidencia de despliegue; CICD-DoS2019.	Evaluación de benchmark; sin evidencia de despliegue más operativo.	Diseño paquete/flujo más rico y datos distintos de QRDQN/CICIDS2017.
Cevallos et al. (2023)	M. Revisión/taxonomía de DRL-IDS, IDS en IoT, no un algoritmo único.	Literatura de IDS en IoT.	Síntesis bibliográfica, benchmark propio.	Contexto de decisiones DRL en IoT, no línea base empírica.
Li et al. (2025)	DDPG.	Detección de ataques de red; datos verificables.	El registro no expone protocolos ni métricas comparables.	Familia de RL distinta; solo antecedente a nivel de título.
Kanimozhi et al. (2025)	DRL-IDS con LFTS-RNN y PC-JTFOA.	SDN; KDD y WPPD.	NSL-Partitiones de benchmark, Tarea, modelo y despliegue SDN vivo.	Modelo y datos distintos; no equivale a una evaluación QRDQN offline.
Shaikh et al. (2025)	CNN-LSTM con DQN y PPO.	IoMT sanitario; CIoMT2024.	Benchmark especializado; el trabajo deja la detección <i>zero-day</i> como trabajo futuro.	Propuesta IoMT específica, no línea base comparable con CIDS2017.

La tesis pertenece, por tanto, principalmente a la primera rama: detección offline a nivel de flujo formulada a través de una interfaz de RL. Puede referenciar la defensa cibernética autónoma como contexto más amplio y dirección futura, pero no debe implicar que el sistema actual realice aprendizaje multiagente, adversarial o de control de red en vivo. La formulación más precisa es que la tesis implementa un benchmark de NIDS con conjunto de datos como entorno y una interfaz de acción de seguridad binaria sensible al coste, no un defensor de cyber-range completo.

3.10 Clasificación como RL y decisiones de seguridad binarias

La formulación PERMIT/BLOCK pertenece a la discusión de clasificación como RL. En el entorno actual, cada observación de flujo se presenta al agente, la acción es binaria y la recompensa se calcula a partir de la acción y la etiqueta de verdad. PERMIT corresponde a aceptar el tráfico como benigno, mientras que BLOCK corresponde a identificarlo como ataque. Un falso negativo permite un flujo de ataque; un falso positivo bloquea tráfico benigno.

Este enfoque tiene un beneficio metodológico real: hace explícitas las decisiones de seguridad sensibles al coste. Los errores de IDS no son simétricos en la práctica. Omitir un ataque y bloquear tráfico benigno tienen significados operativos diferentes, y la compensación preferida depende del entorno defendido (Lee et al. 2002; Cárdenas et al. 2006). El diseño de la recompensa puede codificar esta asimetría directamente, lo que es útil para una tesis que estudia reglas de decisión defensivas en lugar de solo la predicción de etiquetas.

La limitación es igualmente importante. Si el conjunto de datos no reacciona a la acción, entonces PERMIT/BLOCK no es una acción de control activa. Es una etiqueta de decisión offline sobre una fila de flujo registrada o extraída. La fila siguiente no cambia porque el agente haya bloqueado o permitido la fila anterior. La formulación es, por tanto, más próxima a una clasificación con recompensa diseñada que a un firewall, un IPS, un controlador SDN o un agente de defensa cibernética autónoma. Esto no es un defecto si se declara con claridad. Solo se convierte en un defecto si el capítulo reclama en exceso el bloqueo en vivo, la preparación para el despliegue o el control secuencial completo.

La función de recompensa también necesita una interpretación cuidadosa. Una recompensa positiva para un verdadero positivo y una penalización mayor para un falso negativo puede expresar la idea de que los ataques omitidos

son costosos. Una penalización por falso positivo puede expresar el coste de interrumpir el tráfico benigno o de sobrecargar a los analistas. Una recompensa de cero o pequeño valor para los verdaderos negativos puede mantener el foco en la detección de ataques. Estas son decisiones de diseño, no economía universal de seguridad. Diferentes organizaciones elegirían distintos costes dependiendo de la continuidad del negocio, el modelo de amenaza, la capacidad del analista y la tolerancia a los ataques omitidos.

Dado que la formulación está próxima a la clasificación sensible al coste, las líneas base supervisadas son obligatorias. Un Random Forest sólido, e idealmente otras líneas base tabulares, ayudan a determinar si QRDQN añade valor empírico más allá de las características canónicas, el preprocesamiento y el diseño de la recompensa. Las mismas métricas deben calcularse tanto a partir de las salidas de RL como de las supervisadas. La recompensa de entrenamiento sola no es suficiente, porque una curva de recompensa puede mejorar mientras la precisión, el recall o el comportamiento de falsos positivos sigue siendo operativamente deficiente. La tesis debe interpretar cualquier ventaja de RL como específica del benchmark y el protocolo, a menos que se disponga de evidencia más amplia.

3.11 Riesgos metodológicos en NIDS basado en ML/DL/RL

La bibliografía de NIDS advierte repetidamente contra sobreinterpretar la exactitud de benchmark. Sommer y Paxson señalan que el ML para detección de intrusiones es difícil de validar en entornos realistas: el tráfico benigno es diverso, los ataques raros, las etiquetas difíciles de obtener y las tasas base operativas difieren de las del benchmark (Sommer et al. 2010). Arp et al. identifican riesgos afines en ML para seguridad, como fuga de información,

sesgo de muestreo, métricas inapropiadas y diseño experimental débil (Arp et al. 2020). La evidencia sobre generalización entre conjuntos y sobre la elección de datos de entrenamiento refuerza que un resultado dentro de un benchmark no basta para inferir el comportamiento en otro entorno (Layeghy et al. 2022; Cantone et al. 2024; Iwanowski et al. 2025; Layeghy et al. 2023).

La fuga de información abarca más que poner por error la misma fila en entrenamiento y prueba: incluye cualquier información relacionada con el objetivo que no estaría legítimamente disponible al predecir (Kaufman et al. 2011). En NIDS puede entrar por identificadores de endpoint, marcas temporales absolutas, Flow ID, puertos específicos del escenario, preprocesamiento global ajustado antes de particionar, flujos duplicados o casi duplicados, y particiones que comparten contexto de captura. Un modelo que aprende que una IP, puerto, día o Flow ID concreto corresponde a un ataque puede puntuar bien sin aprender comportamiento malicioso portable.

La fuga temporal y de escenario es especialmente importante. Los conjuntos públicos de NIDS suelen organizarse por día, archivo de captura, ventana de ataque o escenario con script. Una partición aleatoria por filas puede dejar tráfico muy relacionado a ambos lados de la frontera de entrenamiento y prueba. Aunque las filas no sean duplicados exactos, pueden compartir hosts, herramientas de ataque, regímenes de temporización o artefactos de extracción. Esto importa en CICIDS2017 porque trabajos posteriores reportan errores o artefactos en extracción de flujos, etiquetado y uso de CSV resumidos (Engelen et al. 2021; Lanvin et al. 2023; L. Liu et al. 2022; Rosay et al. 2022; Dube 2024).

El preprocesamiento también puede introducir fugas. El escalado, la imputación, la selección de características, el recorte de valores atípicos o la reducción de dimensionalidad deben ajustarse sobre entrenamiento y aplicarse después a los datos de validación o de prueba. Si un escalador se ajusta sobre todos los datos antes de particionar, la información distribucional de prueba entra en el entrenamiento. Puede parecer menos grave que filtrar etiquetas, pero con

estructura fuerte por día o escenario también puede inflar la confianza. Por ello, los objetos `StandardScaler` ajustados sobre entrenamiento en el repositorio forman parte del diseño metodológico, no solo de la codificación.

El desbalanceo de clases y el problema de la tasa base complican la interpretación. En redes operativas, los ataques suelen ser raros frente al tráfico benigno. Un detector puede lograr exactitud elevada mientras omite muchos ataques, o alto recall mientras genera demasiadas alertas falsas para investigarlas. El argumento de la tasa base de Axelsson sigue siendo relevante: incluso tasas bajas de falsos positivos pueden dominar los flujos de alertas cuando la prevalencia real de ataque es baja (Axelsson 2000). Por eso la exactitud no debe ser la métrica principal para NIDS.

El RL introduce riesgos adicionales: el entrenamiento profundo puede ser estocástico y sensible a semillas aleatorias, hiperparámetros, modelado de la recompensa, configuración del buffer de repetición, exploración y detalles del entorno. En un conjunto de datos estático usado como entorno, buenos resultados pueden reflejar las mismas ventajas de características y partición que benefician a modelos supervisados. Una afirmación de superioridad de QRDQN requiere el mismo protocolo de comparación y reconocer la varianza entre ejecuciones cuando no hay semillas repetidas. Las ganancias de una única ejecución deben tratarse como preliminares salvo que las sostengan experimentos repetidos o artefactos de validación sólidos.

3.12 Protocolos de evaluación, métricas y reproducibilidad

La evaluación debe ajustarse a la solidez de la afirmación. Las particiones aleatorias sobre el mismo conjunto de datos son aceptables para comprobaciones internas de aprendizaje, pero son evidencia débil para el despliegue. Una escalera

de validación práctica para esta tesis comienza con particiones aleatorias estratificadas, avanza hacia particiones temporales o por CSV/día, la validación leave-one-scenario o leave-one-CSV-out, las pruebas entre conjuntos de datos con un esquema de características armonizado, y finalmente el tráfico capturado de forma independiente o similar al de producción. La tesis no alcanza este último peldaño; su inferencia sobre tráfico de laboratorio, generado por el operador en un entorno doméstico cerrado y no adversarial, se sitúa por debajo. Cada paso aumenta la separación distribucional entre datos de entrenamiento y prueba, y por tanto permite afirmaciones de generalización más sólidas.

El repositorio actual refleja esta escalera. El entrenamiento con partición aleatoria verifica que la implementación de QRDQN, el esquema de características, la función de recompensa y el bucle de entrenamiento pueden aprender el benchmark controlado. La comprobación B con etiquetas barajadas comprueba si el rendimiento colapsa hacia una línea base de clase mayoritaria cuando las etiquetas ya no contienen señal real. La comprobación C utiliza una partición por CSV/día más difícil. El script leave-one-CSV-out comprueba si retener un CSV original de CICIDS2017 modifica el comportamiento. La inferencia offline de Fase 2 sobre CSV de flujos de laboratorio prueba la cuestión separada de si un modelo entrenado sobre el benchmark puede procesar flujos capturados por el operador en un laboratorio doméstico cerrado.

Las métricas también deben interpretarse operativamente. La exactitud es útil pero insuficiente para NIDS desequilibrados. La precisión indica cuán fiables son las alertas positivas. El recall indica qué proporción del tráfico de ataque es detectada. F1 proporciona un equilibrio compacto, pero oculta la compensación real entre falsos positivos y falsos negativos. La tasa de falsos positivos mapea al tráfico benigno bloqueado o alertado incorrectamente. La tasa de falsos negativos mapea al tráfico malicioso omitido por el detector (Axelsson 2000; Fawcett 2006; Saito et al. 2015). Las matrices de confusión, la precisión, el recall, F1, TP, FP, FN, TN, FPR y FNR por clase deben preferirse sobre un único número resumen (Milenkoski et al. 2015).

La evaluación sensible al coste es relevante porque el punto de operación correcto depende del contexto. La guía del NIST y la bibliografía de evaluación de IDS enfatizan ambas las compensaciones entre falsos positivos y falsos negativos (Scarfone et al. 2007; Cárdenas et al. 2006). En esta tesis, el diseño de recompensa asimétrico debe tratarse como una suposición de escenario y evaluarse empíricamente, no como un modelo de coste universal para todas las redes. Si la recompensa penaliza fuertemente los falsos negativos, la política resultante puede preferir bloquear más flujos. Ese comportamiento debe juzgarse mediante precisión, recall, tasa de falsos positivos y las suposiciones de coste específicas declaradas en el experimento.

La reproducibilidad es parte de la evidencia, no un detalle administrativo. Los resultados de ML en ciberseguridad deben preservar la versión del conjunto de datos, la variante exacta del conjunto de datos, el esquema de características, los campos excluidos, el orden del preprocesamiento, la lógica de partición, el estado del escalador, las semillas aleatorias, la configuración del modelo, la configuración de la recompensa, el identificador de ejecución, las métricas y las predicciones donde sea factible. Olszewski et al. muestran que la reproducibilidad sigue siendo un problema importante en la investigación de ML en seguridad (Olszewski et al. 2023). El uso en el repositorio de configuraciones guardadas, métricas, artefactos del modelo, salidas de validación y carpetas de ejecución es, por tanto, una fortaleza metodológica, pero los artefactos faltantes deben seguir claramente marcados como pendientes.

Esta disciplina en los informes también protege contra las afirmaciones excesivas accidentales. Un resultado vinculado a la ejecución C03 de QRDQN con partición aleatoria completa es un resultado para esa ejecución, con esa partición, configuración de recompensa, ruta de preprocesamiento y estado de artefactos. No debe convertirse silenciosamente en una afirmación sobre todas las configuraciones de QRDQN, todas las variantes de CICIDS2017 o todo el tráfico de red. Cuanto más sólida sea la afirmación, más completo debe ser el rastro de artefactos.

3.13 Eficiencia de datos y evaluación a escala de entrenamiento

La eficiencia de datos pregunta cuánto tráfico etiquetado se necesita para alcanzar un rendimiento útil, y cómo cambia la curva de aprendizaje a medida que crece el conjunto de entrenamiento. Esta pregunta es importante para NIDS porque el tráfico de ataque etiquetado es costoso, las distribuciones de tráfico cambian y la abundancia de benchmarks públicos no implica abundancia de datos de despliegue (Ring et al. 2019; Apruzzese et al. 2022). También es importante para el RL porque los métodos de RL profundo pueden ser exigentes en muestras, especialmente cuando las señales de recompensa son escasas, ruidosas, retardadas o altamente diseñadas (Sutton et al. 2018; Hessel et al. 2018).

La bibliografía existente de RL-IDS con frecuencia entrena sobre conjuntos de datos públicos completos e informa de puntuaciones finales en el benchmark, mientras que las ablaciones controladas a escala de entrenamiento son menos comunes (W. Yang et al. 2026; Kheddar et al. 2025). El trabajo de NIDS supervisado con pocos ejemplos y aprendizaje incremental por clases aborda preguntas relacionadas desde una perspectiva diferente (Di Monda et al. 2024), pero la pregunta de la tesis es más estrecha: bajo un mismo esquema de características y protocolo de evaluación, ¿cómo escala QRDQN con las filas de entrenamiento disponibles, y cómo se compara eso con las líneas base supervisadas?

La respuesta importa porque la elección del modelo y las necesidades de datos están conectadas. Si un Random Forest alcanza un rendimiento estable con muchas menos filas que QRDQN bajo la misma partición, eso es evidencia importante incluso si QRDQN lo alcanza posteriormente. Si QRDQN es más sensible a los datos reducidos o al desbalanceo de la clase de ataque, la tesis debe reportarlo como una limitación. Si QRDQN es competitivo con presupuestos

menores, la evidencia debe seguir vinculada al benchmark y al protocolo de validación, no generalizada a todos los escenarios de NIDS.

La tesis posiciona los experimentos a escala de entrenamiento como evidencia metodológica. Presupuestos como 100k, 250k, 500k, 1M y 2M muestras pueden comprobar si la formulación de RL requiere sustancialmente más datos que una línea base supervisada para alcanzar un rendimiento comparable. Tales experimentos no deben enmarcarse como prueba de aprendizaje con pocos ejemplos ni de preparación para el despliegue. Son evidencia interna sobre los requisitos de datos de esta formulación particular. La salida útil es una comparación de curvas de aprendizaje con preprocesamiento consistente, semillas aleatorias donde sea factible, métricas compartidas y rutas de artefactos explícitas.

3.14 Brecha de investigación y posicionamiento de esta tesis

La bibliografía respalda una brecha de investigación estrecha pero significativa. El NIDS y el ML basado en flujos son áreas maduras; conjuntos de datos públicos como CICIDS2017 son útiles pero frágiles; las líneas base tabulares supervisadas son sólidas; el RL y el DRL para IDS ya existen; y la defensa cibernética autónoma es un campo más amplio y más secuencial que la implementación actual (Ring et al. 2019; H. Liu et al. 2019; W. Yang et al. 2026; Kheddar et al. 2025). La brecha no es la ausencia de RL para la detección de intrusiones.

La contribución de esta tesis es un estudio experimental reproducible de una formulación offline de decisión de seguridad binaria a nivel de flujo. El sistema mapea CICIDS2017 y entradas posteriores de flujos de laboratorio en un esquema canónico fijo de 76 características, aumenta las observaciones con

una máscara de presencia/ausencia para producir entradas de 152 dimensiones, expone las filas a través de un entorno compatible con Gymnasium, y entrena un agente QRDQN para elegir entre PERMIT y BLOCK. Esto hace explícita la interfaz de decisión sensible al coste mientras preserva una obligación de comparación clara contra las líneas base supervisadas.

La tesis también está posicionada metodológicamente. Trata a CICIDS2017 como el benchmark interno principal, no como prueba de preparación para la producción. Separa los resultados con partición aleatoria de la validación más estricta. Utiliza preprocesamiento con control de fugas y comprobaciones de validación para reducir el aprendizaje evidente de artefactos. Trata el tráfico de laboratorio capturado externamente como el siguiente paso evidencial preferido, reconociendo al mismo tiempo que la inferencia actual de Fase 2 es offline y no realiza bloqueo activo.

La brecha final es, por tanto, una brecha de integración: un benchmark QRDQN reproducible, con control de fugas y sensible al coste para el NIDS a nivel de flujo, evaluado contra líneas base supervisadas e interpretado a través de una escalera de validación. La tesis puede contribuir haciendo explícito el contrato de características, guardando artefactos de ejecución, documentando las suposiciones de la recompensa, comparando contra Random Forest bajo particiones emparejadas, y mostrando cómo el rendimiento interno de CICIDS2017 cambia bajo una validación más estricta y restricciones de escala de entrenamiento.

La afirmación final defendible no es que QRDQN sea universalmente mejor que los modelos de NIDS supervisados, ni que el proyecto implemente defensa cibernética autónoma. La afirmación defendible es que la tesis evalúa un defensor a nivel de flujo basado en QRDQN, sensible al coste, bajo un benchmark reproducible y un pipeline de validación, y utiliza líneas base supervisadas, comprobaciones con control de fugas, análisis de escala de entrenamiento e inferencia sobre tráfico de laboratorio para determinar hasta qué punto puede confiarse en esa formulación. Si experimentos posteriores muestran un

rendimiento débil entre archivos o con tráfico de laboratorio, eso no invalida la tesis. Agudiza su contribución al demostrar los límites del RL entrenado sobre benchmark bajo un cambio de distribución más realista.

Diseño del sistema

4.1 Visión general metodológica

Este capítulo describe el diseño metodológico empleado para construir y evaluar el defensor de ciberseguridad basado en aprendizaje por refuerzo (RL) propuesto. El objetivo no es presentar un sistema de prevención de intrusiones (IPS) listo para producción, sino definir un marco experimental controlado, reproducible y offline para decisiones de seguridad binarias sobre registros de flujos de red. Cada flujo se trata como un único punto de decisión: el defensor observa una representación numérica del flujo y decide si permitirlo (**PERMIT**) como tráfico benigno o bloquearlo (**BLOCK**) como tráfico malicioso.

La metodología es deliberadamente conservadora. Los benchmarks públicos de detección de intrusiones pueden producir resultados inflados cuando el preprocesamiento, la construcción de las particiones, los controles de fuga de información y las comparaciones con líneas base son débiles (Arp et al. 2020; Engelen et al. 2021; Lanvin et al. 2023). Por ello, el pipeline se ha diseñado en torno a contratos de datos explícitos, artefactos de ejecución reproducibles, normalización ajustada únicamente sobre datos de entrenamiento, y una validación progresivamente más estricta. La misma representación canónica se utiliza

tanto para el benchmark interno de CICIDS2017 como para la inferencia offline sobre tráfico de laboratorio en la Fase 2, de modo que la pregunta experimental no queda ligada a una convención de nomenclatura de CSV concreta ni a un extractor de características particular.

A grandes rasgos, la metodología comprende seis etapas encadenadas:

1. ingerir los datos CSV de CICIDS2017 a nivel de flujo y mapear las etiquetas a una tarea binaria de defensa;
2. eliminar los identificadores susceptibles de provocar fuga de información y normalizar los valores numéricos malformados;
3. mapear las columnas específicas de cada extractor a un esquema canónico de características fijo de 76 características;
4. añadir una máscara de presencia/ausencia de 76 dimensiones, produciendo observaciones de 152 dimensiones;
5. entrenar y evaluar un defensor QRDQN compatible con Gymnasium bajo protocolos de partición controlados;
6. comprobar la robustez mediante comprobaciones de validación, líneas base supervisadas e inferencia offline en la Fase 2.

Este diseño mantiene la tesis dentro de un marco experimental offline. Las salidas PERMIT y BLOCK son predicciones del modelo sobre registros de flujo almacenados. No son órdenes de cortafuegos y no modifican ninguna ruta de red activa.

La Figura 4.1 resume esta arquitectura en dos recorridos separados: entrenamiento y evaluación sobre CICIDS2017, e inferencia offline posterior sobre tráfico de laboratorio. La separación visual refuerza una idea metodológica central: la Fase 2 reutiliza el esquema canónico, el escalador y el modelo entrenado, pero no reentrena ni reajusta el pipeline sobre la captura de laboratorio.

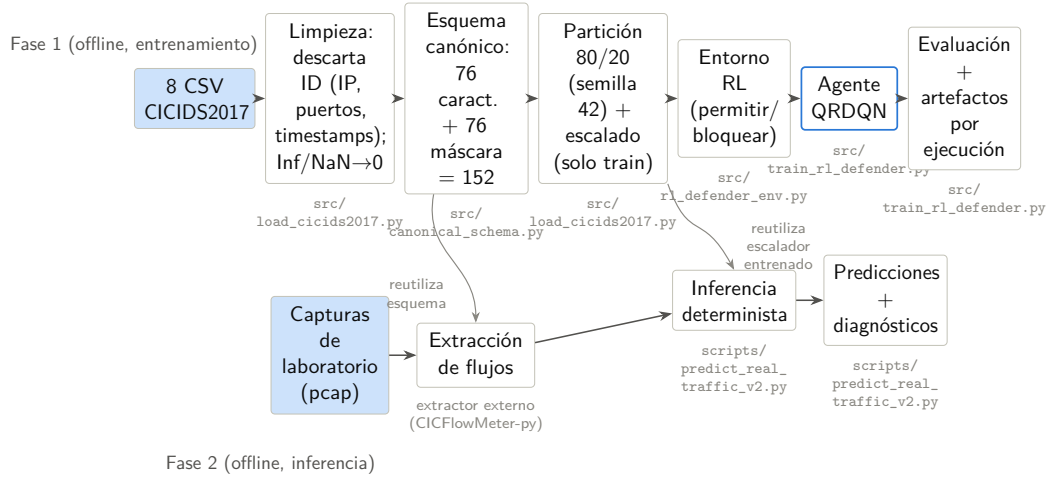


Figura 4.1: Pipeline general del sistema: entrenamiento offline sobre CICIDS2017 e inferencia offline en la Fase 2. Fuente: elaboración propia a partir de `src/load_cicids2017.py`, `src/canonical_schema.py`, `src/train_rl_defender.py` y `scripts/predict_real_traffic_v2.py`.

4.2 Fuente de datos y representación de la entrada

La tesis utiliza una representación tabular a nivel de flujo. Esta representación es un compromiso deliberado entre relevancia operativa, manejabilidad y privacidad. Captura el comportamiento estadístico a lo largo de secuencias de paquetes evitando la inspección de la carga útil.

4.2.1 CSVs de flujos de CICIDS2017

La fuente de datos principal es la publicación de CSVs de flujos de CICIDS2017. Cada fila de un CSV corresponde a un flujo de red bidireccional resumido mediante estadísticas numéricas como duración, conteos de paquetes, totales de bytes, tiempos de llegada entre paquetes, estadísticas de longitud de paquete y conteos de flags TCP. Las representaciones basadas en flujos se utilizan ampliamente en la investigación sobre NIDS porque comprimen el comportamiento de la red en filas numéricas de anchura fija que pueden

procesarse mediante métodos estándar de aprendizaje automático (Sperotto et al. 2010; Umer et al. 2017).

El pipeline de carga local espera los archivos CSV oficiales de CICIDS2017 y admite presupuestos de filas tanto completos como limitados. Los presupuestos limitados permiten pruebas rápidas de humo e iteración ágil; los presupuestos completos son los utilizados en las ejecuciones finales del benchmark. El cargador lee los CSVs grandes en fragmentos, estandariza los nombres de columna eliminando espacios en blanco inconsistentes, identifica la columna de etiqueta de forma robusta y aplica después la limpieza de características antes de cualquier escalado dependiente de la partición.

4.2.2 Mapeo de etiquetas binarias

CICIDS2017 incluye varias etiquetas de ataque. La tesis las colapsa en una decisión binaria del defensor:

- etiqueta 0: BENIGN, mapeada a la acción deseada PERMIT;
- etiqueta 1: cualquier clase de ataque, mapeada a la acción deseada BLOCK.

Este mapeo binario se corresponde con el modelo de decisión de primera línea estudiado aquí. Además, hace que el espacio de acciones de RL sea sencillo y auditable. La limitación es que la información sobre la familia de ataque no es el objetivo de aprendizaje principal en la ruta de RL actual. Cualquier análisis de errores por familia debe por tanto estar respaldado por etiquetas de familia preservadas o artefactos independientes; no puede inferirse a partir de las etiquetas binarias por sí solas.

4.2.3 Representación tabular a nivel de flujo

El sustrato de aprendizaje es una tabla a nivel de flujo en lugar de paquetes en bruto o bytes de carga útil. Esto reduce la dimensionalidad y hace factible

el entrenamiento offline a gran escala. También evita algunas complicaciones legales y técnicas de la inspección profunda de paquetes, especialmente cuando las cargas útiles están cifradas (Sperotto et al. 2010; Umer et al. 2017). Sin embargo, la abstracción de flujo pierde el contenido, el orden preciso de los paquetes y parte del contexto de largo alcance entre múltiples flujos. La metodología estudia por tanto un defensor basado en flujos, no un sistema completo de análisis forense de red.

4.3 Limpieza de datos y preprocesamiento

El preprocesamiento se trata como parte del método experimental, no como un detalle de implementación. En los benchmarks de NIDS, pequeñas decisiones de preprocesamiento pueden cambiar el significado de la tarea al filtrar etiquetas, suprimir flujos malformados o hacer que los datos de prueba influyan en el entrenamiento.

4.3.1 Normalización de columnas y coerción numérica

Las columnas CSV en bruto pueden contener espacios iniciales, capitalización inconsistente y tipos de datos mixtos. El cargador primero normaliza los nombres de columna y después convierte las columnas de características a valores numéricos. La matriz de características final se representa como `float32`, mientras que las etiquetas se representan como `int64`. Este contrato de tipos mantiene estable la interfaz del entorno, el escalador y el modelo en las etapas posteriores.

No se permite que las filas introduzcan cadenas de texto arbitrarias en el modelo. Las columnas de características no numéricas que sobreviven a la eliminación de identificadores quedan excluidas. Esto evita la introducción accidental de cadenas categóricas cuya codificación sería no controlada o específica

de un conjunto de datos.

4.3.2 Valores inválidos, valores ausentes e imputación

La extracción de flujos de red puede producir valores numéricos inválidos. Las características de tasa, por ejemplo, pueden volverse infinitas o indefinidas cuando se calculan sobre una duración nula o casi nula. El pipeline sustituye los infinitos y los valores numéricos ausentes por cero antes del mapeo canónico. Esta elección es pragmática: muchos contadores de flujo, tasas y estadísticas agregadas pueden utilizar de forma segura el cero como marcador de posición neutro cuando se acompaña de una señal explícita de presencia/ausencia.

La imputación por cero sola sería insegura porque el modelo no podría distinguir un cero medido real de un marcador sintético. El esquema canónico empareja por tanto la imputación con una máscara de presencia/ausencia. La máscara hace visible la disponibilidad de características tanto para el modelo como para los diagnósticos posteriores.

4.3.3 Exclusión de campos susceptibles de fuga

El pipeline de preprocesamiento aplica una política anti-fuga fundamentada en los riesgos de evaluación conocidos en seguridad con ML (Arp et al. 2020; Kaufman et al. 2011). Se excluyen las direcciones IP de origen y destino, los Flow IDs, las marcas temporales absolutas y los identificadores de endpoints. Los campos de puerto directo también se excluyen porque pueden actuar como indicadores del escenario en conjuntos de datos controlados por scripts. Por ejemplo, un día de ataque puede concentrar tráfico malicioso en servicios concretos, lo que permitiría al modelo memorizar un script de captura en lugar de aprender estadísticas de comportamiento de flujo.

Esta exclusión se aplica antes del mapeo canónico. El modelo recibe por tanto únicamente características estadísticas de flujo y los valores de máscara

correspondientes. Si en el futuro se introducen nuevos conjuntos de datos o extractores, sus adaptadores deben preservar la misma política: sin identificadores, sin campos de tiempo absoluto y sin puertos utilizados directamente como atajos de etiqueta.

4.4 Esquema canónico de características

El esquema canónico es el contrato de datos central de la tesis. Separa la interfaz del modelo de los nombres de columna, el orden y la disponibilidad de características de un extractor concreto.

4.4.1 Características canónicas de flujo

El esquema define 76 características numéricas de flujo ordenadas. Incluyen duración, conteos de paquetes hacia adelante y hacia atrás, totales de bytes, estadísticas de longitud de paquete, estadísticas de tiempo entre llegadas de flujo y por dirección, tasas de paquetes y bytes, conteos de flags TCP, longitudes de cabecera, tamaños de ventana, estadísticas de subflujos y características de temporización activa/inactiva. Estas características se han elegido porque son estadísticas de flujo y no identificadores, y porque son ampliamente compatibles con la extracción al estilo CICFlowMeter.

Para CICIDS2017, un adaptador mapea las columnas originales del CSV a estos nombres canónicos. Para la Fase 2, el script de inferencia robusta mapea las columnas del extractor de flujos de laboratorio a los mismos nombres canónicos. El modelo se entrena y evalúa por tanto frente a un contrato de características fijo. Este diseño sigue la motivación metodológica más amplia de la estandarización de características en NIDS: la comparación entre dominios resulta más auditable cuando las definiciones de características son explícitas y estables (Sarhan et al. [2021](#)).

La Tabla 4.1 enumera los grupos lógicos dentro del esquema de 76 características y el número de características aportadas por cada grupo. La agrupación es informativa; el modelo recibe los 76 valores como un vector plano sin señales de agrupación estructural.

La enumeración ordenada de las 76 características, junto con las columnas fuente de CICIDS2017 y de Flowmeter-py que alimentan cada posición, se presenta en el Anexo C. Ese anexo también explicita la correspondencia entre las posiciones 1–76 del vector de valores y las posiciones 77–152 de la máscara.

Grupo de características	Cantidad	Estadísticas representativas
Duración del flujo	1	Duración total del flujo bidireccional
Estadísticas de paquetes hacia adelante	12	Conteo de paquetes, bytes totales, media/desv. típica/mínimo/máximo de longitudes de paquete
Estadísticas de paquetes hacia atrás	12	Las mismas estadísticas para la dirección inversa
Agregados bidireccionales	8	Conteos combinados, bytes totales, promedios de tasa de ráfaga
Estadísticas de tiempo entre llegadas	10	Media, desv. típica, mínimo y máximo del tiempo entre llegadas del flujo, hacia adelante y hacia atrás
Conteos de flags TCP	8	Conteos de FIN, SYN, RST, PSH, ACK, URG, CWE, ECE
Agregados de longitud de cabecera	4	Totales de bytes de cabecera hacia adelante y hacia atrás
Estadísticas de tamaño de ventana	3	Tamaño de ventana inicial y medio por dirección
Características de tasa de flujo	6	Bytes/s, paquetes/s por dirección y combinados
Estadísticas de subflujos	6	Promedios de paquetes y bytes de subflujo por dirección
Temporización activa/inactiva	6	Media, desv. típica y máximo de las duraciones de los periodos activos e inactivos

Tabla 4.1: Agrupación lógica de las 76 características canónicas de flujo. Todos los grupos se fusionan en un único vector plano antes de la entrada al modelo.

4.4.2 Máscara de presencia/ausencia

Diferentes extractores pueden no proporcionar todas las características canónicas, e incluso una columna fuente presente puede contener valores inválidos. El pipeline produce por tanto una máscara de presencia/ausencia de 76 dimensiones. Para cada característica canónica x_i , el valor de máscara correspondiente m_i es:

- $m_i = 1$ cuando la característica fuente está presente y es válida;
- $m_i = 0$ cuando la característica está ausente o ha tenido que ser imputada.

La máscara desempeña dos funciones metodológicas. En primer lugar, evita la imputación silenciosa: el modelo puede condicionar su decisión a si un valor fue observado o rellenado. En segundo lugar, facilita los diagnósticos de la Fase 2, porque un extractor de laboratorio que carece de muchas características al estilo CICIDS2017 puede detectarse mediante un patrón de máscara diferente en lugar de quedar oculto dentro de un vector numérico denso.

Conviene precisar la semántica exacta de la máscara en la implementación actual. La máscara se calcula *después* de la limpieza que sustituye los valores no finitos por cero (`src/load_cicids2017.py`: $\pm\infty \rightarrow \text{NaN} \rightarrow 0$), de modo que $m_i = 1$ siempre que la característica canónica tiene una columna fuente mapeada. Como el mapeo $\text{CICIDS2017} \rightarrow \text{canónico}$ cubre las 76 características, sobre los datos nativos de CICIDS2017 la máscara es **constante e igual a 1** en todas las filas: codifica la *presencia de la columna fuente*, no la ausencia de valores fila a fila. La máscara solo se vuelve informativa en la inferencia entre dominios o en la Fase 2 de laboratorio, donde algunas columnas fuente no están disponibles y su valor de máscara permanece en 0; un mapeo histórico como NSL-KDD, por ejemplo, solo cubre 3 de las 76 características canónicas. Así, las 152 dimensiones de la observación se componen de 76 valores de características más 76 indicadores de presencia.

4.4.3 Vector de observación final

La observación final es:

$$o = [x_1, x_2, \dots, x_{76}, m_1, m_2, \dots, m_{76}]$$

Esto produce un vector `float32` de 152 dimensiones. La primera mitad contiene los valores de las características, y la segunda mitad contiene la máscara. El tamaño de la observación es por tanto invariante entre CICIDS2017, las particiones de validación y la inferencia en la Fase 2. Esta invariancia es importante porque la red de política del QRDQN espera una dimensión de entrada fija.

El Algoritmo 1 resume de forma operativa el proceso descrito en los párrafos anteriores: para cada fila del DataFrame de entrada construye, característica a característica, el valor x_i y el indicador de presencia m_i a partir del mapeo columna-fuente $\rightarrow$ nombre canónico, y concatena ambos vectores en la observación final de 152 dimensiones.

La Figura 4.2 muestra la misma construcción desde el punto de vista de la entrada al modelo: columnas del extractor, mapeo al contrato canónico, 76 valores de características, 76 indicadores de presencia y vector final de 152 dimensiones.

4.5 Formulación del aprendizaje por refuerzo

El defensor se implementa como un entorno compatible con Gymnasium (Farama Foundation 2023). Esto permite que el código estándar de RL basado en valores interactúe con el conjunto de datos de flujos a través de la interfaz habitual `reset()` y `step()`.

Algoritmo 1 Mapeo al esquema canónico y construcción de la observación. Fuente: elaboración propia a partir de `src/canonical_schema.py`.

Require: DataFrame D con las columnas producidas por el extractor; mapeo μ : columna_fuente $\rightarrow$ nombre_canónico

Ensure: Matriz de observaciones O , con una fila $o \in \mathbb{R}^{152}$ (`float32`) por cada fila de D

```

1: for cada fila  $d$  de  $D$  do
2:    $x \leftarrow$  vector nulo de dimensión 76
3:    $m \leftarrow$  vector nulo de dimensión 76
4:   for  $i = 1$  hasta 76 do
5:      $c_i \leftarrow$  nombre canónico  $i$ -ésimo de FEATURES_CANON
6:     if existe columna_fuente tal que  $\mu(\text{columna\_fuente}) = c_i$  y columna_fuente  $\in$  columnas( $D$ ) then
7:        $v \leftarrow$  CoerciónNumérica( $d[\text{columna\_fuente}]$ )  $\triangleright$  no numérico  $\rightarrow$  NaN
8:       if  $v$  no es finito (NaN o  $\pm\infty$ ) then
9:          $v \leftarrow 0$ 
10:      end if
11:       $x_i \leftarrow v$ 
12:       $m_i \leftarrow 1$   $\triangleright$  columna fuente mapeada y presente
13:    else
14:       $x_i \leftarrow 0$   $\triangleright$  imputación por ausencia de columna fuente
15:       $m_i \leftarrow 0$ 
16:    end if
17:  end for
18:   $o \leftarrow \text{concat}(x_1, \dots, x_{76}, m_1, \dots, m_{76})$   $\triangleright$  vector float32 de 152 dimensiones
19:  añadir  $o$  a  $O$ 
20: end for
21: return  $O$ 

```

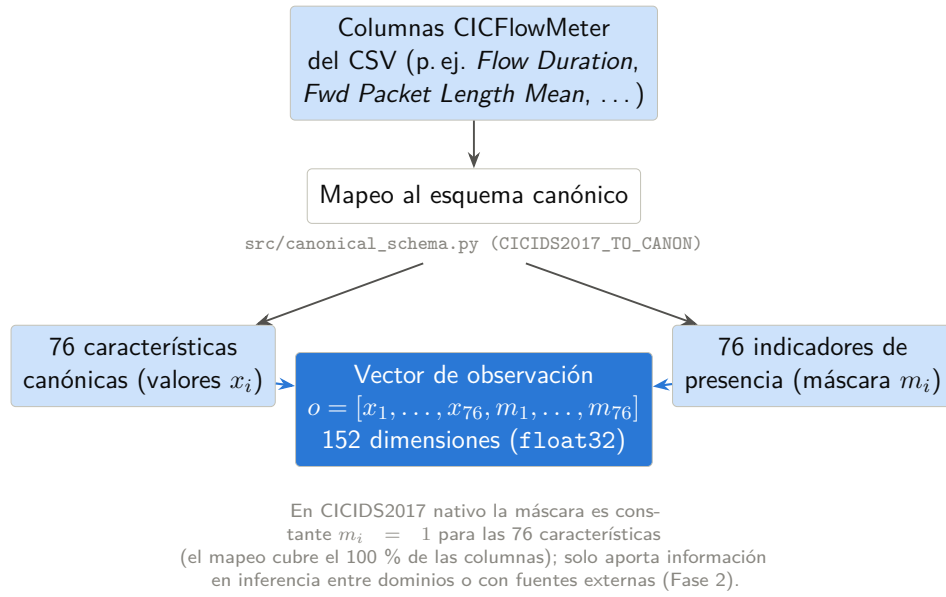


Figura 4.2: Construcción del vector de observación canónico de 152 dimensiones. Fuente: elaboración propia a partir de `src/canonical_schema.py`.

4.5.1 Espacio de observación

El espacio de observación es un espacio vectorial continuo de dimensión 152. Cada observación representa un vector de flujo canónico escalado junto con su máscara de presencia/ausencia. Durante el entrenamiento, las observaciones se extraen de la partición de entrenamiento. Durante la evaluación determinista, el entorno itera a través de la partición de prueba sin barajado, lo que permite comparar las predicciones directamente con las etiquetas verdaderas.

4.5.2 Espacio de acciones

El espacio de acciones es discreto con dos acciones:

- **Acción 0 (PERMIT):** clasificar el flujo como benigno y permitirlo en el modelo de decisión experimental;
- **Acción 1 (BLOCK):** clasificar el flujo como malicioso y bloquearlo o generar una alerta en el modelo de decisión experimental.

El valor de la acción está alineado intencionalmente con el valor de la etiqueta binaria: benigno es 0, ataque es 1. Esto simplifica la evaluación directa porque una acción predicha puede compararse con la etiqueta objetivo sin necesidad de una capa de decodificación adicional.

4.5.3 Función de recompensa

La función de recompensa codifica costes de seguridad asimétricos (Lee et al. 2002; Cárdenas et al. 2006). Un falso negativo significa que se permite un ataque. Un falso positivo significa que se bloquea tráfico benigno. En muchos escenarios defensivos, los ataques no detectados son más graves que los bloqueos innecesarios, aunque la relación exacta depende del contexto operativo.

La implementación actual utiliza la siguiente configuración de recompensa por defecto:

$$TP = 1.5, \quad FP = -2.0, \quad FN = -5.0, \quad TN/\text{omisión} = 0.0.$$

La Tabla 4.2 organiza estos valores como una matriz de decisión para los cuatro resultados posibles.

Decisión del agente	Etiqueta real: Ataque	Etiqueta real: Benigno
BLOCK (Acción 1)	+1.5 (Verdadero Positivo)	-2.0 (Falso Positivo)
PERMIT (Acción 0)	-5.0 (Falso Negativo)	0.0 (Verdadero Negativo)

Tabla 4.2: Matriz de recompensa por defecto para el entorno del defensor binario. Los valores son supuestos experimentales registrados con cada ejecución.

La Figura 4.3 representa el bucle agente–entorno usado en esta formulación. Aunque se utiliza una interfaz de RL estándar, la recompensa depende únicamente de la acción actual y de la etiqueta real del flujo evaluado; con $\gamma = 0$, el objetivo aprendido se reduce a la recompensa inmediata.

La penalización por falso negativo domina la señal de recompensa. Un agente

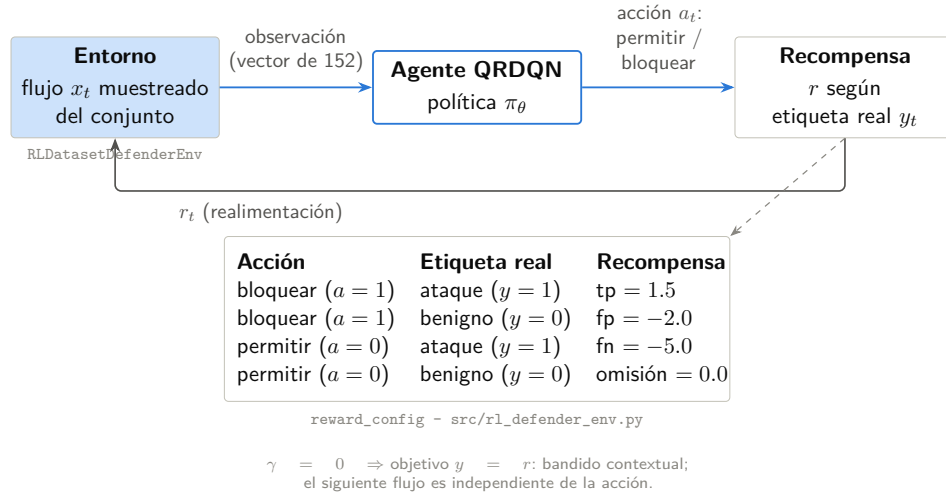


Figura 4.3: Interacción agente-entorno y matriz de recompensa de la decisión PERMIT/BLOCK. Fuente: elaboración propia a partir de `src/rl_defender_env.py` y de la configuración MAIN.

que emite PERMIT en cada observación acumula un retorno muy negativo en tráfico con alta proporción de ataques, mientras que un agente que emite BLOCK en cada observación acumula penalizaciones por falsos positivos sin obtener crédito por detección de ataques. La estructura de recompensa está por tanto diseñada para alejar al agente de políticas constantes degeneradas y conducirlo hacia un comportamiento discriminativo. La recompensa cero para el verdadero negativo es deliberada: recompensar explícitamente cada paso benigno correcto sesgaría al agente hacia maximizar la tasa de permiso en lugar de la calidad de detección. Estos valores son supuestos experimentales, no costes empresariales medidos. Se registran con cada ejecución para que las ejecuciones históricas puedan distinguirse de los valores por defecto actuales.

Los valores de la Tabla 4.2 admiten además una lectura como compromiso multiobjetivo implícito entre dos objetivos operativos, seguridad (minimizar ataques admitidos) e impacto (minimizar el bloqueo de tráfico legítimo), que se resume en la Tabla 4.3.

Resultado	Recompensa	Objetivo operativo	Lectura
FN (ataque permitido)	-5.0	Seguridad	Coste dominante: un ataque queda admitido
FP (benigno bloqueado)	-2.0	Impacto	Penaliza el bloqueo de tráfico legítimo
TP (ataque bloqueado)	+1.5	Seguridad	Crédito positivo por detección correcta de un ataque
TN/omisión (benigno permitido)	0.0	Neutralidad deliberada	Evita sesgar al agente hacia maximizar la tasa de permiso

Tabla 4.3: Lectura multiobjetivo de la función de recompensa del entorno del defensor binario: cada resultado se asocia al objetivo operativo de seguridad o de impacto al que responde. Fuente: elaboración propia a partir de `src/rl_defender_env.py`.

4.5.4 Estructura del episodio

Un episodio es una secuencia de decisiones sobre flujos. Al reiniciar, el entorno inicializa un índice sobre la partición seleccionada. Cuando el barajado está activado, el orden de entrenamiento se aleatoriza. En cada paso, el agente recibe una observación, selecciona `PERMIT` o `BLOCK`, recibe una recompensa calculada a partir de la etiqueta real y la acción, y avanza al flujo siguiente. Un episodio termina cuando se agota el conjunto de datos o se alcanza un número máximo de pasos configurado.

Este diseño proporciona a los algoritmos estándar de RL un flujo de transiciones manteniendo el entrenamiento offline y seguro. Ninguna acción afecta al conjunto de datos fuente, al flujo siguiente ni al entorno de red.

4.5.5 Estado del entorno y protocolo de transición

En cada llamada a `reset()`, el entorno selecciona la partición asignada al modo actual (entrenamiento o evaluación), baraja opcionalmente el índice si el

modo de barajado está activo, reinicia el contador de pasos interno y devuelve la primera observación junto con un diccionario de metadatos. En cada llamada a `step()`, el entorno registra la acción del agente, recupera la etiqueta de verdad del índice de flujo actual, calcula la recompensa escalar a partir de la matriz de recompensa, avanza el índice y devuelve la siguiente observación, la recompensa, un indicador de terminación, un indicador de truncación y el diccionario de metadatos.

La condición de terminación es el agotamiento del conjunto de datos o un número máximo de pasos configurable. En el modo de entrenamiento, el orden barajado garantiza que las transiciones del minibatch muestreen una distribución amplia de tipos de flujo durante un único episodio. En el modo de evaluación, el entorno es determinista: la misma partición recorrida en el mismo orden produce la misma secuencia de observaciones, recompensas y etiquetas. Este determinismo es esencial para la evaluación reproducible porque implica que las diferencias en métricas entre dos modelos guardados reflejan únicamente el comportamiento del modelo y no el ruido de muestreo en el bucle de evaluación.

La separación entre los modos de entrenamiento y evaluación se aplica a nivel del entorno y no se deja al llamante. Pasar el indicador de modo correcto en el momento de la construcción establece tanto el conjunto de índices como el comportamiento de barajado, lo que evita un error frecuente en el que la evaluación hereda accidentalmente el orden barajado de un entorno de entrenamiento.

El Algoritmo 2 muestra el pseudocódigo del paso de decisión `step(a)` tal y como está implementado, que aplica la matriz de recompensas de la Tabla 4.2 a la etiqueta real del índice actual, hace avanzar el puntero de muestras y evalúa de forma independiente las condiciones de terminación (agotamiento del conjunto de índices) y truncación (límite de pasos por episodio).

Algoritmo 2 Paso de decisión del entorno (`env.step`). Fuente: elaboración propia a partir de `src/rl_defender_env.py`.

Require: Índice actual i , contador de pasos t y vector de índices $\mathcal{I}$, inicializados en RESET (con $\mathcal{I}$ barajado si el indicador *shuffle* está activo, según el modo fijado en la construcción del entorno)

```

1: function STEP( $a$ )
2:   if  $a \notin \{0, 1\}$  then
3:     return error (acción inválida)
4:   end if
5:    $idx \leftarrow \mathcal{I}[i]$ 
6:    $y \leftarrow Y[idx]$  ▷ etiqueta real de la muestra actual
7:   if  $y = \text{ataque}$  then
8:     if  $a = 1$  (BLOCK) then
9:        $r \leftarrow tp = 1.5$  ▷ verdadero positivo
10:    else
11:       $r \leftarrow fn = -5.0$  ▷ falso negativo
12:    end if
13:  else ▷  $y = \text{benigno}$ 
14:    if  $a = 0$  (PERMIT) then
15:       $r \leftarrow \text{omission} = 0.0$  ▷ verdadero negativo
16:    else
17:       $r \leftarrow fp = -2.0$  ▷ falso positivo
18:    end if
19:  end if
20:   $i \leftarrow i + 1$ ;  $t \leftarrow t + 1$  ▷ avance del índice y del contador de pasos
21:   $terminated \leftarrow (i \geq N)$  ▷  $N$ : tamaño del conjunto de muestras
22:   $truncated \leftarrow (t \geq T_{\text{máx}})$  ▷  $T_{\text{máx}}$ : máximo de pasos por episodio
23:  if  $terminated \vee truncated$  then
24:     $s' \leftarrow X[idx]$  ▷ se repite la observación de la muestra actual
25:  else
26:     $s' \leftarrow X[\mathcal{I}[i]]$  ▷ observación de la siguiente muestra
27:  end if
28:   $info \leftarrow \{sample\_index : idx, true\_label : y\}$ 
29:  return ( $s', r, terminated, truncated, info$ )
30: end function

```

4.5.6 Limitaciones de la formulación conjunto-de-datos-como-entorno

Dado que el entorno es un conjunto de datos estático, las acciones del agente no afectan a los estados futuros. Esto no es un problema completo de defensa cibernética autónoma en el que bloquear un flujo podría cambiar el comportamiento posterior del atacante, el estado del host, el enrutamiento o el contexto de alertas. La formulación se asemeja más a un problema de decisión contextual sensible al coste que a un proceso de decisión de Markov rico (Sutton et al. 2018; Levine et al. 2020).

Esta limitación es explícita en la metodología. El RL se utiliza aquí para estudiar el aprendizaje de decisiones basadas en valor y sensibles al coste sobre registros de flujo, y para establecer un pipeline offline seguro. Las afirmaciones sobre respuesta adversarial en múltiples pasos, adaptación en línea o defensa cibernética activa quedan fuera del alcance implementado.

4.6 Agente QRDQN

El algoritmo de RL principal es Quantile Regression Deep Q-Network (QRDQN) (Dabney et al. 2018). QRDQN pertenece a la familia del RL distribucional, que modela una distribución sobre los retornos en lugar de únicamente un retorno esperado (Bellemare et al. 2017).

4.6.1 Papel algorítmico

El DQN estándar aproxima los valores-acción con una red neuronal y selecciona acciones según el retorno esperado estimado (Mnih et al. 2015). QRDQN extiende esta idea aprendiendo los cuantiles de la distribución del retorno. Esto es relevante para la tesis porque la estructura de recompensa

es asimétrica: los falsos negativos conllevan una penalización mucho mayor que la recompensa que generan los permisos benignos correctos. Una visión distribucional puede representar más información sobre la variabilidad del retorno que una única estimación de la media.

La metodología no asume que QRDQN sea inherentemente superior a los clasificadores supervisados para datos de flujo tabulares. En cambio, evalúa QRDQN como el representante principal de RL bajo el mismo preprocesamiento, las mismas particiones y las mismas métricas utilizadas por las líneas base.

La arquitectura de red concreta de la ejecución MAIN se resume en la Figura 4.4: una entrada de 152 dimensiones, tres capas densas ocultas y una salida distribucional de 200 cuantiles por cada una de las dos acciones.

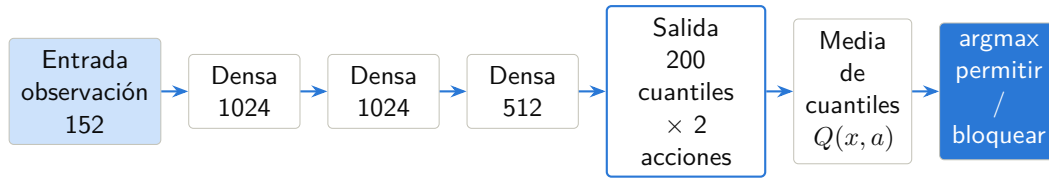


Figura 4.4: Arquitectura QRDQN de la ejecución MAIN: observación canónica, capas densas, cuantiles por acción y decisión final por media de cuantiles. Fuente: elaboración propia a partir de `config.json` de la ejecución MAIN.

4.6.2 Función de pérdida de regresión cuantílica

QRDQN representa la distribución del retorno de acción mediante N estimaciones de cuantiles aprendidas $\{\theta_i(s, a)\}_{i=1}^N$ en puntos medios de cuantil fijos $\hat{\tau}_i = \frac{2i-1}{2N}$ (Dabney et al. 2018). Durante la selección de acción, estos cuantiles se promedian para producir una estimación del retorno esperado, recuperando la regla codiciosa estándar del DQN bajo expectativa.

Para una única transición (s, a, r, s') , sea $a^* = \arg \max_a \sum_j \theta_j^-(s', a)$ la acción codiciosa bajo la red objetivo, y sea

$$\delta_{ij} = r + \gamma \theta_j^-(s', a^*) - \theta_i(s, a)$$

el error de diferencia temporal entre el cuantil fuente i y el cuantil objetivo j calculado mediante bootstrapping. QRDQN minimiza la función de pérdida de regresión cuantílica de Huber sumada sobre todos los N^2 pares de cuantiles:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \left| \hat{\tau}_i - \mathbf{1}[\delta_{ij} < 0] \right| \cdot L_{\kappa}(\delta_{ij}),$$

donde L_{κ} es la pérdida de Huber elemento a elemento con umbral κ :

$$L_{\kappa}(\delta) = \begin{cases} \frac{1}{2}\delta^2 & \text{si } |\delta| \leq \kappa, \\ \kappa\left(|\delta| - \frac{1}{2}\kappa\right) & \text{en caso contrario.} \end{cases}$$

A diferencia de C51, que representa los retornos sobre un soporte categórico fijo, QRDQN no impone ninguna restricción sobre el soporte de la distribución del retorno y se adapta automáticamente a la escala de las recompensas (Bellemare et al. 2017). En el entorno de recompensa asimétrica de esta tesis, la distribución del retorno puede desarrollar colas negativas pronunciadas cuando los eventos de falso negativo son frecuentes, lo que hace que una visión distribucional sea más informativa que una estimación escalar de la media para comprender el comportamiento del agente en torno a la frontera de decisión PERMIT/BLOCK.

4.6.3 Factor de descuento $\gamma = 0$: formulación de un solo paso

En todas las ejecuciones de esta tesis, incluida la ejecución oficial, el factor de descuento se fija en $\gamma = 0$ (valor registrado en el artefacto de configuración de la ejecución). La elección es deliberada y coherente con la formulación conjunto-de-datos-como-entorno: dado que el entorno es un conjunto de datos estático y las acciones del agente no afectan a los estados futuros, no existe dependencia temporal entre flujos que un factor de descuento positivo pudiera explotar.

Con $\gamma = 0$, el error de diferencia temporal definido más arriba se reduce a

$$\delta_{ij} = r - \theta_i(s, a),$$

es decir, el término de bootstrapping $\gamma \theta_j^-(s', a^*)$ se anula y la red objetivo deja de contribuir. Cada cuantil aprendido $\theta_i(s, a)$ regresa por tanto hacia la recompensa inmediata de la decisión, y QRDQN actúa como un estimador distribucional de un solo paso: formalmente, un *bandit contextual sensible al coste*, y no un proceso de decisión de Markov secuencial.

Esta reducción no vacía de contenido la elección de QRDQN. La cabeza distribucional sigue modelando la distribución completa de la recompensa asimétrica en torno a la frontera PERMIT/BLOCK —y no únicamente su media—, lo que aporta información sobre la incertidumbre del valor bajo coste asimétrico. La diferencia frente a un clasificador supervisado no reside en la presencia de dinámica secuencial, inexistente aquí por diseño, sino en que el aprendizaje se realiza mediante valores sobre una matriz de coste explícita (con la penalización de falso negativo muy superior a la de falso positivo) en lugar de mediante entropía cruzada sobre etiquetas. La tesis no reivindica autonomía secuencial ni respuesta adversarial en múltiples pasos: tal y como se recoge en las limitaciones de la formulación, esas capacidades quedan fuera del alcance implementado.

4.6.4 Estrategia de exploración

Durante el entrenamiento, el agente emplea una política ε -greedy. Con probabilidad ε selecciona una acción uniformemente aleatoria de {PERMIT, BLOCK}; con probabilidad $1 - \varepsilon$ selecciona la acción cuya estimación media de cuantiles es más alta (Mnih et al. 2015). ε decae linealmente desde un valor inicial cercano a uno hasta un valor final pequeño a lo largo de una fracción fija del total de pasos de entrenamiento, y luego permanece constante. En el entorno de acción binaria, el presupuesto de exploración es relativamente barato porque

solo existen dos alternativas. La fracción de exploración y el valor final de ε se almacenan en el artefacto de configuración de la ejecución para que las ejecuciones con diferentes programas no se mezclen o comparen en silencio sin tener en cuenta sus diferencias de exploración.

4.6.5 Configuración del entrenamiento

La implementación utiliza el módulo QRDQN de `sb3-contrib` (Stable-Baselines3 Contributors 2023). El modelo emplea una política de perceptrón multicapa sobre las observaciones de 152 dimensiones. El entrenamiento utiliza un buffer de repetición, actualizaciones por minibatch, una red objetivo y predicción determinista durante la evaluación. El script de entrenamiento mantenido admite preajustes rápidos y completos, modos de partición aleatoria y por día, semillas explícitas, límites de filas configurables y pasos temporales configurables.

Los valores por defecto del preajuste completo actual utilizan lotes más grandes y más pasos de gradiente que el preajuste rápido. Esta separación permite al proyecto ejecutar comprobaciones rápidas de humo sin confundirlas con las ejecuciones finales del benchmark. Cualquier resultado reportado debe incluir el identificador exacto de la ejecución y la configuración.

El Algoritmo 3 formaliza este pipeline de entrenamiento de extremo a extremo, desde la carga de los datos brutos de CICIDS2017 hasta la persistencia de los artefactos finales, con independencia del preajuste concreto o de los valores de hiperparámetros seleccionados en cada ejecución. La secuencia de pasos que ejecuta el script mantenido es única; solo los valores de las variables de configuración, documentados aparte, distinguen una ejecución de otra.

Algoritmo 3 Entrenamiento del agente defensor QRDQN sobre CICIDS2017: carga y limpieza de datos, mapeo canónico, partición train/test, ajuste del escalador exclusivamente sobre train, bucle de entrenamiento QRDQN con $\gamma = 0$ y persistencia de artefactos. Fuente: elaboración propia a partir de `src/train_rl_defender.py`.

Require: CSV de CICIDS2017; semilla; preajuste (rápido/completo); modo de partición (aleatorio/día); hiperparámetros QRDQN $\eta, B, G, \tau, \varepsilon_0, \varepsilon_T, f_\varepsilon$

Ensure: Modelo QRDQN, escalador, percentiles de entrenamiento, configuración y métricas de test persistentes

- 1: Fijar la semilla en `numpy/random/torch`; cargar y limpiar los CSV según el preajuste $\triangleright$ sin escalar
 - 2: Mapear a esquema canónico (76 características + máscara de 76) $\rightarrow$ observaciones de 152 dimensiones; particionar en $(X_{\text{train}}, y_{\text{train}}), (X_{\text{test}}, y_{\text{test}})$
 - 3: Calcular percentiles $p_{0.5}, p_{99.5}$ de X_{train} sin escalar; ajustar $\Sigma = \text{StandardScaler}$ solo sobre X_{train} $\triangleright X_{\text{test}}$ no interviene en el ajuste
 - 4: $X_{\text{train}} \leftarrow \Sigma(X_{\text{train}})$; $X_{\text{test}} \leftarrow \Sigma(X_{\text{test}})$; persistir Σ , percentiles, nombres de características y metadatos del entorno
 - 5: Construir el entorno Gymnasium sobre $(X_{\text{train}}, y_{\text{train}})$ con la matriz de recompensa fijada
 - 6: Inicializar red de cuantiles θ , red objetivo $\theta^- \leftarrow \theta$, buffer $\mathcal{D} \leftarrow \emptyset$, $\varepsilon \leftarrow \varepsilon_0$, $t \leftarrow 0$
 - 7: **while** $t < \text{total de pasos temporales}$ **do**
 - 8: Observar s ; con probabilidad ε , $a \leftarrow$ acción aleatoria en $\{\text{PERMIT}, \text{BLOCK}\}$; en caso contrario $a \leftarrow \arg \max_{a'} \frac{1}{N} \sum_i \theta_i(s, a')$
 - 9: Ejecutar a ; observar r, s' , terminado; almacenar $(s, a, r, s', \text{terminado})$ en $\mathcal{D}$; $t \leftarrow t + 1$
 - 10: Actualizar ε por decaimiento lineal de ε_0 a ε_T sobre la fracción f_ε del total de pasos
 - 11: **if** $t \bmod \text{intervalo de red objetivo} = 0$ **then**
 - 12: $\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-$ $\triangleright \tau = 1$: copia dura
 - 13: **end if**
 - 14: **if** $t \bmod \text{train_freq} = 0$ y $t \geq \text{learning_starts}$ **then**
 - 15: **for** $g = 1, \dots, G$ **do**
 - 16: Muestrear de $\mathcal{D}$ un minibatch $\{(s_k, a_k, r_k, s'_k, \text{terminado}_k)\}_{k=1}^B$
 - 17: Para cada k y cada cuantil objetivo $j = 1, \dots, N$: $y_j \leftarrow r_k$ $\triangleright \gamma = 0$: sin bootstrap ni θ^- ; regresión hacia r_k
 - 18: Pérdida $\mathcal{L} \leftarrow$ Huber cuantílica entre $\theta_i(s_k, a_k)$ y $\{y_j\}$, sumada sobre todos los pares (i, j)
 - 19: Actualizar θ por descenso de gradiente sobre $\mathcal{L}$ (Adam, tasa η); recortar norma del gradiente si corresponde
 - 20: **end for**
 - 21: **end if**
 - 22: **end while**
 - 23: Guardar el modelo; evaluar de forma determinista sobre $(X_{\text{test}}, y_{\text{test}})$; calcular matriz de confusión y métricas
 - 24: Persistir configuración final, métricas de test y manifiesto de artefactos
-

4.6.6 Procedencia de los hiperparámetros

Los hiperparámetros de la ejecución oficial ($\gamma = 0$, arquitectura [1024, 1024, 512], 200 cuantiles, tasa de aprendizaje 5×10^{-5} , 20 pasos de gradiente y 3 000 000 de pasos de entrenamiento) constituyen un *perfil fijo establecido manualmente*, no el resultado de una búsqueda automática. El repositorio incluye una sonda de optimización con Optuna (`src/tune_hparams.py`) con fines exploratorios, pero su espacio de búsqueda — $\gamma \in [0,95, 0,999]$, arquitecturas en $\{[256, 128], [512, 256], [256, 256]\}$ y pasos de gradiente en $\{10, 50, 100\}$ — *no contiene* la configuración oficial. Por tanto, dicha configuración no pudo originarse en esa búsqueda y no debe interpretarse como un óptimo ajustado por Optuna; responde a un diseño manual coherente con la formulación de un solo paso, y queda congelada y verificada por las pruebas del proyecto.

El listado completo de valores empleados en la ejecución oficial MAIN se recoge en la Tabla 4.4.

4.6.7 Persistencia del modelo y el preprocesamiento

El entrenamiento produce un directorio de ejecución y un artefacto de modelo. El directorio de ejecución almacena la configuración, las métricas, el escalador ajustado y los límites de percentiles de entrenamiento. Persistir los artefactos de preprocesamiento es esencial porque la inferencia en la Fase 2 debe utilizar el mismo estado de normalización que el entrenamiento. Recalcular un escalador sobre el tráfico de laboratorio filtraría información de la distribución de inferencia hacia la interfaz del modelo y haría la ejecución irreproducible.

Hiperparámetro	Valor	Observación
Arquitectura de la red (<code>net_arch</code>)	[1024, 1024, 512]	MLP de tres capas ocultas
Número de cuantiles (<code>n_quantiles</code>)	200	distribución de retorno QRDQN
Tasa de aprendizaje	5×10^{-5}	
Tamaño del buffer de repetición	1 000 000	transiciones
Pasos de calentamiento (<code>learning_starts</code>)	50 000	
Tamaño de lote (<code>batch_size</code>)	2048	
Factor de descuento (γ)	0.0	formulación de un solo paso
Tasa de actualización objetivo (τ)	1.0	con $\gamma = 0$ la red objetivo no contribuye al error de diferencia temporal
Intervalo de actualización objetivo	10 000	pasos
Frecuencia de entrenamiento (<code>train_freq</code>)	100	pasos
Pasos de gradiente por actualización	20	
ϵ inicial de exploración	1.0	decaimiento lineal
ϵ final de exploración	0.02	decaimiento lineal
Fracción de exploración	0.1	decaimiento lineal
Norma máxima de gradiente	10.0	
Pasos totales de entrenamiento	3 000 000	
Semilla	42	

Tabla 4.4: Hiperparámetros de la ejecución oficial MAIN. Perfil fijado a mano, no procedente de la sonda Optuna. Fuente: elaboración propia a partir de `runs/cicids2017/MAIN_qrdqn_cicids2017_canonical_full_random_20260609_193655/config.json`.

4.7 Marco de implementación

El pipeline experimental está implementado en Python y se apoya en un conjunto de bibliotecas de código abierto bien establecidas. El componente de RL profundo principal utiliza la extensión `sb3-contrib` de Stable Baselines3, que proporciona una implementación contrastada de QRDQN con arquitectura de red configurable, buffer de repetición, programa de exploración y actualizaciones de la red objetivo (Stable-Baselines3 Contributors 2023). El envoltorio del entorno sigue la API de Gymnasium (Farama Foundation 2023), haciendo que el conjunto de datos de flujos sea compatible con cualquier bucle de entrenamiento que cumpla con Gymnasium sin cambios en la implementación del algoritmo.

4.7.1 Pila de procesamiento de datos y aprendizaje supervisado

La carga y el preprocesamiento de datos se apoyan en `pandas` para la ingesta de CSV, la estandarización de columnas y la lectura fragmentada de ficheros grandes. Las matrices de características se convierten a arrays de `numpy` y se castean a `float32` antes de entrar en el entorno de Gymnasium o en la línea base supervisada. La estandarización utiliza `StandardScaler` de `scikit-learn`, elegido por su compatibilidad con el protocolo de serialización de `sklearn`, que permite persistir y recargar los escaladores ajustados sin reimplementación. La línea base de Random Forest también utiliza `scikit-learn`, consumiendo los mismos arrays de entrada que entran en el entorno de RL. Esta ruta de datos compartida garantiza que cualquier diferencia de rendimiento entre los dos modelos refleje el comportamiento algorítmico y no una discrepancia en el manejo de los datos.

4.7.2 Gestión de artefactos

Cada ejecución de entrenamiento produce un `RUN_ID` único compuesto por una marca temporal y un sufijo aleatorio corto. El directorio de ejecución reúne la política QRDQN serializada, el escalador ajustado, los límites de percentiles, un JSON de configuración y un JSON de métricas. Las ejecuciones de validación y de inferencia de la Fase 2 producen subdirectorios independientes con sus propios registros de configuración y métricas. Este esquema de directorios plano evita dependencias de bases de datos al tiempo que hace las ejecuciones recuperables de forma independiente y comparables mediante inspección del sistema de ficheros.

4.7.3 Controles de reproducibilidad

Las semillas se fijan para los generadores de números aleatorios de `numpy`, `random` y `torch` al comienzo de cada ejecución. Los reinicios del entorno de `Gymnasium` también reciben una semilla explícita. El pipeline está preparado para que, cuando se utilicen múltiples semillas para la estimación de la varianza, cada semilla genere un directorio de ejecución independiente de modo que los resultados por semilla se preserven individualmente en lugar de promediarse en silencio; no obstante, los resultados reportados en esta tesis corresponden a una única semilla y no incluyen una estimación de varianza entre ejecuciones. Estos controles buscan reducir la varianza interna a la ejecución lo suficiente como para distinguir diferencias de rendimiento significativas del ruido, aunque no pueden eliminar todo el indeterminismo derivado del ordenamiento de los kernels de la GPU.

Protocolo experimental

5.1 Fases experimentales

La metodología experimental se divide en dos fases. La primera fase comprueba el aprendizaje y la generalización interna sobre un benchmark público. La segunda fase comprueba si el pipeline de preprocesamiento y modelo resultante es capaz de procesar tráfico capturado por el operador en un laboratorio doméstico cerrado bajo cambio de dominio.

5.1.1 Fase 1: benchmark interno de CICIDS2017

La Fase 1 utiliza CICIDS2017 como benchmark interno principal (Sharafaldin et al. [2018](#); Canadian Institute for Cybersecurity [2017](#)). CICIDS2017 es adecuado para esta tesis porque proporciona tráfico benigno y de ataque etiquetado, capturas de paquetes en bruto y CSVs de flujos derivados de una extracción al estilo CICFlowMeter (Habibi Lashkari et al. [2017](#)). También contiene múltiples escenarios de ataque en diferentes días de captura, lo que permite diseños de partición más estrictos que una única partición aleatoria entrenamiento-prueba.

La fase no se trata como un sustituto limpio de un entorno de producción. CICIDS2017 tiene problemas conocidos que incluyen flujos duplicados o inconsistentes, errores de etiquetado y de extracción, y sensibilidad a la partición (Engelen et al. 2021; Lanvin et al. 2023; L. Liu et al. 2022). Por esa razón, la Fase 1 se enmarca como un benchmark interno con controles explícitos. El pipeline elimina los campos de tipo identificador, ajusta la normalización únicamente sobre los datos de entrenamiento, registra las configuraciones de ejecución y evalúa el modelo mediante una escalera de validación. Una puntuación elevada en la partición aleatoria se interpreta como evidencia de que el modelo es capaz de aprender la tarea del benchmark; no se interpreta como preparación para el despliegue.

La estructura por día de captura de CICIDS2017 (Tabla 5.1) muestra una proporción de ataque muy desigual entre ficheros, desde el 0 % del lunes hasta más del 55 % en las capturas del viernes por la tarde. Esta heterogeneidad motiva los protocolos de partición por día, más estrictos que una partición aleatoria, que se describen más adelante en este capítulo.

Día	Filas	Benigno	Ataque	Ataque (%)
Lunes	529 918	529 918	0	0.0 %
Martes	445 909	432 074	13 835	3.1027 %
Miércoles	692 703	440 031	252 672	36.4762 %
Jueves (mañana, ataques web)	170 366	168 186	2 180	1.2796 %
Jueves (tarde, infiltración)	288 602	288 566	36	0.0125 %
Viernes (mañana)	191 033	189 067	1 966	1.0291 %
Viernes (tarde, PortScan)	286 467	127 537	158 930	55.4793 %
Viernes (tarde, DDoS)	225 745	97 718	128 027	56.7131 %

Tabla 5.1: Composición por día de CICIDS2017: número de filas y proporción de tráfico benigno frente a ataque por fichero oficial de captura. Fuente: elaboración propia a partir de `memoria/figuras/data_composicion_dia.json`.

La Figura 5.1 ofrece la misma información como comparación visual entre días. La lectura relevante para el protocolo no es solo el volumen total, sino la variación extrema de prevalencia de ataque entre capturas, que justifica no

depender únicamente de una partición aleatoria por filas.

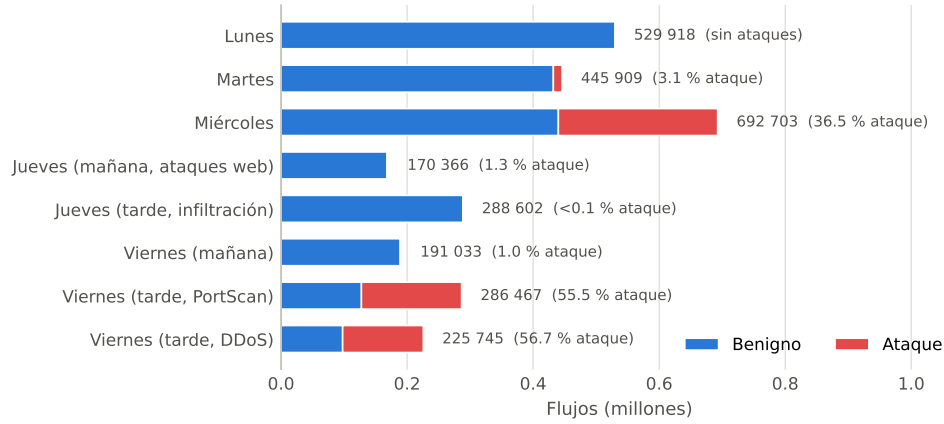


Figura 5.1: Composición visual por día de CICIDS2017. Fuente: elaboración propia a partir de `memoria/figuras/data_composicion_dia.json`.

5.1.2 Fase 2: inferencia offline sobre tráfico de laboratorio

La Fase 2 evalúa la transferencia más allá de la distribución del benchmark. El tráfico se captura en un entorno de laboratorio privado, se convierte offline en CSVs de flujos, se mapea al mismo esquema canónico, se normaliza con los artefactos de entrenamiento persistidos y se procesa con el modelo entrenado. Esta fase es una prueba de inferencia sobre una distribución externa, no un despliegue en vivo.

El propósito metodológico de la Fase 2 es exponer el cambio de dominio. Los estudios públicos sobre NIDS suelen reportar resultados sólidos dentro del mismo conjunto de datos que se degradan cuando el modelo se evalúa sobre tráfico producido por redes, herramientas de captura, periodos temporales o definiciones de características diferentes (Apruzzese et al. 2022; Layeghy et al. 2023). La Fase 2 se centra por tanto en la inferencia offline reproducible, los diagnósticos y el seguimiento de artefactos. No garantiza un rendimiento estable en redes empresariales arbitrarias y no implementa bloqueo activo en tiempo real.

5.2 Separación entrenamiento-prueba y escalado

La metodología separa el ajuste y la evaluación en cada etapa de preprocesamiento que pueda aprender de los datos. Esto es necesario porque la fuga de información puede producirse no solo a través de columnas, sino también a través de estadísticas como medias globales, varianzas, percentiles y distribuciones de clases.

5.2.1 Partición aleatoria estratificada

La partición aleatoria estratificada es el primer peldaño de validación. Preserva las proporciones de clases entre entrenamiento y prueba y es útil como comprobación de la capacidad de aprendizaje. Si el modelo no puede aprender bajo esta partición favorable, las evaluaciones más estrictas posteriores tienen pocas probabilidades de tener éxito.

En la ejecución MAIN, la estratificación mantiene prácticamente idéntica la prevalencia de ataque entre entrenamiento y prueba, como muestra la Figura 5.2. Esto permite atribuir los cambios de rendimiento entre entrenamiento y evaluación al modelo y al protocolo, no a una variación accidental de la proporción de clases en la partición aleatoria.

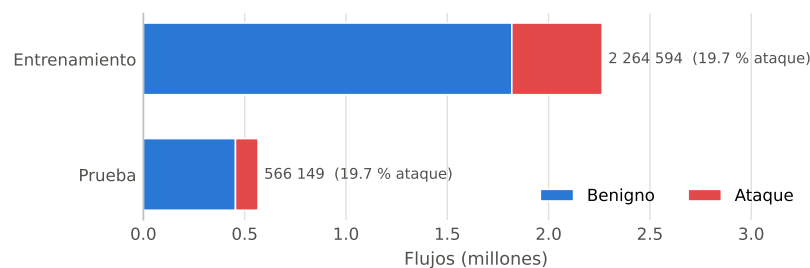


Figura 5.2: Balance de clases en la partición aleatoria fija de la ejecución MAIN. Fuente: elaboración propia a partir de `config.json` de la ejecución MAIN.

Sin embargo, la partición aleatoria por filas es la evidencia más débil de este proyecto. Los conjuntos de datos de flujos pueden contener registros correlacionados, patrones repetidos y artefactos del contexto de captura. Situar flujos adyacentes o muy similares a ambos lados de la partición puede sobreestimar la generalización (Lanvin et al. 2023; Arp et al. 2020). Los resultados de la partición aleatoria se reportan por tanto únicamente como evidencia del benchmark interno.

5.2.2 Partición por CSV/día

La partición por CSV/día entrena sobre ficheros seleccionados de CICSIDS2017 y evalúa sobre ficheros diferentes. Esta partición comprueba si el modelo se transfiere entre días de captura y contextos de ataque. En la implementación mantenida, la partición por día por defecto entrena sobre los patrones del lunes, martes y miércoles, y prueba sobre los patrones del jueves y viernes. Se trata de una configuración más estricta que la partición aleatoria porque ficheros enteros quedan fuera del entrenamiento.

La partición por día también se alinea con la estructura conocida de CICSIDS2017: diferentes días contienen diferentes cargas de trabajo benignas y escenarios de ataque. Por ello es más adecuada para revelar si el modelo ha aprendido un comportamiento de flujo general o artefactos específicos de un día.

5.2.3 Partición leave-one-CSV-out

El protocolo interno más estricto es la validación leave-one-CSV-out. Cada fold retiene un CSV oficial de CICSIDS2017 como datos de prueba y entrena con los CSV restantes. El script de validación implementado agrega métricas por fold, incluyendo exactitud, exactitud balanceada, precisión de ataque, recall de ataque, F1 de ataque, precisión de benigno, recall de benigno, tasa de falsos

positivos, tasa de falsos negativos, tasa de bloqueo, retorno total y costes temporales.

Este protocolo es metodológicamente importante porque trata cada fichero de captura como una unidad de tipo dominio. Es más sólido que una partición por día basada en subcadenas porque utiliza los nombres de fichero CSV oficiales exactos y evita solapamientos accidentales. En el momento de redacción de este capítulo, la implementación existe, mientras que las métricas agregadas finales solo deben reportarse cuando se disponga de un artefacto de ejecución comprometido.

5.2.4 Estandarización ajustada sobre entrenamiento

El entrenamiento de redes neuronales es sensible a la escala de las características. El pipeline utiliza por tanto normalización mediante `StandardScaler`, ajustando el escalador únicamente sobre la partición de entrenamiento y aplicando la transformación ajustada a los datos de prueba. El mismo principio se aplica a los artefactos de inferencia persistidos: el escalador guardado durante el entrenamiento se reutiliza en la Fase 2, en lugar de recalcularse a partir de los flujos de laboratorio.

Para la inferencia robusta en la Fase 2, el pipeline de entrenamiento también persiste los límites de percentiles sobre las características canónicas en bruto. El script de inferencia de la Fase 2 puede aplicar recorte por percentiles sobre las características en bruto y recorte por z-score después del escalado, y luego exportar diagnósticos. Estos controles no se presentan como una solución al cambio de dominio; son salvaguardas que hacen visibles y acotados los valores extremos fuera de distribución durante la inferencia offline.

5.3 Línea base supervisada

Un método de RL para NIDS a nivel de flujo debe compararse con líneas base supervisadas sólidas. Los datos de flujo tabulares son un entorno donde los modelos basados en árboles, especialmente los Random Forests y los ensamblados con gradient boosting, suelen ser competitivos (H. Liu et al. 2019; Apruzzese et al. 2022).

5.3.1 Línea base con Random Forest

La línea base supervisada principal es un clasificador Random Forest. La línea base es metodológicamente útil porque obtiene buenos resultados en datos tabulares, gestiona las interacciones no lineales entre características y requiere menos complejidad de entrenamiento que el RL profundo. También proporciona una comprobación de cordura frente a la sobreestimación del valor de la formulación de RL en una tarea que puede comportarse como una clasificación sensible al coste.

5.3.2 Comparación bajo el mismo protocolo

La línea base debe consumir la misma representación canónica de observaciones, utilizar los mismos protocolos de partición y reportar las mismas métricas que QRDQN. Esta regla de mismo protocolo evita comparaciones sesgadas. Si la línea base utiliza un preprocesamiento diferente, controles de fuga distintos o particiones más sencillas, la comparación no aísla la contribución algorítmica de QRDQN.

La metodología trata por tanto la comparación con la línea base como un requisito del protocolo y no como un apéndice opcional. También evita afirmar la superioridad de QRDQN hasta que existan artefactos de línea base con la misma partición.

5.3.3 Desbalanceo de clases y ponderación sensible al coste

CICIDS2017 presenta desbalanceo de clases: los flujos benignos superan sustancialmente en número a los flujos de ataque, y dentro de los flujos de ataque las familias individuales varían en prevalencia. Un Random Forest entrenado con pesos de clase uniformes por defecto tenderá por tanto a sesgar sus predicciones hacia la clase benigna mayoritaria. Para que la comparación sea significativa bajo los supuestos de coste asimétrico de la tesis, la línea base de Random Forest utiliza ponderación de clases balanceada, donde cada clase recibe un peso inversamente proporcional a su frecuencia en la partición de entrenamiento. Esto garantiza que el modelo supervisado no sea penalizado en la misma dirección que la función de recompensa de RL por razones diferentes.

La configuración de los pesos de clase se registra en el artefacto de configuración de la línea base junto al modo de partición, el esquema de características y la semilla de evaluación, de modo que el supuesto de ponderación es trazable. Si la tesis encuentra que el Random Forest con ponderación balanceada alcanza un rendimiento en falsos negativos similar o superior al de QRDQN bajo la recompensa asimétrica, esa comparación es informativa precisamente porque ambos modelos han tenido acceso a mecanismos de sensibilidad al coste comparables. Comparar un QRDQN ajustado al coste frente a un Random Forest sin corrección no aportaría esta evidencia.

5.4 Protocolo de escala de entrenamiento y eficiencia de datos

La metodología contempla un componente de escala de entrenamiento, planificado pero aún no ejecutado en esta tesis, motivado por una pregunta

práctica: ¿cuántos datos de flujo etiquetados necesita cada enfoque antes de alcanzar un rendimiento estable? Esto es relevante porque el tráfico de ataque etiquetado es costoso de obtener, la abundancia de benchmarks no refleja la disponibilidad en el despliegue, y los métodos de RL profundo pueden ser sustancialmente más exigentes en cuanto a muestras que los clasificadores supervisados basados en árboles (Sutton et al. 2018; Ring et al. 2019).

5.4.1 Protocolo de curva de aprendizaje

El protocolo de curva de aprendizaje, planificado pero todavía no ejecutado en esta tesis (sin artefactos comprometidos), evaluaría tanto el agente QRDQN como la línea base de Random Forest con presupuestos de entrenamiento progresivamente mayores, todos extraídos de la misma partición estratificada. El conjunto de evaluación se mantendría constante en todos los presupuestos como la partición de prueba completa, de modo que los cambios en las métricas reflejarían únicamente el tamaño de los datos de entrenamiento y no las condiciones de evaluación. Las métricas se registrarían en cada nivel de presupuesto utilizando el mismo código de evaluación que las ejecuciones principales del benchmark.

En este protocolo, cada presupuesto utilizaría un escalador ajustado de nuevo calculado únicamente a partir de las filas incluidas en ese subconjunto de entrenamiento. Los presupuestos más grandes se beneficiarían por tanto de una normalización más representativa sin contaminar la evaluación fija de prueba. Esto significa que los artefactos de preprocesamiento producidos en cada nivel de presupuesto serían distintos e independientemente reproducibles.

5.4.2 Niveles de presupuesto y comparación

Los niveles de presupuesto candidatos incluyen 100 k, 250 k, 500 k, 1 M y 2 M registros de flujo. No todos los niveles son alcanzables bajo todos los modos

de partición; cuando una partición no produce conjuntos de entrenamiento del tamaño objetivo, se registra el tamaño disponible más cercano. El presupuesto final es la partición de entrenamiento completa bajo la partición seleccionada.

Comparar QRDQN y Random Forest en cada nivel de presupuesto revela si la formulación de RL tiene un requisito de muestras o una trayectoria de convergencia diferente. Si el Random Forest alcanza una meseta de rendimiento estable con sustancialmente menos filas, esto constituye evidencia interna de que la tarea del benchmark no requiere la complejidad de muestras adicional del RL profundo sobre el esquema de características actual. Por el contrario, si el comportamiento de RL sensible al coste mejora desproporcionadamente a medida que crece el presupuesto, los resultados de escala de entrenamiento pueden identificar a qué escala la estructura de recompensa distribucional comienza a aportar una ventaja medible. Cuando este protocolo se ejecute, todos los resultados de escala de entrenamiento se almacenarían bajo el mismo esquema de artefactos de ejecución que los experimentos principales, etiquetados por nivel de presupuesto y semilla, de modo que permanecerían recuperables de forma independiente y comparables en experimentos futuros. En el momento de redacción de este capítulo no se ha comprometido ningún artefacto de curva de aprendizaje.

Figura pendiente

Curvas de aprendizaje de QRDQN y Random Forest por presupuesto de entrenamiento

Figura 5.3: Pendiente: Curvas de aprendizaje de QRDQN y Random Forest por presupuesto de entrenamiento.

5.5 Salidas de evaluación y reproducibilidad

La reproducibilidad es una restricción de diseño fundamental, especialmente porque la investigación en seguridad con ML suele adolecer de código faltante, detalles de procesamiento de datos incompletos y registros de evaluación incompletos (Olszewski et al. 2023).

5.5.1 Métricas

Las métricas binarias principales son exactitud, precisión, recall y F1 tanto para la clase benigna como para la de ataque. El análisis también rastrea los conteos de la matriz de confusión: verdaderos positivos, falsos positivos, falsos negativos y verdaderos negativos. Estos conteos son necesarios porque dos modelos con exactitud agregada similar pueden tener implicaciones de seguridad muy diferentes.

La tasa de falsos positivos y la tasa de falsos negativos son especialmente importantes. La tasa de falsos positivos mide la proporción de tráfico benigno bloqueado incorrectamente, lo que se relaciona con la disponibilidad y la carga de trabajo del analista. La tasa de falsos negativos mide la proporción de ataques permitidos incorrectamente, lo que se relaciona directamente con el riesgo de intrusiones no detectadas. El análisis de precisión y recall es particularmente relevante bajo desbalanceo de clases, donde la exactitud sola puede ser engañosa (Fawcett 2006; Saito et al. 2015).

5.5.2 Escalera de validación

La escalera de evaluación está ordenada de la evidencia más débil a la más sólida:

1. partición aleatoria estratificada, utilizada como comprobación de la capa-

- cidad de aprendizaje;
2. comprobación de predicción directa, que verifica las salidas del modelo frente a las etiquetas de prueba sin depender de los metadatos del entorno;
 3. comprobación de etiquetas barajadas, que ayuda a detectar fugas de información confirmando que el rendimiento se degrada cuando las etiquetas de entrenamiento se aleatorizan;
 4. partición por CSV/día, que retiene grupos de captura completos;
 5. validación leave-one-CSV-out, que retiene cada CSV oficial por turnos;
 6. inferencia sobre tráfico de laboratorio en la Fase 2, que prueba el pipeline entrenado sobre tráfico capturado por el operador en un laboratorio doméstico cerrado.

En la implementación, los peldaños segundo a cuarto se corresponden con las comprobaciones *Check A* (predicción directa), *Check B* (etiquetas barajadas) y *Check C* (partición por CSV/día) del script `src/validate_checks.py`; el quinto peldaño se implementa en `src/validate_leave_one_csv_out.py`.

La Figura 5.4 sintetiza esta escalera de validación y sitúa cada peldaño junto al artefacto o resultado que lo respalda. La figura no añade métricas nuevas: reordena visualmente las cifras ya documentadas para hacer explícito el aumento progresivo de exigencia.

Cada peldaño apunta a un modo de fallo diferente. La escalera no hace que ningún resultado individual sea definitivo, pero hace la evaluación más informativa que una única puntuación aleatoria de entrenamiento-prueba.

La partición aleatoria estratificada comprueba si el modelo y la implementación pueden resolver el benchmark en absoluto. Un modelo que falla aquí tiene un problema de entrenamiento o de arquitectura, no un problema de generalización. La comprobación de etiquetas barajadas es un detector de fugas: un modelo que alcanza una exactitud muy por encima de la de la clase

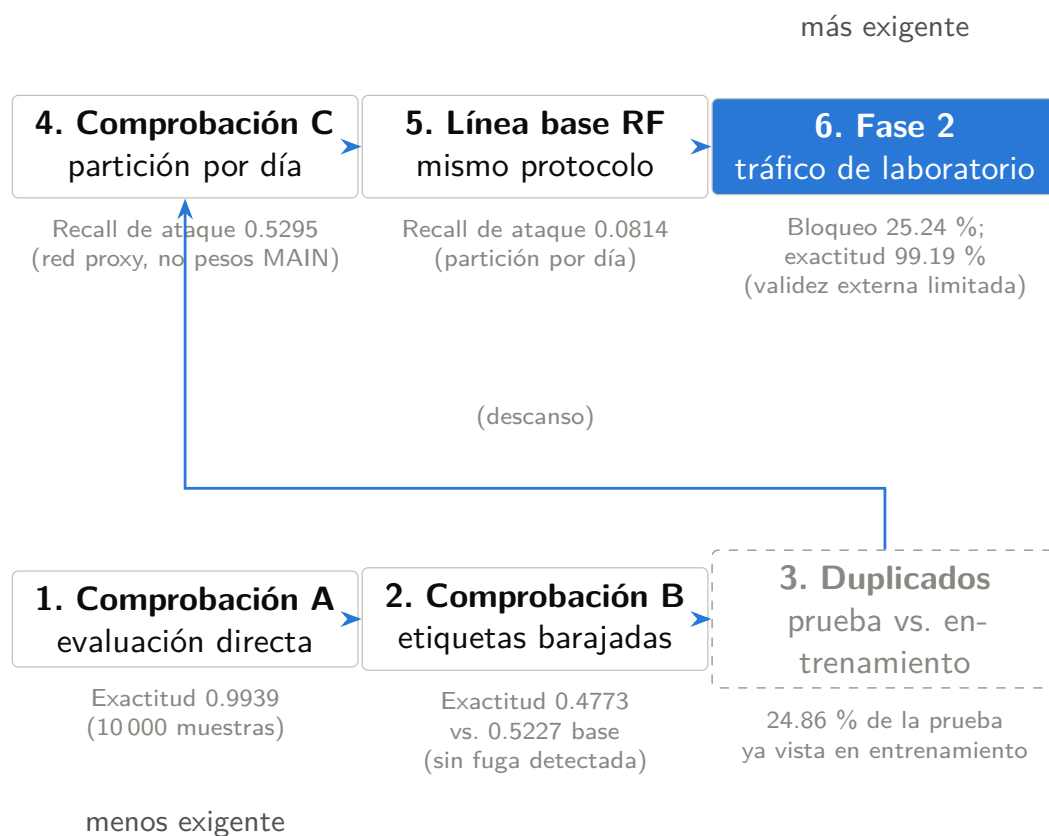


Figura 5.4: Escalera de validación del proyecto, desde la evaluación directa hasta la inferencia de Fase 2. Fuente: elaboración propia a partir de los artefactos de validación y Fase 2 bajo runs/.

mayoritaria en una ejecución con etiquetas barajadas ha aprendido a partir de artefactos correlacionados con las etiquetas en las características y no a partir de estadísticas de flujo reales (Arp et al. 2020). La partición por CSV/día comprueba la generalización temporal y por escenario reteniendo grupos de captura completos; la degradación del rendimiento respecto a la partición aleatoria revela si el modelo depende del contexto específico del día. El protocolo leave-one-CSV-out va más lejos: cada fichero oficial de CICIDS2017 se retiene por turnos, y la distribución de métricas agregada muestra si algún fichero domina el benchmark o si el rendimiento es estable entre escenarios de captura. La inferencia en la Fase 2 es el peldaño final y más estricto, que comprueba si todo el pipeline puede operar sobre tráfico producido fuera del entorno del benchmark.

5.5.3 Artefactos de ejecución

Toda ejecución de entrenamiento o evaluación significativa debe persistir un `RUN_ID` único, un fichero de configuración, métricas y cualquier estado de preprocesamiento necesario para su reutilización exacta. Las ejecuciones de entrenamiento persisten los pesos del modelo y los artefactos del escalador. Las ejecuciones de validación persisten los resultados de validación. Las ejecuciones de inferencia de la Fase 2 persisten las predicciones, las métricas, la configuración y los diagnósticos opcionales.

Esta disciplina de artefactos evita afirmaciones ambiguas. Un resultado solo se trata como medido cuando puede vincularse a una carpeta de ejecución y una configuración específicas. Las ejecuciones históricas no se fusionan en silencio con los valores por defecto actuales; los valores de recompensa, la lógica de partición, el preprocesamiento, los límites de filas y las semillas deben leerse del artefacto.

La Tabla 5.2 recoge el inventario completo de las ejecuciones oficiales – organizadas por el peldaño de la escalera de validación o el análisis que respaldan–

junto con los artefactos concretos que permiten reproducir o auditar cada cifra citada en esta memoria. El Anexo D amplía este resumen con las rutas exactas, la función de cada fichero y su condición de rastreado o local. Las ejecuciones no listadas en la tabla (variantes históricas, sondeos previos al diseño o repeticiones posteriores con configuración distinta) se consideran no oficiales y no deben citarse como respaldo de ninguna cifra reportada.

5.6 Pipeline de inferencia offline en la Fase 2

El punto de entrada principal de la Fase 2 mantenido es el script de inferencia offline robusta. Acepta un CSV de flujos extraído, un modelo entrenado, un escalador persistido, límites de percentiles opcionales y recorte por z-score opcional. A continuación mapea las columnas de entrada al esquema canónico, aplica la misma representación de características y máscara, escala con el escalador guardado, calcula diagnósticos, predice acciones en lotes y escribe las salidas en un nuevo directorio de ejecución de la Fase 2.

La Figura 5.5 resume el recorrido operativo de esa inferencia. La clave metodológica es que el escalador de MAIN se reutiliza sin reajuste y que las salidas son ficheros de predicción, métricas y diagnósticos: no existe actuación activa sobre la red.

El Algoritmo 4 formaliza el orden exacto de estas etapas, indicando cuáles son opcionales y bajo qué indicador de línea de comandos se activan.

5.6.1 Mapeo canónico para flujos de laboratorio

Los extractores de flujos de laboratorio pueden utilizar nombres de columna diferentes a los de CICIDS2017. El script de la Fase 2 define por tanto un mapeo explícito de los nombres del extractor a los nombres canónicos. Las columnas de metadatos como la IP de origen, la IP de destino, los puertos, el

Ejecución (RUN_ID)	Propósito	Artefactos clave
MAIN_qrdqn_cicids2017_canonical_full_random_20260609_193655	Entrenamiento principal QRDQN y evaluación en la partición aleatoria estratificada fija (semilla 42).	config.json; metrics.json; modelo entrenado en models/ (mismo RUN_ID); scaler.joblib; train_percentiles.npz; feature_names.json; artifact_manifest.json
VAL_checks_A_20260212_235443	Comprobación A: predicción directa sin depender de los metadatos del entorno.	config.json; validation_results.json
VAL_checks_B_20260212_235736	Comprobación B: etiquetas barajadas (detección de fugas de información).	config.json; validation_results.json
VAL_checks_C_20260213_004847	Comprobación C: partición por día.	config.json; validation_results.json
bootstrap_ci_seed42.json	Intervalos de confianza bootstrap (10 000 remuestreos estratificados) sobre las métricas de MAIN en la partición de prueba fija, sin reentrenamiento.	bootstrap_ci_seed42.json (incluye los conteos de la matriz de confusión y la verificación del hash SHA-256 del modelo)
duplicate_analysis_seed42.json	Análisis de duplicados y de fuga cruzada entre entrenamiento y prueba sobre la partición aleatoria fija.	duplicate_analysis_seed42.json
rf_cicids2017_canonical_20260628_024735	Línea base supervisada Random Forest (balanceada y escalada) sobre tres particiones: aleatoria, por día y leave-one-out (miércoles).	config.json + metrics.json por subejecución: __random_split, __day_split, __leave_one_out_wednesday
P2v2_pred_20260610_161231_MAIN	Inferencia de la Fase 2 sobre tráfico de laboratorio etiquetado, con el modelo MAIN.	config.json; metrics.json; diagnostics.json; muestra rastreada predictions_head_10000.csv; predictions.csv.gz completo, local y no rastreado

Tabla 5.2: Inventario de las ejecuciones oficiales cuyos artefactos respaldan las cifras presentadas en esta memoria: entrenamiento y evaluación principal (MAIN), escalera de validación (comprobaciones A, B y C), análisis estadísticos sobre la partición fija (intervalos de confianza bootstrap y análisis de duplicados), el barrido de la línea base Random Forest sobre las tres particiones consideradas, y la inferencia oficial de la Fase 2 sobre tráfico de laboratorio etiquetado. Fuente: elaboración propia a partir de runs/.

Algoritmo 4 Inferencia offline robusta de la Fase 2. Fuente: elaboración propia a partir de `scripts/predict_real_traffic_v2.py`.

Require: CSV de flujos D (`-flows`); modelo entrenado M ; escalador persistido S (omitido si `-no-scale`); límites de percentiles opcionales (p_{low}, p_{high}) (`-percentiles`); umbral de recorte z opcional τ_z (`-clip-z`)

Ensure: Directorio de ejecución `runs/phase2/RUN_ID` con predicciones, configuración, métricas y diagnósticos opcionales

```

1: RUN_ID  $\leftarrow$  "P2v2_pred_" + marca de tiempo actual
2: Resolver rutas confiables de  $M$ ,  $S$  y de los percentiles frente al manifiesto de -run-dir, salvo que se indique -allow-unsafe-artifacts
3:  $D \leftarrow$  cargar el CSV de flujos
4: Separar de  $D$  las columnas de metadatos (IP, puertos, protocolo, marca temporal) y las columnas de verdad (truth_y, truth_label)
5: if la mediana de flow_duration en  $D$  es menor que 1 then
6:   Convertir las columnas temporales de segundos a microsegundos ( $\times 10^6$ )
7: end if
8:  $(X, \text{mask}) \leftarrow$  mapear las columnas de  $D$  al esquema canónico de 76 características, imputando con 0 las ausentes
9:  $X \leftarrow [X \mid \text{mask}]$   $\triangleright$  vector combinado de 152 dimensiones
10: if se proporcionaron límites de percentiles (-percentiles) then
11:   Recortar las 76 características crudas de  $X$  a  $[p_{low}, p_{high}]$  (percentiles de entrenamiento p0.5 / p99.5)
12: end if
13: if no se indicó -no-scale then
14:    $X \leftarrow S.\text{transform}(X)$   $\triangleright$  escalador persistido de entrenamiento
15: end if
16: if se proporcionó  $\tau_z$  (-clip-z) then
17:    $X \leftarrow \text{clip}(X, -\tau_z, \tau_z)$   $\triangleright$  recorte tras el escalado
18: end if
19: Calcular diagnósticos de cambio de distribución sobre  $X$ :  $z$  absoluto máximo,  $z$  absoluto medio, proporción con  $|z| > 10$  y características más desviadas
20: if -export-diagnostics then
21:   Guardar los diagnósticos en diagnostics.json
22: end if
23: Cargar  $M$  (QRDQN, con reserva a DQN solo si sb3_contrib no está instalado)
24:  $y_{pred} \leftarrow$  predecir acciones sobre  $X$  en lotes de 4096 filas
25: if  $|y_{pred}| \neq |D|$  then
26:   abortar  $\triangleright$  las predicciones deben alinearse fila a fila con  $D$ 
27: end if
28: Escribir predictions.csv con  $y_{pred}$ ; si -include-sensitive-metadata, escribir también predictions_sensitive_local.csv con metadatos
29: Calcular estadísticas de predicción:  $n_{flows}$ , tasa de bloqueo, tasa de permiso y diagnósticos de  $z$ -score
30: if  $D$  contiene truth_y o truth_label con valores válidos then
31:   Añadir métricas binarias (matriz de confusión y métricas derivadas) comparando  $y_{pred}$  con la verdad
32: end if
33: Escribir config.json y metrics.json en runs/phase2/RUN_ID return Directorio de ejecución runs/phase2/RUN_ID

```

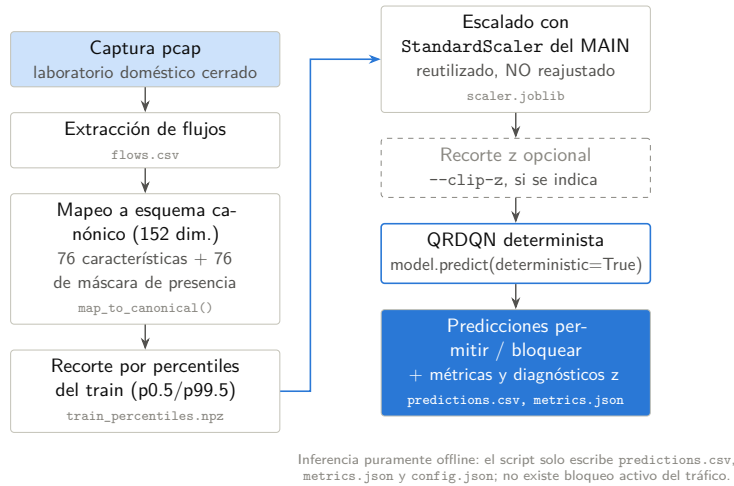


Figura 5.5: Pipeline de inferencia offline de la Fase 2. Fuente: elaboración propia a partir de `scripts/predict_real_traffic_v2.py`.

protocolo y la marca temporal pueden preservarse en los ficheros de salida para su inspección, pero no forman parte del vector de características del modelo. Esto mantiene separadas la interpretabilidad y la entrada al modelo.

5.6.2 Diagnósticos de cambio de distribución

La inferencia de la Fase 2 calcula diagnósticos sobre las características canónicas escaladas. Los z-scores absolutos elevados indican que los valores de flujo de laboratorio están lejos de la distribución de entrenamiento de CICIDS2017. El script también puede exportar las características con las mayores desviaciones. Estos diagnósticos no son etiquetas y no prueban la corrección, pero ayudan a explicar los cambios en la tasa de bloqueo e identificar discrepancias entre el tráfico del benchmark y el del laboratorio.

Más allá de los z-scores por característica, la salida de diagnóstico registra la proporción de características canónicas que fueron imputadas a partir de la máscara de presencia/ausencia, la media y la varianza de los valores de características escaladas relativas a las estadísticas de entrenamiento, y cualquier característica que caiga consistentemente fuera de los límites de percentiles observados durante el entrenamiento con CICIDS2017. En conjunto, estos

indicadores proporcionan un resumen estructurado de la distancia distribucional entre el tráfico de laboratorio y el benchmark, permitiendo una investigación dirigida en lugar de requerir la inspección manual de los registros brutos de predicción.

5.6.3 Métricas de verdad cuando están disponibles

Si un CSV de flujos de la Fase 2 incluye columnas de verdad como `truth_y` o `truth_label`, el script puede calcular las mismas métricas binarias utilizadas en la evaluación interna. Si no se dispone de la verdad fundamental, el script reporta estadísticas de predicción como la tasa de bloqueo, la tasa de permiso y los diagnósticos de z-score. Esta distinción es importante: una ejecución de laboratorio no etiquetada, solo benigna o mixta, no puede respaldar las mismas afirmaciones que un conjunto de validación externo etiquetado.

5.7 Limitaciones metodológicas

La metodología realiza compromisos deliberados que definen la interpretación correcta de los resultados de la tesis. Esta sección organiza esos compromisos en cuatro categorías.

5.7.1 Entorno estático y toma de decisiones secuencial

La limitación más fundamental es que el entorno de RL es un conjunto de datos estático. Las acciones del agente no alteran las observaciones de flujo futuras, la topología de la red, el comportamiento del atacante ni el estado del sistema. La formulación se asemeja por tanto más a un problema de decisión contextual sensible al coste que a un proceso de decisión de Markov completo (Sutton et al. [2018](#); Levine et al. [2020](#)). Las dependencias secuenciales,

las cadenas de ataque de múltiples etapas, la adaptación del atacante y las contramedidas activas del defensor quedan todas fuera del alcance implementado. Las afirmaciones sobre autonomía secuencial, defensa cibernética adaptativa o aprendizaje por refuerzo en línea no están respaldadas por el pipeline actual.

5.7.2 Mapeo de etiquetas binarias y detalle por familia de ataque

El mapeo de etiquetas binarias colapsa todos los tipos de ataque en una única clase BLOCK. La información sobre la familia de ataque se descarta en la ruta de aprendizaje principal de RL. Este diseño está justificado por el modelo de decisión binario estudiado aquí, pero significa que el agente no puede especializar su calidad de detección para familias de ataque individuales. El análisis de errores por familia requiere etiquetas de ataque preservadas y código de evaluación independiente, y no puede inferirse a partir de las métricas de exactitud binaria reportadas por el pipeline principal.

5.7.3 Fragilidad del benchmark y generalización

CICIDS2017 sigue siendo un benchmark público controlado con problemas documentados: artefactos de extracción, inconsistencias en las etiquetas, flujos duplicados y sensibilidad a la partición (Engelen et al. 2021; Lanvin et al. 2023; L. Liu et al. 2022). Incluso con preprocesamiento anti-fuga y protocolos de partición más estrictos, el rendimiento sobre CICIDS2017 no equivale al rendimiento sobre una red de producción. La inferencia de laboratorio en la Fase 2 comprueba el cambio de dominio, pero es inferencia offline sobre una captura privada y no una validación sobre un flujo de producción etiquetado de forma independiente. Una evidencia de generalización más sólida requeriría conjuntos de datos etiquetados externos, múltiples entornos de captura y, en última instancia, un análisis de seguridad para cualquier despliegue activo.

5.7.4 Algoritmo único y alcance de la reproducibilidad

El algoritmo de RL principal es una única configuración de QRDQN. Los resultados de una sola ejecución no pueden cuantificar la varianza entre ejecuciones a menos que se evalúen múltiples semillas y sus resultados se agreguen (Sutton et al. 2018). La línea base supervisada se limita a Random Forest en la comparación principal; otros modelos tabulares se discuten en la revisión de la literatura pero no forman parte del protocolo evaluado. Estas restricciones de alcance son prácticas: completar el pipeline central y la escalera de validación es metodológicamente más informativo que implementar parcialmente muchas comparaciones. Afirmaciones más sólidas en el futuro requerirían artefactos completos de leave-one-CSV-out, métricas de línea base supervisada con el mismo protocolo, múltiples semillas, tráfico de laboratorio etiquetado más rico y, en última instancia, un análisis de seguridad independiente para cualquier despliegue activo.

Resultados

Este capítulo reporta únicamente cifras respaldadas por artefactos de ejecución comprometidos en el repositorio. Cada resultado se identifica mediante su `RUN_ID` o mediante el fichero de artefacto que lo contiene, de modo que puede recuperarse e inspeccionarse de forma independiente. Siguiendo la escalera de validación descrita en el capítulo del protocolo experimental, los resultados se presentan desde el peldaño más permisivo (partición aleatoria) hasta los más exigentes (partición por día y comparación *leave-one-CSV-out* cuando existe artefacto), seguidos de la línea base supervisada y de la inferencia de la Fase 2.

6.1 Condiciones generales y trazabilidad

La Tabla 5.2 actúa como tabla T-G de esta memoria: enumera las ejecuciones oficiales, su propósito y los artefactos clave que respaldan las cifras citadas. Por tanto, este capítulo no mezcla ejecuciones exploratorias con resultados finales. La ejecución principal es `MAIN_qrdqn_cicids2017_canonical_full_random_20260609_193655`; los controles A/B/C proceden de `VAL_checks_A_20260212_235443`, `VAL_checks_B_20260212_235736` y `VAL_checks_C_20260213_004847`; los análisis estadísticos proceden de `bootstrap_`

`ci_seed42.json` y `duplicate_analysis_seed42.json`; la línea base supervisada procede del barrido `rf_cicids2017_canonical_20260628_024735`; y la transferencia a laboratorio procede de `P2v2_pred_20260610_161231_MAIN`.

La regla de trazabilidad seguida en este capítulo es estricta: una cifra solo aparece si existe en un artefacto bajo `runs/`, en `models/` o en el manifiesto de figuras `memoria/figuras/figures_manifest.json`. Las figuras F8–F15 ya fueron generadas desde esos artefactos y el manifiesto conserva, para cada una, la ruta de origen y los valores representados. Esta disciplina evita dos errores: convertir resultados históricos en conclusiones oficiales y completar huecos experimentales con valores no ejecutados. En particular, el *leave-one-CSV-out* de QRDQN y la curva de escala de entrenamiento siguen diseñados e implementados, pero pendientes de GPU; por tanto, este capítulo los marca como pendientes y no les asigna métricas.

6.2 Experimento 1: ejecución MAIN sobre la partición aleatoria fija

6.2.1 Condiciones de ejecución

El experimento principal y oficial de la tesis es la ejecución `MAIN_qrdqn_cicids2017_canonical_full_random_20260609_193655`. Se entrenó un agente QRDQN sobre el conjunto de datos completo de CICIDS2017 con la representación canónica de 152 dimensiones, una partición aleatoria estratificada con semilla 42 (entrenamiento de 2 264 594 filas y prueba de 566 149 filas) y la configuración de recompensa `tp=1.5`, `fp=-2.0`, `fn=-5.0`, `omission=0.0`. Los hiperparámetros (factor de descuento $\gamma = 0$, arquitectura [1024, 1024, 512], 200 cuantiles, tasa de aprendizaje $5e-5$, 20 pasos de gradiente, 3 000 000 pasos de entrenamiento) constituyen un perfil fijado a mano, no el resultado de una

búsqueda de Optuna.

6.2.2 Dinámica de entrenamiento

La Figura 6.1 muestra las curvas registradas durante la ejecución MAIN. La recompensa media de episodio pasa de $-11\,325.875$ al inicio del registro a $2\,451.735$ al final, y la pérdida de entrenamiento baja de 8.0318 a 1.6016 en el último paso registrado. Estas curvas sirven como evidencia descriptiva del proceso de entrenamiento, no como sustituto de la evaluación final sobre la partición de prueba fija.

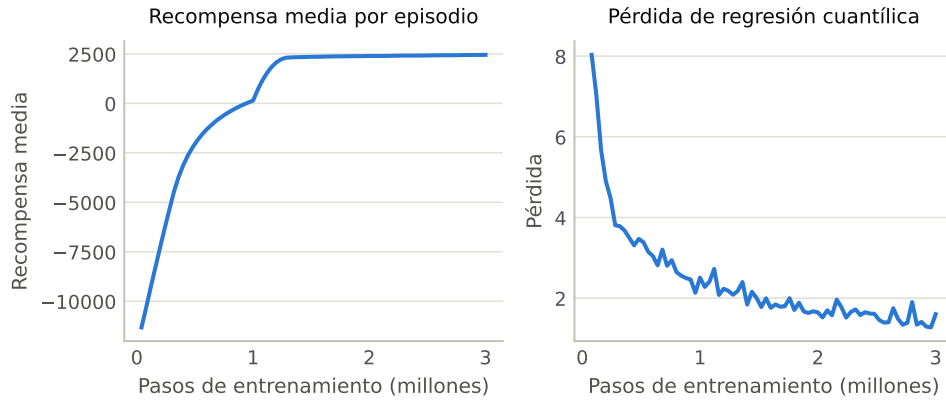


Figura 6.1: Curvas de entrenamiento de la ejecución MAIN. Fuente: elaboración propia a partir de los escalares TensorBoard exportados de MAIN.

6.2.3 Resultados en prueba

Sobre la partición aleatoria fija, el modelo MAIN alcanza una exactitud de 0.99381 , con un recall de ataque de 0.99536 y una puntuación F1 de ataque de 0.98445 (Tabla 6.1). La matriz de confusión sobre las $566\,149$ filas de prueba es $TN = 451\,631$, $FP = 2\,989$, $FN = 518$, $TP = 111\,011$. La Figura 6.2 desagrega el resultado completo.

Métrica	Valor
Exactitud	0.99381
Precisión (ataque)	0.97378
Recall (ataque)	0.99536
F1 (ataque)	0.98445
Precisión (benigno)	0.99885
Recall (benigno)	0.99343
F1 (benigno)	0.99613

Tabla 6.1: Métricas del modelo MAIN sobre la partición aleatoria fija (566 149 filas de prueba). Fuente: `metrics.json` de la ejecución MAIN.

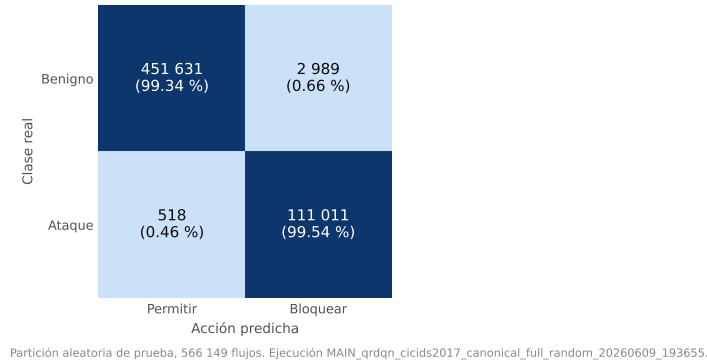


Figura 6.2: Matriz de confusión del modelo MAIN sobre la partición aleatoria fija. Fuente: elaboración propia a partir de `runs/validation/bootstrap_ci_seed42.json`.

6.2.4 Discusión breve

El resultado confirma que la arquitectura y el contrato de datos permiten aprender la tarea del benchmark bajo el peldaño más favorable de la escalera de validación. La cifra crítica para la interpretación de seguridad es el bajo número de falsos negativos en esta partición aleatoria fija. Sin embargo, esta partición no se interpreta como evidencia fuerte de generalización: CICIDS2017 contiene duplicados y patrones repetidos entre particiones, y esa fragilidad se cuantifica explícitamente en la Sección 6.5.

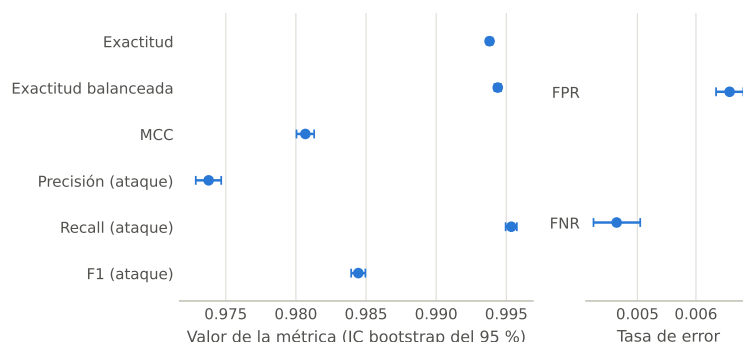
6.3 Intervalos de confianza bootstrap

Para cuantificar la precisión de muestreo de estas métricas sobre el conjunto de prueba fijo, se aplicó un bootstrap estratificado de 10 000 remuestreos sobre las 566 149 filas de prueba, sin reentrenar el modelo (artefacto `bootstrap_ci_seed42.json`). Los intervalos de confianza al 95 % son estrechos (Tabla 6.2). Es importante delimitar su alcance: estos intervalos describen la precisión de muestreo del conjunto de prueba para este *único* modelo entrenado; no cuantifican la varianza asociada a la semilla de entrenamiento, que requeriría múltiples ejecuciones y queda fuera del alcance de esta tesis.

Métrica	Valor	IC 95 %
Exactitud	0.99381	[0.99360, 0.99401]
Exactitud balanceada	0.99439	[0.99416, 0.99462]
MCC	0.98068	[0.98004, 0.98130]
Precisión (ataque)	0.97378	[0.97286, 0.97468]
Recall (ataque)	0.99536	[0.99495, 0.99575]
F1 (ataque)	0.98445	[0.98394, 0.98496]
FPR	0.00658	[0.00634, 0.00681]
FNR	0.00465	[0.00425, 0.00505]

Tabla 6.2: Intervalos de confianza bootstrap al 95 % (10 000 remuestreos estratificados) de las métricas del modelo MAIN. Fuente: `bootstrap_ci_seed42.json`.

La Figura 6.3 representa los mismos intervalos como barras de error. La lectura debe mantenerse acotada: son intervalos de precisión de muestreo sobre un único modelo entrenado, no intervalos de variabilidad entre semillas.



Bootstrap estratificado, 10 000 remuestreos del conjunto de prueba fijo (566 149 flujos): precisión de muestreo, no varianza entre semillas.

Figura 6.3: Intervalos bootstrap al 95 % de las métricas de MAIN. Fuente: elaboración propia a partir de `runs/validation/bootstrap_ci_seed42.json`.

6.4 Escalera de validación: comprobaciones A, B y C

Las tres comprobaciones de `validate_checks.py` cumplen propósitos distintos. La **Comprobación A** (`VAL_checks_A_20260212_235443`) evalúa el modelo mediante predicción directa sobre `X_test`, sin depender de la etiqueta que informa el entorno, y obtiene una exactitud de 0.9939 ($TP = 4772$, $FP = 60$, $FN = 1$, $TN = 5167$ sobre 10 000 muestras). Esta comprobación confirma que las buenas métricas no dependen de un artefacto del bucle del entorno. La **Comprobación B** (`VAL_checks_B_20260212_235736`) reentrena con etiquetas barajadas y obtiene una exactitud de 0.4773, por debajo de la línea base de clase mayoritaria (0.5227), con `leakage_detected = false`: cuando se destruye la relación entre características y etiquetas, el rendimiento no se mantiene artificialmente alto, como es deseable en ausencia de fuga.

La **Comprobación C** (`VAL_checks_C_20260213_004847`) es la evidencia de generalización dura del proyecto. Entrenando en lunes/martes/miércoles y evaluando en jueves/viernes (entrenamiento de 1 668 530 filas, prueba de 1 162 213 filas), la exactitud desciende a 0.84135 y el recall de ataque a 0.52954

(precisión de ataque 0.76479, F1 de ataque 0.62578). La caída frente al rendimiento in-distribution es exactamente el tipo de evidencia que distingue un benchmark cómodo de una evaluación honesta: el pipeline aprende muy bien dentro de la distribución, pero generalizar a días y patrones de ataque distintos es sustancialmente más difícil.

La matriz de confusión de la Comprobación C se muestra en la Figura 6.4. Esta comprobación usó una red proxy [512,256] entrenada durante 30 000 pasos, no los pesos del modelo MAIN; por tanto, se interpreta como peldaño de validación por día y no como evaluación directa del modelo oficial sobre la partición MAIN.

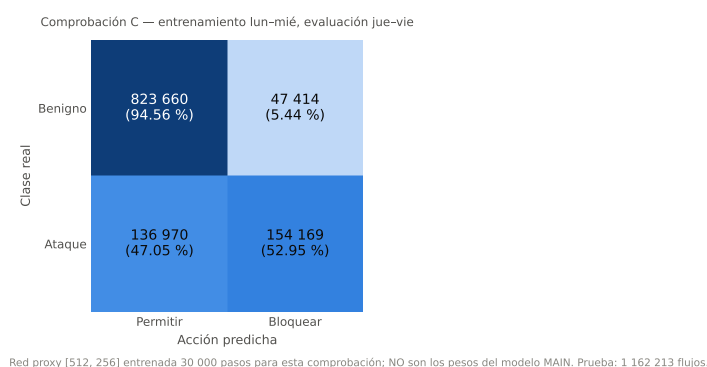


Figura 6.4: Matriz de confusión de la Comprobación C sobre la partición por día. Fuente: elaboración propia a partir de VAL_checks_C_20260213_004847/validation_results.json.

6.5 Análisis de duplicados y fuga en la partición aleatoria

Para explicar por qué la partición aleatoria es el peldaño más débil, se cuantificó la duplicación de filas y la fuga entre particiones sobre la misma partición con semilla 42, sin entrenar ningún modelo (artefacto `duplicate_analysis_seed42.json`). El 22.30 % de las filas del conjunto completo son duplicados exactos a nivel de características. Más relevante

para la evaluación: el 24.86 % de las filas de prueba son duplicados exactos de una fila de entrenamiento, proporción que asciende al 40.12 % entre las filas de ataque de prueba (frente al 21.12 % entre las benignas). Esto significa que una fracción considerable del conjunto de prueba aleatorio es, en efecto, memorizable a partir del entrenamiento, lo que infla las métricas in-distribution. Es la razón metodológica por la que la Comprobación C —y no la partición aleatoria— se interpreta como la señal creíble de generalización. Estas cifras se reportan como contexto honesto del peldaño aleatorio, no como una objeción al modelo MAIN, cuyo rendimiento sobre tráfico de laboratorio se examina en la Sección 6.7.

La Figura 6.5 resume visualmente estas cuatro tasas de duplicación y fuga. La barra de ataques de prueba ya presentes en entrenamiento es la más relevante para la interpretación de recall de ataque en la partición aleatoria.

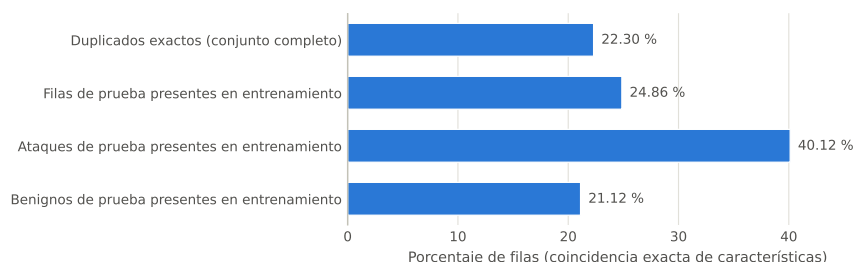


Figura 6.5: Duplicados exactos y fuga entre entrenamiento y prueba en la partición aleatoria fija. Fuente: elaboración propia a partir de `runs/validation/duplicate_analysis_seed42.json`.

6.6 Línea base de Random Forest bajo el mismo protocolo

La línea base de Random Forest se ejecutó bajo el mismo protocolo que QRDQN (representación canónica, escalado, particiones equivalentes y ponderación de clases balanceada), persistiendo `config.json` y `metrics.json`

por barrido (RUN_ID `rf_cicids2017_canonical_20260628_024735`). Sobre la partición aleatoria fija —el mismo conjunto de prueba de 566 149 filas que MAIN— el Random Forest alcanza una exactitud de 0.99872 y una F1 de ataque de 0.99676, por encima de QRDQN. Bajo la partición por día equivalente a la Comprobación C (entrenamiento lunes/martes/miércoles, prueba jueves/viernes), su recall de ataque colapsa a 0.08135 (F1 de ataque 0.15005, exactitud 0.76913), y bajo el leave-one-CSV-out con miércoles reservado el recall de ataque cae a 0.00719. La Tabla 6.3 resume la comparación con el mismo protocolo.

Modelo	Partición	Exactitud	Prec. ataque	Recall ataque	F1 ataque
QRDQN (MAIN)	Aleatoria fija	0.99381	0.97378	0.99536	0.98445
Random rest	Fo- Aleatoria fija	0.99872	0.99501	0.99853	0.99676
QRDQN (Check C)	Día (Lun–Mié → Jue–Vie)	0.84135	0.76479	0.52954	0.62578
Random rest	Fo- Día (Lun–Mié → Jue–Vie)	0.76913	0.96473	0.08135	0.15005
QRDQN	Leave-one-CSV-out (miércoles)	Pendiente de GPU, sin artefacto comprometido			
Random rest	Fo- Leave-one-CSV-out (miércoles)	0.63782	0.98482	0.00719	0.01427

Tabla 6.3: Comparación con el mismo protocolo entre QRDQN y la línea base de Random Forest balanceada. Las filas aleatorias comparten el conjunto de prueba de 566 149 filas con semilla 42; las filas por día comparten la partición lunes/martes/miércoles → jueves/viernes (1 162 213 filas de prueba); la fila leave-one-CSV-out de QRDQN se mantiene pendiente porque no existe artefacto comprometido. Fuentes: ejecuciones MAIN, VAL_checks_C_20260213_004847 y rf_cicids2017_canonical_20260628_024735.

La Figura 6.6 hace visible la diferencia principal de esta comparación: Random Forest supera a QRDQN en la partición aleatoria, pero su recall de ataque se desploma en la partición por día. El hueco leave-one-CSV-out de QRDQN se mantiene como pendiente de GPU, sin barra fabricada.

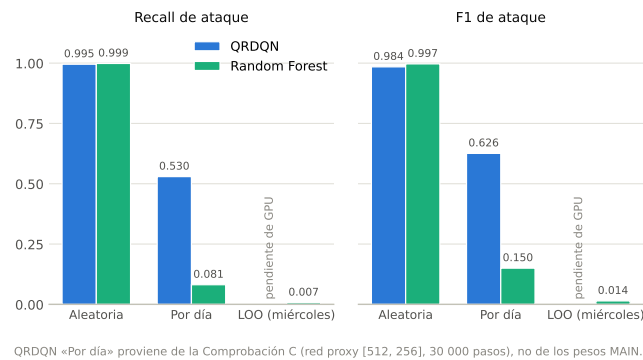


Figura 6.6: Comparación visual de QRDQN y Random Forest en recall y F1 de ataque bajo particiones equivalentes. Fuente: elaboración propia a partir de MAIN, VAL_checks_C_20260213_004847 y rf_cicids2017_canonical_20260628_024735.

Tabla pendiente Comparación leave-one-CSV-out entre QRDQN y Random Forest por escenario retenido
--

Tabla 6.4: Pendiente: Comparación leave-one-CSV-out entre QRDQN y Random Forest por escenario retenido.

La matriz de confusión de Random Forest en la partición por día (Figura 6.7) explica el bajo recall de ataque: el modelo conserva muy pocos falsos positivos, pero omite una gran mayoría de los ataques en jueves/viernes.

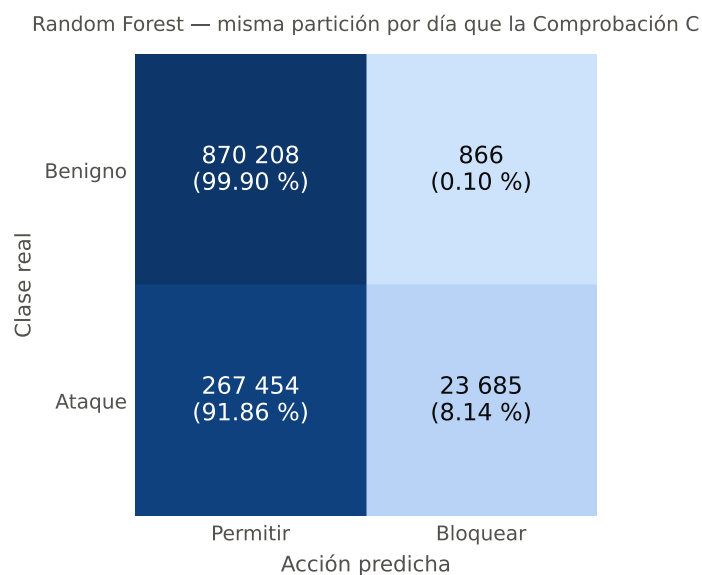


Figura 6.7: Matriz de confusión de Random Forest en la partición por día. Fuente: elaboración propia a partir de `rf_cicids2017_canonical_20260628_024735__day_split/metrics.json`.

6.7 Inferencia de la Fase 2 sobre tráfico de laboratorio

La inferencia offline de la Fase 2 aplicó el modelo MAIN, junto con su escalador y percentiles persistidos, sobre tráfico capturado por el operador en un laboratorio doméstico cerrado y no adversarial (archivo `lab_capture_traffic.csv`, 2 000 000 de flujos etiquetados; artefacto `P2v2_pred_20260610_161231_MAIN`). El modelo alcanza una exactitud de 0.991862, con una precisión de ataque de 0.97919, un recall de ataque de 0.988452 y una F1 de ataque de 0.98380 ($TP = 494\,226$, $TN = 1\,489\,498$, $FP = 10\,502$, $FN = 5\,774$), con una tasa de bloqueo de 0.252364. Los diagnósticos de distribución son benignos: el valor absoluto máximo de z observado es 9.547 y el medio 0.3417, sin ninguna característica que supere el recorte en $|z| = 10$. El modelo se comporta, por tanto, de forma razonable sobre esta captura de laboratorio concreta. No obstante,

dado que se trata de tráfico generado por el operador en un único entorno cerrado y no adversarial, este resultado es una comprobación controlada de validez externa limitada y no constituye evidencia de rendimiento de detección en el mundo real.

La Figura 6.8 resume la matriz de confusión y la tasa de bloqueo de esta ejecución. Su función es documentar el comportamiento del pipeline sobre esta captura concreta, manteniendo explícita su validez externa limitada.

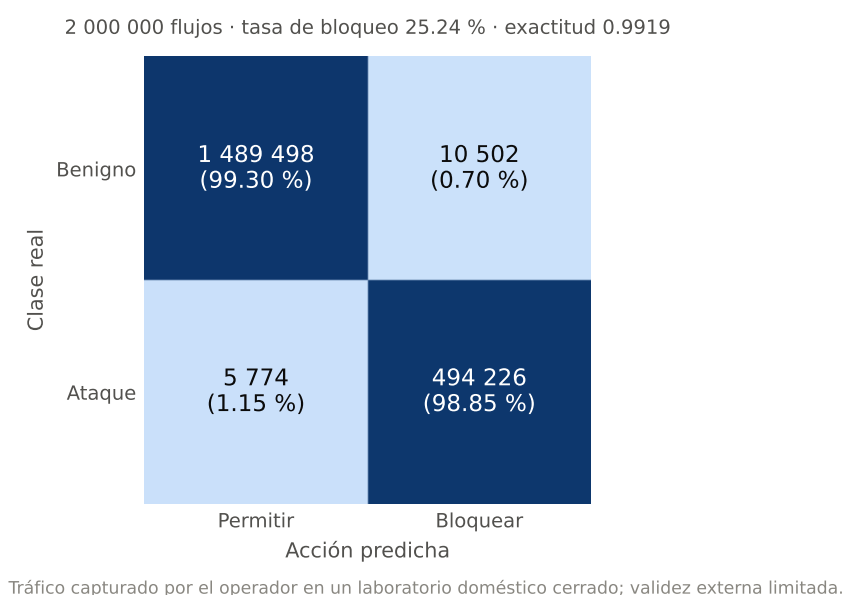


Figura 6.8: Resultados de la inferencia de la Fase 2 sobre tráfico de laboratorio etiquetado. Fuente: elaboración propia a partir de `runs/phase2/P2v2_pred_20260610_161231_MAIN/metrics.json`.

6.8 Síntesis y cobertura de objetivos

La lectura conjunta de los resultados es deliberadamente mixta. QRDQN aprende con solidez la tarea en distribución y mantiene pocos falsos negativos en la partición aleatoria fija, pero esa evidencia está afectada por duplicados y fuga entre entrenamiento y prueba. La Comprobación C muestra una degradación fuerte al cambiar de días, aunque conserva mucho más recall de ataque que la

línea base Random Forest en la misma partición. La línea base supervisada, a su vez, gana claramente en la partición aleatoria, lo que impide presentar QRDQN como superior de forma global. La Fase 2 ofrece una comprobación positiva del pipeline sobre una captura concreta de laboratorio, siempre dentro del marco de laboratorio doméstico cerrado, generado por el operador y con validez externa limitada.

La Tabla 6.5 resume cómo las cifras de este capítulo cubren los objetivos específicos definidos en el Capítulo 2. La tabla no sustituye a la matriz de trazabilidad general (Tabla 2.5); la complementa desde el punto de vista empírico, separando los objetivos ya respaldados por resultados de los que siguen pendientes por falta de artefactos de ejecución.

Objetivo	Evidencia en este capítulo	Lectura de cobertura
OBJ-01	Todas las ejecuciones oficiales operan sobre flujos mapeados al esquema canónico; MAIN usa 152 dimensiones.	Cubierto como condición común de los experimentos.
OBJ-02	MAIN, RF y Fase 2 reutilizan la representación canónica y los artefactos persistidos.	Cubierto; las cifras son comparables porque comparten contrato de observación.
OBJ-03	Resultados MAIN con recompensa $tp=1.5$, $fp=-2.0$, $fn=-5.0$, 0 y lectura de bandido contextual. $omission=0.0$.	Cubierto en ejecución oficial, con $\gamma = 0$ y lectura de bandido contextual.
OBJ-04	MAIN completa 3 000 000 pasos y alcanza F1 de ataque 0.98445 en prueba aleatoria fija.	Cubierto para entrenamiento y evaluación in-distribution.
OBJ-05	RF supera a QRDQN en partición aleatoria (F1 ataque 0.99676 vs. 0.98445), pero cae en recall por día (0.08135).	Cubierto con comparación honesta bajo mismo protocolo.
OBJ-06	Checks A/B/C, bootstrap y duplicados cuantifican controles de fuga y robustez de la partición.	Cubierto parcialmente: A/B/C y duplicados ejecutados; QRDQN leave-one-CSV-out pendiente.
OBJ-07	No hay métricas de curva de aprendizaje ni presupuestos de entrenamiento comprometidos.	Pendiente de GPU; no se reportan cifras.
OBJ-08	Fase 2 etiquetada alcanza exactitud 0.991862, recall ataque 0.988452 y tasa de bloqueo 0.252364.	Cubierto como inferencia offline en laboratorio concreto, con validez externa limitada.

Tabla 6.5: Cobertura empírica de los objetivos específicos OBJ-01–OBJ-08 a partir de los resultados reportados en este capítulo. Fuente: elaboración propia a partir de las ejecuciones oficiales y de la matriz de trazabilidad de objetivos.

Discusión

Este capítulo interpreta los resultados del Capítulo 6 sin introducir una nueva capa experimental. La lectura es deliberadamente conservadora: se separa lo que el sistema demuestra dentro del benchmark, lo que ocurre al romper la comodidad de la partición aleatoria, y lo que puede afirmarse sobre la Fase 2 sin convertir una captura de laboratorio en una prueba de despliegue.

7.1 Lectura cuantitativa del rendimiento principal

La ejecución MAIN confirma que la arquitectura implementada aprende la tarea de decisión binaria sobre CICIDS2017 cuando entrenamiento y prueba proceden de la misma distribución de partición aleatoria. Sobre 566 149 flujos de prueba, el modelo alcanza una exactitud de 0.99381, un recall de ataque de 0.99536 y una F1 de ataque de 0.98445. La matriz de confusión asociada lo concreta mejor que la exactitud agregada: 111 011 ataques quedan correctamente bloqueados y 518 ataques quedan permitidos por error, frente a 451 631 benignos permitidos y 2 989 benignos bloqueados. En un problema defensivo,

esta descomposición es más informativa que el porcentaje global porque separa el error operativo caro, el falso negativo, del coste de impacto sobre tráfico legítimo.

Los intervalos bootstrap refuerzan una idea distinta: el resultado está bien medido sobre ese conjunto de prueba fijo. La F1 de ataque queda en $[0.98394, 0.98496]$, el recall de ataque en $[0.99495, 0.99575]$, la tasa de falsos positivos en $[0.00634, 0.00681]$ y la tasa de falsos negativos en $[0.00425, 0.00505]$. Estos intervalos no convierten el experimento en una media robusta entre entrenamientos; cuantifican precisión de muestreo para este modelo concreto. Por eso la tesis usa el bootstrap como evidencia de estabilidad del test, no como sustituto de una evaluación con varias semillas.

7.2 Partición aleatoria frente a generalización por día

La cifra más importante del trabajo no es el rendimiento alto de MAIN, sino su contraste con la partición por día. En la Comprobación C, al entrenar en lunes/martes/miércoles y probar en jueves/viernes, la exactitud baja a 0.84135 y el recall de ataque cae a 0.52954. Además, esa comprobación se hizo con una red proxy [512,256] entrenada durante 30 000 pasos, no con los pesos MAIN. La evidencia debe leerse, por tanto, como un peldaño de generalización dura y no como una segunda medida directa del modelo oficial.

La caída no invalida el modelo; delimita el significado real de la partición aleatoria. El análisis de duplicados muestra que el 24.86% de las filas de prueba de la partición aleatoria duplican filas de entrenamiento, y que la proporción sube al 40.12% entre las filas de ataque. Bajo esas condiciones, parte del rendimiento in-distribution puede provenir de patrones repetidos o muy cercanos a los ya vistos. La partición por día rompe esa continuidad y

obliga a evaluar escenarios de captura distintos. Por eso el resultado de 0.52954 de recall de ataque, aunque mucho peor que el aleatorio, es metodológicamente más valioso para discutir generalización.

7.3 QRDQN frente a Random Forest

La comparación con Random Forest impide una conclusión simplista. En la partición aleatoria fija, el clasificador supervisado balanceado supera a QRDQN: F1 de ataque 0.99676 frente a 0.98445, con exactitud 0.99872 frente a 0.99381. La tesis no sostiene que QRDQN sea superior en distribución; para datos tabulares de flujo y una partición cómoda, Random Forest es una línea base muy fuerte y debe tratarse como tal.

La lectura cambia bajo el desplazamiento temporal. En la misma partición por día, Random Forest mantiene una precisión de ataque alta (0.96473), pero su recall de ataque cae a 0.08135 y su F1 de ataque a 0.15005. En términos prácticos, el modelo supervisado bloquea muy pocos ataques cuando el escenario retenido cambia: su tasa de falsos negativos es 0.91865. QRDQN, aun en la Comprobación C con red proxy, conserva un recall de ataque de 0.52954 y una F1 de 0.62578. La afirmación defendible no es que «RL gane» de forma general, sino que la formulación sensible al coste conserva mejor la métrica crítica de seguridad bajo este cambio de dominio concreto.

La Fase 2 añade una señal complementaria. Sobre 2 000 000 de flujos de laboratorio etiquetados, generados por el operador en un entorno doméstico cerrado y no adversarial, MAIN alcanza exactitud 0.991862, recall de ataque 0.988452, F1 de ataque 0.98380 y tasa de bloqueo 0.252364. Los diagnósticos de distribución son compatibles con una transferencia controlada del pipeline: z absoluto máximo 9.547, z absoluto medio 0.3417 y ningún valor por encima del umbral de recorte $|z| = 10$. Aun así, esta evidencia tiene validez externa limitada: prueba el contrato de características y la inferencia offline sobre una

captura concreta, no la robustez frente a tráfico adversarial independiente.

7.4 Posicionamiento frente al estado del arte

El posicionamiento de esta tesis frente a la literatura se apoya menos en superar una cifra publicada y más en hacer explícitas las condiciones bajo las que esa cifra se obtiene. La literatura de NIDS basada en aprendizaje automático reporta con frecuencia métricas muy altas sobre CICIDS2017 y otros benchmarks, pero varios trabajos han mostrado que la generalización entre conjuntos, días, capturas o dominios suele degradarse cuando se abandona la evaluación intraconjunto (Layeghy et al. 2022; Cantone et al. 2024). Por esa razón, comparar solo la exactitud aleatoria de MAIN contra resultados de la literatura sería metodológicamente pobre: mezclaría protocolos, particiones, filtros de características y posibles fugas de información.

La contribución aquí es más acotada. CICIDS2017 sigue siendo un banco de pruebas útil y ampliamente utilizado, pero requiere validación cuidadosa de sus particiones y de sus artefactos (Boukhamla et al. 2021). Esta tesis conserva el resultado aleatorio porque demuestra capacidad de aprendizaje y permite comparar QRDQN con Random Forest bajo el mismo protocolo. Sin embargo, desplaza la interpretación fuerte hacia la escalera de validación: duplicados, Comprobación C, comparación supervisada equivalente y Fase 2 offline. Ese enfoque está alineado con la advertencia de la literatura sobre la brecha entre rendimiento intradominio y generalización real, y evita presentar una cifra cómoda como si fuese una garantía operativa.

7.5 Lectura multiobjetivo de seguridad e impacto

El objetivo contractual C3 se refleja en la forma de leer los errores, no solo en la tabla de recompensa. La configuración `tp=1.5`, `fp=-2.0`, `fn=-5.0` y `omission=0.0` da más peso al ataque permitido que al benigno bloqueado. Esta asimetría codifica una preferencia de seguridad: en caso de incertidumbre, el coste de dejar pasar un ataque se considera más grave que el coste de interrumpir un flujo legítimo.

Esa preferencia no elimina el impacto. MAIN bloquea 2 989 flujos benignos en la partición aleatoria y 10 502 flujos benignos en la captura etiquetada de Fase 2. En un sistema activo, esos falsos positivos podrían traducirse en interrupciones, tickets, degradación de servicio o pérdida de confianza en el detector. La tesis, sin embargo, se mantiene en inferencia offline: BLOCK es una etiqueta experimental, no una actuación sobre una red real. La lectura multiobjetivo consiste precisamente en reconocer ambos lados: la recompensa empuja hacia la reducción del falso negativo por motivos de seguridad, pero cualquier despliegue futuro tendría que controlar explícitamente el impacto de los falsos positivos.

7.6 Amenazas a la validez

La discusión anterior depende de varias condiciones que no deben diluirse. La ejecución principal usa una sola semilla; el bootstrap mide precisión de muestreo y no varianza entre entrenamientos; la partición aleatoria contiene duplicados entre entrenamiento y prueba; la Comprobación C no usa los pesos MAIN; y la Fase 2 procede de un único laboratorio doméstico cerrado, generado por el operador y con validez externa limitada. Estas amenazas no son notas

menores, sino parte del resultado: definen dónde termina la evidencia y dónde empieza el trabajo pendiente.

El Capítulo 8 desarrolla esas limitaciones de forma separada para no mezclar interpretación y alcance. Aquí basta con fijar la conclusión: el proyecto demuestra un pipeline reproducible, una política QRDQN sensible al coste y una comparación honesta contra Random Forest, pero no demuestra preparación para despliegue activo ni robustez general frente a tráfico independiente de producción.

Limitaciones

El capítulo del protocolo experimental ya expuso las limitaciones *conceptuales* del enfoque: el entorno estático, la formulación de bandido contextual sensible al coste, el mapeo de etiquetas binarias, la fragilidad del benchmark y el alcance de un único algoritmo. Este capítulo las retoma a la luz de los resultados medidos. La finalidad no es suavizar las conclusiones, sino dejar claro qué parte de la evidencia está respaldada por artefactos y qué parte queda todavía como trabajo pendiente.

8.1 Modelo y semilla únicos

Los resultados principales corresponden a un único modelo QRDQN entrenado con una sola semilla. La ejecución MAIN está bien documentada y sus artefactos permiten reproducir la lectura de métricas, pero no permite estimar la variabilidad propia del entrenamiento. En aprendizaje por refuerzo, incluso con la misma arquitectura y los mismos hiperparámetros, cambios de semilla pueden alterar la exploración, el orden efectivo de muestras, la inicialización de la red y la trayectoria de optimización. Esta tesis no mide esa variabilidad porque solo existe una ejecución oficial completa comprometida.

Los intervalos de confianza bootstrap reportados (± 0.0002 a ± 0.001 al 95 %) cuantifican la precisión de muestreo del conjunto de prueba fijo, no la varianza entre ejecuciones de entrenamiento. Caracterizan cuán estable es la métrica sobre remuestreos del mismo conjunto de prueba para *este* modelo, pero no cuánto variaría el rendimiento al reentrenar con semillas distintas. Las cifras deben leerse, por tanto, como el comportamiento de un punto de configuración bien caracterizado, no como una media sobre la aleatoriedad del entrenamiento.

Tabla pendiente

Varianza entre semillas de QRDQN: media, desviación e intervalo por métrica principal

Tabla 8.1: Pendiente: Varianza entre semillas de QRDQN: media, desviación e intervalo por métrica principal.

8.2 Fuga en la partición aleatoria

El análisis de duplicados muestra que el 22.30 % de las filas del conjunto completo son duplicados exactos y que el 24.86 % de las filas de prueba de la partición aleatoria duplican una fila de entrenamiento (40.12 % entre los ataques de prueba). Esto no significa que el modelo no aprenda nada: MAIN mantiene una matriz de confusión coherente y supera ampliamente una comprobación trivial. Sí significa, en cambio, que la partición aleatoria contiene información repetida que facilita la tarea y que sus métricas deben interpretarse como una cota optimista dentro del benchmark.

Esta limitación se mitiga metodológicamente reportando la Comprobación C, basada en partición por día, como señal de generalización más exigente. Aun así, no se elimina por completo. La Comprobación C no sustituye al experimento principal porque usa una red proxy [512, 256] entrenada durante 30 000 pasos, no los pesos MAIN. Por tanto, la tesis conserva ambos peldaños con significados

distintos: la partición aleatoria demuestra capacidad de aprendizaje bajo el protocolo principal; la partición por día muestra la fragilidad de la generalización cuando se rompe la continuidad del benchmark.

8.3 Comparación con la línea base

La comparación con Random Forest se ejecutó bajo el mismo protocolo: representación canónica, escalado, ponderación de clases balanceada y particiones equivalentes. Esto hace que la comparación sea mucho más fuerte que una comparación contra cifras externas de la literatura. En la partición aleatoria fija, Random Forest supera a QRDQN en F1 de ataque (0.99676 frente a 0.98445), y esta ventaja debe declararse sin matices artificiales. La contribución de QRDQN no es dominar el benchmark cómodo, sino ofrecer una lectura sensible al coste bajo el cambio de partición por día.

Persisten dos limitaciones. La primera es que la comparación abarca un único modelo supervisado; otros clasificadores tabulares competitivos, como los ensamblados con gradient boosting, se discuten en la revisión de la literatura pero no forman parte del protocolo ejecutado. La segunda es que el barrido leave-one-CSV-out de Random Forest (miércoles reservado, recall de ataque 0.00719) no tiene todavía un artefacto QRDQN comprometido equivalente. En ese peldaño, la comparación es solo de la línea base consigo misma y no entre modelos. La comparación entre QRDQN y Random Forest es directa en la partición aleatoria fija y en la partición por día, pero no en el leave-one-CSV-out.

Tabla pendiente

Comparación ampliada con modelos tabulares supervisados bajo el mismo protocolo

Tabla 8.2: Pendiente: Comparación ampliada con modelos tabulares supervisados bajo el mismo protocolo.

8.4 Validez externa de la Fase 2

El alto rendimiento de la Fase 2 (0.991862 de exactitud) se obtiene sobre tráfico generado por el operador en un laboratorio doméstico cerrado y no adversarial, de un único entorno y con etiquetas de intención del generador. No es tráfico de producción, no contiene un adversario real y no procede de una fuente independiente. Su validez externa es, por tanto, limitada: un buen resultado en la Fase 2 es una comprobación controlada de que el contrato de características y el preprocesamiento robusto transfieren a una captura cercana a la distribución, no una prueba de detección en el mundo real (Sommer et al. 2010; Layeghy et al. 2023).

La principal aportación de la Fase 2 es de ingeniería experimental: demuestra que el pipeline puede procesar una captura externa al CSV de entrenamiento, aplicar el escalador persistido, conservar el esquema canónico y producir diagnósticos reproducibles. Ese resultado es necesario para defender que el sistema no vive únicamente dentro del cargador de CICIDS2017. Pero no basta para afirmar robustez operativa. Para ello harían falta capturas etiquetadas de varios entornos, condiciones benignas más diversas, tráfico adversarial independiente y un análisis de deriva más amplio.

8.5 Experimentos planificados pero no ejecutados

Por honestidad metodológica, se enumeran explícitamente los análisis previstos en el diseño cuyos artefactos no se han comprometido. El protocolo de curva de aprendizaje (escala de entrenamiento a distintos presupuestos) está diseñado e implementado, pero pendiente de GPU: no existe todavía ningún artefacto de curva de aprendizaje con métricas oficiales. La validación leave-one-CSV-out

está implementada en el código para QRDQN, pero no se ha comprometido un artefacto agregado de sus métricas. La evaluación con múltiples semillas para estimar la varianza entre ejecuciones tampoco se ha llevado a cabo.

Ninguna de estas tres capacidades se reporta con cifras en esta tesis. Esta ausencia no es un detalle administrativo: afecta a la fuerza de las conclusiones. Sin curva de aprendizaje, no se puede cuantificar cómo cambia QRDQN al variar el presupuesto de entrenamiento; sin leave-one-CSV-out agregado, no se puede medir la estabilidad por escenario retenido; y sin múltiples semillas, no se puede separar el comportamiento del perfil MAIN de la varianza del entrenamiento. Declararlas como pendientes evita que el texto convierta un diseño experimental en evidencia ejecutada.

Figura pendiente

Escalera futura de validación externa: varios entornos etiquetados, tráfico independiente y análisis previo a despliegue activo

Figura 8.1: Pendiente: Escalera futura de validación externa: varios entornos etiquetados, tráfico independiente y análisis previo a despliegue activo.

Consideraciones éticas y legales

Las herramientas de detección de intrusiones operan sobre tráfico de red que puede contener información sensible, y un sistema que decide entre permitir y bloquear conlleva consecuencias operativas directas. Aunque esta tesis se limita a un prototipo experimental de inferencia offline, las decisiones de diseño que la hacen reproducible y segura tienen también una dimensión ética y legal que conviene hacer explícita.

9.1 Uso dual y alcance estrictamente defensivo

La investigación en ciberseguridad tiene una naturaleza intrínsecamente dual: las mismas técnicas que permiten detectar tráfico malicioso pueden, en principio, informar su evasión. El trabajo aquí descrito es defensivo por construcción y, además, deliberadamente inerte respecto a cualquier red real. Las salidas **PERMIT** y **BLOCK** son clasificaciones experimentales sobre registros de flujo estáticos: el sistema no realiza bloqueo en vivo, descarte de paquetes ni manipulación de **iptables**, y no se integra con cortafuegos, controladores de redes definidas por software (SDN) ni sistemas de prevención de intrusiones (IPS), tal como se delimita en los no objetivos del Capítulo 2. Esta ausencia de

actuación en línea elimina el riesgo de que el artefacto, por sí mismo, provoque un bloqueo indebido de tráfico legítimo en una red en producción. El sistema es un instrumento de medición experimental, no un mecanismo de defensa desplegable.

Esta decisión de alcance también limita el riesgo de uso dual práctico. El repositorio contiene código de entrenamiento, validación e inferencia offline, pero no instrucciones para operar el modelo como herramienta de interrupción activa de tráfico ni para integrarlo en una infraestructura ajena. Incluso en la Fase 2, el resultado es un fichero de predicciones y métricas, no una acción ejecutada sobre la red. La utilidad defensiva del trabajo reside en estudiar el comportamiento de un detector sensible al coste y en documentar sus límites, no en proporcionar una capacidad operativa inmediata.

9.2 Privacidad y tratamiento de los datos

El sistema opera exclusivamente sobre características estadísticas a nivel de flujo y no sobre el contenido (*payload*) de los paquetes, evitando la inspección profunda de paquetes y buena parte de sus implicaciones legales y de privacidad. El contrato canónico de características refuerza esta posición al excluir a nivel de esquema los identificadores potencialmente sensibles o reidentificadores —direcciones IP de origen y destino, marcas temporales absolutas, **Flow ID** y números de puerto—, que se descartan antes de cualquier entrenamiento. El benchmark CICIDS2017 es un conjunto de datos público publicado por el Canadian Institute for Cybersecurity (Sharafaldin et al. 2018; Canadian Institute for Cybersecurity 2017). El tráfico empleado en la Fase 2 fue generado por el propio operador en un laboratorio doméstico cerrado y no adversarial: se trata de tráfico propio, no contiene datos personales de terceros y no fue capturado de redes ajenas, por lo que su uso no plantea problemas de consentimiento ni de exposición de información personal.

La Fase 2 se diseñó además con separación entre resultados reproducibles y metadatos sensibles. Las salidas que se pueden conservar como artefacto de investigación contienen métricas, diagnósticos y predicciones agregadas o depuradas; cualquier exportación con información operacional más sensible queda tratada como local y no forma parte de la evidencia pública de la memoria. Esta separación es importante porque incluso los metadatos de flujo, sin carga útil, pueden revelar patrones de actividad, horarios o servicios. La tesis evita apoyarse en esos identificadores y centra la evaluación en la representación canónica no identificativa.

9.3 Riesgo operativo y asimetría de errores

Las consecuencias de los dos tipos de error son profundamente asimétricas. Un falso negativo —un ataque clasificado como benigno y permitido— puede habilitar exfiltración de datos o interrupción del servicio; un falso positivo —tráfico benigno bloqueado— interrumpe comunicaciones legítimas y consume tiempo de análisis. El propio diseño de la función de recompensa codifica esta asimetría penalizando los falsos negativos con mayor severidad. Esta tesis reporta estos riesgos de forma honesta en lugar de ocultarlos tras la exactitud agregada: sobre la partición aleatoria fija *in-distribution*, el modelo oficial MAIN presenta una tasa de falsos positivos de 0.00658 y una tasa de falsos negativos de 0.00465; sin embargo, bajo la partición por días, sustancialmente más exigente (Check C), el recall de ataque desciende hasta 0.52954. Esta caída del rendimiento ante un cambio de condiciones es precisamente la razón por la que el sistema se presenta como un prototipo de investigación y no como una herramienta lista para el despliegue: un uso operativo responsable exigiría validación sobre tráfico de producción etiquetado, múltiples entornos de captura y un análisis de seguridad independiente antes de delegar en el modelo cualquier decisión con efectos reales (Apuzzese et al. 2022; Sommer et al. 2010).

El hecho de que la recompensa penalice más el falso negativo no autoriza a ignorar el falso positivo. En una red real, bloquear tráfico legítimo puede cortar sesiones de trabajo, provocar pérdida de disponibilidad y desplazar carga a operadores humanos. Por eso el prototipo no se conecta a un mecanismo de bloqueo activo y por eso las conclusiones se formulan en términos de inferencia offline. Antes de convertir una etiqueta BLOCK en una actuación, harían falta umbrales de decisión revisables, registro de auditoría, posibilidad de intervención humana y evaluación específica del impacto operativo.

9.4 Reproducibilidad como práctica responsable

Por último, la disciplina de reproducibilidad descrita en la metodología es también una práctica ética: la investigación en seguridad con aprendizaje automático adolece con frecuencia de código ausente y evaluaciones irreproducibles, lo que dificulta verificar o refutar las afirmaciones publicadas (Olszewski et al. 2023; Arp et al. 2020). La persistencia de artefactos por ejecución, la partición de prueba fija con semilla 42 y su manifiesto de referencia permiten que cualquier tercero audite exactamente bajo qué condiciones se obtuvo cada cifra reportada en esta tesis. Hacer las limitaciones verificables, en lugar de presentarlas como una declaración de buenas intenciones, es la forma de honestidad metodológica que este trabajo persigue.

Esta práctica también protege contra una forma habitual de sobrepromesa: presentar como resultado lo que solo está diseñado. En esta memoria, los experimentos pendientes de GPU, la validación leave-one-CSV-out de QRDQN y la estimación multi-semilla se describen como pendientes cuando no existe artefacto comprometido. Esa frontera entre lo ejecutado y lo planificado es una obligación ética básica en un trabajo aplicado a seguridad, donde una

afirmación exagerada podría inducir a confiar en un sistema fuera de su alcance real.

Conclusiones

Este capítulo cierra la memoria a partir de los artefactos y resultados ya presentados. La síntesis no transforma un diseño pendiente en evidencia ejecutada ni extiende la validez de una captura de laboratorio más allá de las condiciones en las que fue obtenida.

10.1 Objetivos contractuales

Los objetivos contractuales C1–C4 permiten leer el resultado del trabajo como un conjunto de compromisos verificables, no como una única cifra de rendimiento. La Tabla 2.5 y la Tabla 6.5 concentran la evidencia por objetivo; esta sección resume su alcance.

C1 — Pipeline experimental reproducible con contrato de datos canónico. El pipeline offline queda materializado en el contrato de observación, los algoritmos documentados y los artefactos de ejecución que inventaría la Tabla 5.2. Por tanto, la contribución no se reduce a un modelo entrenado: incluye la trazabilidad de configuración, métricas y preprocesamiento necesaria para auditar las cifras reportadas.

C2 — Defensor RL evaluado mediante una escalera de valida-

ción con control de fugas. La ejecución MAIN sustenta la evaluación in-distribution; las comprobaciones A/B/C son controles con artefactos separados. En particular, la Comprobación B reentrena con etiquetas barajadas y la Comprobación C utiliza una red proxy, no los pesos MAIN. Junto con el análisis de duplicados, estos resultados muestran que la lectura de la partición aleatoria debe complementarse con controles más exigentes. El protocolo *leave-one-CSV-out* de QRDQN está implementado, pero no dispone todavía de artefactos de ejecución en GPU; permanece como una extensión pendiente y no sostiene ninguna conclusión numérica de esta memoria.

C3 — Decisión sensible al coste con lectura multiobjetivo. La función de recompensa hace explícita la prioridad de reducir ataques permitidos, sin ocultar el impacto de bloquear tráfico legítimo. La evidencia por clase y la discusión de falsos positivos y falsos negativos permiten interpretar el sistema en términos de seguridad e impacto operativo, en vez de presentar la exactitud agregada como criterio suficiente.

C4 — Comparación honesta con línea base y transferencia a tráfico externo. La comparación común con Random Forest se limita a las particiones aleatoria fija y por día con resultados para ambos modelos; para QRDQN, la segunda corresponde a la Comprobación C con una red proxy, no con los pesos MAIN. La comparación *leave-one-CSV-out* de QRDQN sigue pendiente de GPU. Esta evidencia evita atribuir una superioridad general a QRDQN: la línea base obtiene mejor F1 de ataque en la partición aleatoria, mientras que la evaluación por día y la Fase 2 muestran por qué el protocolo y el dominio importan. La inferencia sobre tráfico de laboratorio confirma que el pipeline puede ejecutarse sobre esa captura concreta, pero sigue siendo offline, generada por el operador y con validez externa limitada.

10.2 Síntesis de los objetivos específicos

OBJ-01. Se completó la representación basada en flujos y se hizo auditable mediante el contrato canónico y el algoritmo de mapeo. Esta condición común permite que los experimentos de CICIDS2017 y la inferencia de laboratorio utilicen la misma interfaz de observación.

OBJ-02. Se completó el preprocesamiento canónico: las 76 características y su máscara de presencia/ausencia producen una observación de 152 dimensiones cuya semántica queda documentada y persistida en la configuración de MAIN.

OBJ-03. Se completó el entorno de RL compatible con Gymnasium y la recompensa asimétrica. La formulación se interpreta con precisión como un bandido contextual sensible al coste, pues $\gamma = 0$; no se presenta como una defensa secuencial que modifica el estado de una red.

OBJ-04. Se completó el entrenamiento y la evaluación in-distribution del agente QRDQN mediante la ejecución MAIN, cuyos artefactos conservan el modelo, el escalador, la configuración y las métricas necesarias para reproducir la lectura de resultados.

OBJ-05. Se completó la comparación con Random Forest bajo la misma representación y métricas en las particiones aleatoria fija y por día con resultados para ambos modelos; la fila de QRDQN por día corresponde a la Comprobación C con una red proxy. La comparación *leave-one-CSV-out* de QRDQN sigue pendiente de GPU. El resultado impide una afirmación de superioridad global de QRDQN y convierte la línea base en una referencia necesaria para interpretar la propuesta.

OBJ-06. Se completaron los controles de fuga implementados en el esquema, el escalado ajustado solo con entrenamiento, las comprobaciones A/B/C y el análisis de duplicados. El *leave-one-CSV-out* de QRDQN sigue pendiente de GPU como complemento de validación, no como evidencia ya disponible.

OBJ-07. El objetivo de escala de entrenamiento mantiene el estado de

diseño completado, ejecución pendiente de GPU. Existen protocolo e instrumentación, pero no artefactos de curva de aprendizaje; por ello no se reportan cifras ni conclusiones sobre eficiencia de datos.

OBJ-08. Se completó la inferencia offline sobre la captura de laboratorio mediante la ejecución P2v2 con el modelo MAIN y su escalador persistido. Esta evidencia comprueba el funcionamiento del pipeline en ese contexto concreto, sin sustituir una validación externa independiente ni un despliegue activo.

10.3 Reflexión del autor

El aprendizaje principal de este trabajo no es que una métrica elevada baste para declarar resuelto un problema de ciberseguridad. El resultado de QRDQN en la partición aleatoria confirma que el pipeline aprende la tarea definida, pero los duplicados, la partición por día, la comparación con Random Forest y la captura de laboratorio obligan a interpretar esa señal con cautela. La contribución más sólida de la memoria es, por tanto, la disciplina con la que el sistema hace visibles sus supuestos: qué entra en la observación, cómo se evita la fuga, qué artefacto respalda cada cifra y qué pruebas continúan pendientes.

También se ha confirmado que la función de recompensa no elimina los compromisos del dominio, sino que obliga a hacerlos explícitos. Penalizar más el falso negativo prioriza la seguridad, pero no convierte en aceptable el coste operativo de los falsos positivos. Mantener la salida **BLOCK** dentro de un flujo offline ha permitido estudiar ese compromiso sin delegar decisiones reales en el modelo. Esta delimitación es una fortaleza metodológica y una condición para que futuras extensiones se evalúen de forma responsable.

10.4 Líneas futuras

10.4.1 Corto plazo: completar la evidencia diseñada

La prioridad inmediata es ejecutar y comprometer los experimentos ya diseñados: la escalera de tamaños de entrenamiento de OBJ-07, el *leave-one-CSV-out* de QRDQN y el estudio con múltiples semillas. Estas tareas convertirían protocolos existentes en evidencia sobre eficiencia de datos, sensibilidad por escenario y variabilidad del entrenamiento, sin modificar las conclusiones que actualmente dependen de MAIN. Completar esta evidencia responde a las buenas prácticas de evaluación que reclaman controles explícitos de reproducibilidad y generalización en DRL aplicado a IDS (Cevallos M. et al. [2023](#)).

10.4.2 Medio plazo: ampliar la comparación y la validación externa

El siguiente paso es ampliar la comparación a más familias supervisadas y de RL bajo el mismo contrato de características, junto con capturas etiquetadas e independientes de varios entornos. La meta no sería acumular benchmarks, sino comprobar qué conclusiones sobreviven al cambio de conjunto de datos, topología, extractor y escenario de tráfico. La evaluación cruzada de NIDS basados en ML muestra que estas variaciones son necesarias para distinguir rendimiento intradominio de capacidad de generalización (Apruzzese et al. [2022](#)).

10.4.3 Largo plazo: tratar incertidumbre y transición operacional

Una línea de investigación a más largo plazo consiste en incorporar mecanismos para reconocer tráfico desconocido, complementar la clasificación binaria con estimación de incertidumbre y evaluar el sistema en entornos controlados antes de considerar cualquier acción activa. Los enfoques de detección de intrusiones con reconocimiento de conjunto abierto ofrecen un antecedente para tratar tráfico que no encaja en las clases conocidas durante el entrenamiento (Zhang et al. 2025). Cualquier transición desde la inferencia offline hacia controles reales debería añadir revisión humana, registro auditable, análisis específico del impacto de los falsos positivos y una evaluación de seguridad independiente; no puede deducirse automáticamente de los resultados presentados aquí.

Bibliografía

- [1] Amrin Maria Khan Adawadkar y Nilima Kulkarni. «Cyber-Security and Reinforcement Learning: A Brief Survey». En: *Engineering Applications of Artificial Intelligence* 114 (2022), pág. 105116. DOI: [10.1016/j.engappai.2022.105116](https://doi.org/10.1016/j.engappai.2022.105116) (vid. pág. 49).
- [2] Zeeshan Ahmad, Adnan Shahid Khan, Cheah Wai Shiang, Johari Abdullah y Farhan Ahmad. «Network Intrusion Detection System: A Systematic Study of Machine Learning and Deep Learning Approaches». En: *Transactions on Emerging Telecommunications Technologies* 32.1 (2021), e4150. DOI: [10.1002/ett.4150](https://doi.org/10.1002/ett.4150) (vid. pág. 42).
- [3] Hooman Alavizadeh, Julian Jang-Jaccard y Hootan Alavizadeh. «Deep Q-Learning Based Reinforcement Learning Approach for Network Intrusion Detection». En: (2021). arXiv: [2111.13978 \[cs.CR\]](https://arxiv.org/abs/2111.13978) (vid. pág. 49).
- [4] Abdullah Alsaedi, Nour Moustafa, Zahir Tari, Abdun Mahmood y Adnan Anwar. «TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Intrusion Detection Systems». En: *IEEE Access* 8 (2020), págs. 165130-165150. DOI: [10.1109/ACCESS.2020.3022862](https://doi.org/10.1109/ACCESS.2020.3022862) (vid. pág. 39).
- [5] Giovanni Apruzzese, Luca Pajola y Mauro Conti. «The Cross-Evaluation of Machine Learning-Based Network Intrusion Detection Systems». En:

- IEEE Transactions on Network and Service Management* 19.4 (2022), págs. 5152-5169. DOI: [10.1109/TNSM.2022.3157344](https://doi.org/10.1109/TNSM.2022.3157344) (vid. págs. 4, 18, 22, 38, 43, 58, 91, 95, 137, 144).
- [6] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro y Konrad Rieck. «Dos and Don'ts of Machine Learning in Computer Security». En: (2020). arXiv: [2010.09470 \[cs.CR\]](https://arxiv.org/abs/2010.09470) (vid. págs. 4, 19, 37, 44, 54, 62, 67, 93, 102, 138).
- [7] Momand Asadullah, Jan Sana Ullah y Naeem Ramzan. «A Systematic and Comprehensive Survey of Recent Advances in Intrusion Detection Systems Using Machine Learning: Deep Learning, Datasets, and Attack Taxonomy». En: *Expert Systems* (2023). DOI: [10.1155/2023/6048087](https://doi.org/10.1155/2023/6048087) (vid. pág. 44).
- [8] Stefan Axelsson. «The Base-Rate Fallacy and the Difficulty of Intrusion Detection». En: *ACM Transactions on Information and System Security* 3.3 (2000), págs. 186-205. DOI: [10.1145/357802.357804](https://doi.org/10.1145/357802.357804) (vid. págs. 2, 55, 56).
- [9] Marc G. Bellemare, Will Dabney y Rémi Munos. «A Distributional Perspective on Reinforcement Learning». En: *Proceedings of the 34th International Conference on Machine Learning (ICML)*. Vol. 70. 2017, págs. 449-458. URL: <https://proceedings.mlr.press/v70/bellemare17a.html> (vid. págs. 48, 79, 81).
- [10] Akram Boukhamla y Javier Coronel Gavira. «CICIDS2017 Dataset: Performance Improvements and Validation as a Robust Intrusion Detection System Testbed». En: *International Journal of Information and Computer Security* 16.1/2 (2021), págs. 20-32. DOI: [10.1504/IJICS.2021.117392](https://doi.org/10.1504/IJICS.2021.117392). URL: <https://www.inderscience.com/info/inarticle.php?artid=117392> (vid. págs. 41, 127).

- [11] Guillermo Caminero, Manuel Lopez-Martin y Belen Carro. «Adversarial Environment Reinforcement Learning Algorithm for Intrusion Detection». En: *2019 International Joint Conference on Neural Networks (IJCNN)*. 2019, págs. 1-8. DOI: [10.1109/IJCNN.2019.8852380](https://doi.org/10.1109/IJCNN.2019.8852380) (vid. pág. 49).
- [12] Canadian Institute for Cybersecurity. *Intrusion Detection Evaluation Dataset (CIC-IDS2017)*. Dataset documentation page. 2017. URL: <https://www.unb.ca/cic/datasets/ids-2017.html> (vid. págs. 7, 36, 40, 89, 136).
- [13] Marco Cantone, Claudio Marrocco y Alessandro Bria. «Machine Learning in Network Intrusion Detection: A Cross-Dataset Generalization Study». En: *IEEE Access* 12 (2024), págs. 144489-144508. DOI: [10.1109/ACCESS.2024.3472907](https://doi.org/10.1109/ACCESS.2024.3472907). URL: <https://doi.org/10.1109/ACCESS.2024.3472907> (vid. págs. 54, 127).
- [14] Alvaro A. Cárdenas, John S. Baras y Karl Seamon. «A Framework for the Evaluation of Intrusion Detection Systems». En: *2006 IEEE Symposium on Security and Privacy (S&P)*. 2006, págs. 63-77. DOI: [10.1109/SP.2006.2](https://doi.org/10.1109/SP.2006.2) (vid. págs. 7, 17, 52, 57, 74).
- [15] Center for Security and Emerging Technology y The Alan Turing Institute. *Autonomous Cyber Defense*. <https://cset.georgetown.edu/publication/autonomous-cyber-defense/>. 2023 (vid. pág. 50).
- [16] Jesús F. Cevallos M., Alessandra Rizzardi, Sabrina Sicari y Alberto Coen Porisini. «Deep Reinforcement Learning for Intrusion Detection in Internet of Things: Best Practices, Lessons Learnt, and Open Challenges». En: *Computer Networks* 236 (2023), pág. 110016. DOI: [10.1016/j.comnet.2023.110016](https://doi.org/10.1016/j.comnet.2023.110016). URL: <https://doi.org/10.1016/j.comnet.2023.110016> (vid. págs. 51, 144).
- [17] Communications Security Establishment y Canadian Institute for Cybersecurity. *A Realistic Cyber Defense Dataset (CSE-CIC-IDS2018)*. 2018.

- URL: <https://registry.opendata.aws/cse-cic-ids2018/> (vid. pág. 39).
- [18] Will Dabney, Mark Rowland, Marc G. Bellemare y Rémi Munos. «Distributional Reinforcement Learning with Quantile Regression». En: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*. 2018, págs. 2892-2901. URL: <https://ojs.aaai.org/index.php/AAAI/article/view/11791> (vid. págs. 3, 7, 17, 48, 79, 80).
- [19] Davide Di Monda et al. «Few-Shot Class-Incremental Learning for Network Intrusion Detection Systems». En: (2024). Exact venue and DOI require verification; see arXiv preprint search for Di Monda 2024 (vid. pág. 58).
- [20] Rohit Dube. «Faulty Use of the CIC-IDS 2017 Dataset in Information Security Research». En: *Journal of Computer Virology and Hacking Techniques* 20.1 (2024), págs. 203-211. DOI: [10.1007/s11416-023-00509-7](https://doi.org/10.1007/s11416-023-00509-7) (vid. págs. 41, 54).
- [21] Gints Engelen, Vera Rimmer y Wouter Joosen. «Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study». En: *2021 IEEE Security and Privacy Workshops (SPW)*. 2021, págs. 7-12. DOI: [10.1109/SPW53761.2021.00009](https://doi.org/10.1109/SPW53761.2021.00009) (vid. págs. 3, 7, 39, 41, 54, 62, 90, 108).
- [22] Farama Foundation. *Gymnasium Documentation*. Software documentation; access date 2026. 2023. URL: <https://gymnasium.farama.org/> (vid. págs. 7, 16, 48, 71, 87).
- [23] Yasir Ali Farrukh, Irfan Khan, Syed Wali, David Bierbrauer, Nathaniel D. Bastian y John Pavlik. «Payload-Byte: A Tool for Extracting and Labeling Packet Capture Files of Modern Network Intrusion Detection Datasets». En: *TechRxiv* (sep. de 2022). Preprint. DOI: [10.36227/techrxiv.20714221.v1](https://doi.org/10.36227/techrxiv.20714221.v1). URL: <https://doi.org/10.36227/techrxiv.20714221.v1> (vid. pág. 36).

- [24] Tom Fawcett. «An Introduction to ROC Analysis». En: *Pattern Recognition Letters* 27.8 (2006), págs. 861-874. DOI: [10.1016/j.patrec.2005.10.010](https://doi.org/10.1016/j.patrec.2005.10.010) (vid. págs. 56, 99).
- [25] Scott Fujimoto, David Meger y Doina Precup. «Off-Policy Deep Reinforcement Learning without Exploration». En: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. Proceedings of Machine Learning Research. 2019, págs. 2052-2062. URL: <https://proceedings.mlr.press/v97/fujimoto19a.html> (vid. pág. 47).
- [26] Patrik Goldschmidt y Daniela Chudá. «Network Intrusion Datasets: A Survey, Limitations, and Recommendations». En: *Computers & Security* 156 (2025), pág. 104510. DOI: [10.1016/j.cose.2025.104510](https://doi.org/10.1016/j.cose.2025.104510). URL: <https://doi.org/10.1016/j.cose.2025.104510> (vid. pág. 38).
- [27] Afrah Gueriani, Hamza Kheddar y Ahmed Cherif Mazari. «Deep Reinforcement Learning for Intrusion Detection in IoT: A Survey». En: (2024). Preprint; published version in IC2EM proceedings 2023. arXiv: [2405.20038 \[cs.CR\]](https://arxiv.org/abs/2405.20038) (vid. págs. 32, 49).
- [28] Arash Habibi Lashkari, Gerard Draper Gil, Mohammad Saiful Islam Mamun y Ali A. Ghorbani. «Characterization of Tor Traffic Using Time Based Features». En: *Proceedings of the 3rd International Conference on Information Systems Security and Privacy (ICISSP)*. 2017, págs. 253-262. DOI: [10.5220/0006105602530262](https://doi.org/10.5220/0006105602530262) (vid. págs. 36, 40, 89).
- [29] Dongkun Han, Zhiping Wang, Yihua Zhong, Wenhan Chen, Jiaze Yang, Shanqing Lu, Xinyan Shi y Xia Yin. «Evaluating Network Intrusion Detection Systems for Different Vulnerability Conditions Using an Adversarial Environment DQN». En: *IEEE Access* 9 (2020). IEEE Xplore document ID 9289884; full author list requires verification, págs. 4345-4359. DOI: [10.1109/ACCESS.2020.3047430](https://doi.org/10.1109/ACCESS.2020.3047430) (vid. pág. 49).

- [30] Hado van Hasselt, Arthur Guez y David Silver. «Deep Reinforcement Learning with Double Q-Learning». En: *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*. 2016, págs. 2094-2100. arXiv: [1509.06461](https://arxiv.org/abs/1509.06461) (vid. pág. 47).
- [31] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Gheshlaghi Azar y David Silver. «Rainbow: Combining Improvements in Deep Reinforcement Learning». En: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*. 2018, págs. 3215-3222. URL: <https://ojs.aaai.org/index.php/AAAI/article/view/11796> (vid. págs. 47, 58).
- [32] Md. Alamgir Hossain et al. «Deep Q-Learning Intrusion Detection System (DQ-IDS): A Novel Reinforcement Learning Approach for Adaptive and Self-Learning Cybersecurity». En: *ICT Express* (2025). Full author list requires verification; accessed via ScienceDirect. DOI: [10.1016/j.icte.2025.02.006](https://doi.org/10.1016/j.icte.2025.02.006) (vid. pág. 49).
- [33] Chia-Wei Hsu, Chia-Hsuan Lin et al. «A Soft Actor-Critic Reinforcement Learning Algorithm for Network Intrusion Detection». En: *Computers & Security* 136 (2023). Full author list requires verification against published paper, pág. 103580. DOI: [10.1016/j.cose.2023.103580](https://doi.org/10.1016/j.cose.2023.103580) (vid. pág. 49).
- [34] Zhisheng Hu, Minghui Zhu y Peng Liu. «Adaptive Cyber Defense Against Multi-Stage Attacks Using Learning-Based POMDP». En: *ACM Transactions on Privacy and Security* 24.1 (2020), págs. 1-31. DOI: [10.1145/3418897](https://doi.org/10.1145/3418897) (vid. pág. 50).
- [35] Marcin Iwanowski, Dominik Olszewski, Waldemar Graniszewski, Jacek Krupski y Franciszek Pelc. «The Choice of Training Data and the Generalizability of Machine Learning Models for Network Intrusion Detection Systems». En: *Applied Sciences* 15.15 (2025), pág. 8466. DOI: [10.3390/](https://doi.org/10.3390/)

- app15158466. URL: <https://www.mdpi.com/2076-3417/15/15/8466> (vid. pág. 54).
- [36] R. Kanimozhi y P. S. Ramesh. «Deep Reinforcement Learning-Based Intrusion Detection Scheme for Software-Defined Networking». En: *Scientific Reports* 15.1 (2025), pág. 38827. DOI: [10.1038/s41598-025-24869-w](https://doi.org/10.1038/s41598-025-24869-w). URL: <https://www.nature.com/articles/s41598-025-24869-w> (vid. pág. 51).
- [37] Shachar Kaufman, Saharon Rosset y Claudia Perlich. «Leakage in Data Mining: Formulation, Detection, and Avoidance». En: *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2011, págs. 556-563. DOI: [10.1145/2020408.2020496](https://doi.org/10.1145/2020408.2020496) (vid. págs. 4, 19, 37, 54, 67).
- [38] KDD Cup. *KDD Cup 1999 Data*. Dataset page. 1999. URL: <http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html> (vid. págs. 38, 39).
- [39] Hamza Kheddar, Diana W. Dawoud, Ali Ismail Awad, Yassine Himeur y Muhammad Khurram Khan. «Reinforcement-Learning-Based Intrusion Detection in Communication Networks: A Review». En: *IEEE Communications Surveys & Tutorials* 27.4 (2025), págs. 2420-2469. DOI: [10.1109/COMST.2024.3484491](https://doi.org/10.1109/COMST.2024.3484491) (vid. págs. 32, 49, 58, 59).
- [40] Nickolaos Koroniotis, Nour Moustafa, Elena Sitnikova y Benjamin Turnbull. «Towards the Development of Realistic Botnet Dataset in the Internet of Things for Network Forensic Analytics: Bot-IoT Dataset». En: *Future Generation Computer Systems* 100 (2019), págs. 779-796. DOI: [10.1016/j.future.2019.05.041](https://doi.org/10.1016/j.future.2019.05.041) (vid. pág. 39).
- [41] Maxime Lanvin, Pierre-François Gimenez, Yufei Han, Frédéric Majorczyk, Ludovic Mé y Éric Totel. «Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes». En: *Risks and Security of Internet and Systems*. Vol. 13857. Lecture Notes in

- Computer Science. Springer, 2023, págs. 19-34. DOI: [10.1007/978-3-031-31108-6_2](https://doi.org/10.1007/978-3-031-31108-6_2) (vid. págs. 3, 39, 41, 54, 62, 90, 93, 108).
- [42] Maxime Lanvin, Pierre-François Gimenez, Yufei Han, Frédéric Majorczyk, Ludovic Mé y Éric Totel. «Faulty Use of the CIC-IDS 2017 Dataset in Information Security Research». En: *Risks and Security of Internet and Systems*. Vol. 13561. Lecture Notes in Computer Science. Earlier presentation of dataset-critique work; see also Lanvin2023Errors. Springer, 2022 (vid. pág. 41).
- [43] Siamak Layeghy y Marius Portmann. «Explainable Cross-Domain Evaluation of ML-Based Network Intrusion Detection Systems». En: *Computers and Electrical Engineering* 108 (2023), pág. 108692. DOI: [10.1016/j.compeleceng.2023.108692](https://doi.org/10.1016/j.compeleceng.2023.108692) (vid. págs. 22, 38, 54, 91, 133).
- [44] Siamak Layeghy y Marius Portmann. «On Generalisability of Machine Learning-Based Network Intrusion Detection Systems». En: (2022). DOI: [10.48550/arXiv.2205.04112](https://doi.org/10.48550/arXiv.2205.04112). arXiv: [2205.04112 \[cs.NI\]](https://arxiv.org/abs/2205.04112). URL: <https://arxiv.org/abs/2205.04112> (vid. págs. 54, 127).
- [45] Wenke Lee, Wei Fan, Matthew Miller, Salvatore J. Stolfo y Erez Zadok. «Toward Cost-Sensitive Modeling for Intrusion Detection and Response». En: *Journal of Computer Security* 10.1–2 (2002), págs. 5-22. DOI: [10.3233/JCS-2002-101-202](https://doi.org/10.3233/JCS-2002-101-202) (vid. págs. 3, 17, 52, 74).
- [46] Sergey Levine, Aviral Kumar, George Tucker y Justin Fu. «Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems». En: *arXiv preprint arXiv:2005.01643* (2020). Preprint. arXiv: [2005.01643 \[cs.LG\]](https://arxiv.org/abs/2005.01643) (vid. págs. 6, 47, 79, 107).
- [47] Yantao Li y Wenhui Ding. «Optimized Network Security Attack Detection Algorithm Based on Deep Deterministic Policy Gradient (DDPG)». En: *2025 International Conference on Artificial Intelligence and Digital Ethics (ICAIDE)*. 2025, págs. 214-217. DOI: [10.1109/ICAIDE65466](https://doi.org/10.1109/ICAIDE65466).

- 2025.11189334. URL: <https://doi.org/10.1109/ICAIDE65466.2025.11189334> (vid. pág. 51).
- [48] Long-Ji Lin. «Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching». En: *Machine Learning* 8.3–4 (1992), págs. 293-321. DOI: [10.1007/BF00992699](https://doi.org/10.1007/BF00992699) (vid. pág. 46).
- [49] Hongyu Liu y Bo Lang. «Machine Learning and Deep Learning Methods for Intrusion Detection Systems: A Survey». En: *Applied Sciences* 9.20 (2019), pág. 4396. DOI: [10.3390/app9204396](https://doi.org/10.3390/app9204396) (vid. págs. 4, 18, 42-44, 59, 95).
- [50] Lisa Liu, Gints Engelen, Timothy M. Lynar, Daryl Essam y Wouter Joosen. «Error Prevalence in NIDS Datasets: A Case Study on CIC-IDS-2017 and CSE-CIC-IDS-2018». En: *2022 IEEE Conference on Communications and Network Security (CNS)*. 2022, págs. 254-262. DOI: [10.1109/CNS56114.2022.9947235](https://doi.org/10.1109/CNS56114.2022.9947235) (vid. págs. 39, 41, 54, 90, 108).
- [51] Manuel Lopez-Martin, Belén Carro y Antonio Sanchez-Esguevillas. «Application of Deep Reinforcement Learning to Intrusion Detection for Supervised Problems». En: *Expert Systems with Applications* 141 (2020), pág. 112963. DOI: [10.1016/j.eswa.2019.112963](https://doi.org/10.1016/j.eswa.2019.112963) (vid. pág. 49).
- [52] Manuel Lopez-Martin, Antonio Sanchez-Esguevillas, Juan I. Arribas y Belén Carro. «Network Intrusion Detection Based on Extended RBF Neural Network With Offline Reinforcement Learning». En: *IEEE Access* 9 (2021), págs. 153153-153170. DOI: [10.1109/ACCESS.2021.3127689](https://doi.org/10.1109/ACCESS.2021.3127689) (vid. pág. 49).
- [53] Malika Malik y Kamaljit Singh Saini. «Network Intrusion Detection System Using Reinforcement Learning Techniques». En: *2023 International Conference on Circuit Power and Computing Technologies (ICCPCT)*. 2023, págs. 1642-1649. DOI: [10.1109/ICCPCT58313.2023.10245608](https://doi.org/10.1109/ICCPCT58313.2023.10245608). URL: <https://doi.org/10.1109/ICCPCT58313.2023.10245608> (vid. pág. 50).

- [54] Malika Malik y Kamaljit Singh Saini. «Network Intrusion Detection System using Reinforcement learning». En: *2023 4th International Conference for Emerging Technology (INCET)*. 2023, págs. 1-4. DOI: [10.1109/INCET57972.2023.10170630](https://doi.org/10.1109/INCET57972.2023.10170630). URL: <https://doi.org/10.1109/INCET57972.2023.10170630> (vid. pág. 50).
- [55] Aleksandar Milenkoski, Marco Vieira, Samuel Kounev, Alberto Avritzer y Bryan D. Payne. «Evaluating Computer Intrusion Detection Systems: A Survey of Common Practices». En: *ACM Computing Surveys* 48.1 (2015), 12:1-12:41. DOI: [10.1145/2808691](https://doi.org/10.1145/2808691) (vid. pág. 56).
- [56] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dhharshan Kumaran, Daan Wierstra, Shane Legg y Demis Hassabis. «Human-Level Control through Deep Reinforcement Learning». En: *Nature* 518.7540 (2015), págs. 529-533. DOI: [10.1038/nature14236](https://doi.org/10.1038/nature14236) (vid. págs. 47, 79, 82).
- [57] Nour Moustafa, Mohiuddin Ahmed y Sherif Ahmed. «Data Analytics-Enabled Intrusion Detection: Evaluations of ToN_IoT Linux Datasets». En: *2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. 2020, págs. 727-735. DOI: [10.1109/TrustCom50675.2020.00080](https://doi.org/10.1109/TrustCom50675.2020.00080) (vid. pág. 39).
- [58] Nour Moustafa y Jill Slay. «The Evaluation of Network Anomaly Detection Systems: Statistical Analysis of the UNSW-NB15 Data Set and the Comparison with the KDD99 Data Set». En: *Information Security Journal: A Global Perspective* 25.1-3 (2016), págs. 18-31. DOI: [10.1080/19393555.2015.1125974](https://doi.org/10.1080/19393555.2015.1125974) (vid. pág. 38).
- [59] Nour Moustafa y Jill Slay. «UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems (UNSW-NB15 Network Data Set)». En: *2015 Military Communications and Information Systems Conference*

- (*MilCIS*). 2015, págs. 1-6. DOI: [10.1109/MilCIS.2015.7348942](https://doi.org/10.1109/MilCIS.2015.7348942) (vid. págs. 38, 39).
- [60] Loc Gia Nguyen y Kohei Watabe. «Flow-Based Network Intrusion Detection Based on BERT Masked Language Model». En: *CoNEXT 2022 Student Workshop*. 2022. DOI: [10.1145/3565477.3569152](https://doi.org/10.1145/3565477.3569152). arXiv: [2306.04920](https://arxiv.org/abs/2306.04920) (vid. pág. 44).
- [61] Daniel Olszewski, Allison Lu, Carson Stillman, Kevin Warren, Cole Kitroser, Alejandro Pascual, Divyajyoti Ukirde, Kevin Butler y Patrick Traynor. «“Get in Researchers; We’re Measuring Reproducibility”: A Reproducibility Study of Machine Learning Papers in Tier 1 Security Conferences». En: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2023, págs. 3433-3459. DOI: [10.1145/3576915.3623130](https://doi.org/10.1145/3576915.3623130) (vid. págs. 57, 99, 138).
- [62] Akinyemi Oyelakin, A. O. Ameen, T. S. Ogundele, T. Salau-Ibrahim, U. T. Abdulrauf, H. I. Olufadi, I. K. Ajiboye, S. Muhammad-Thani e I. A. Adeniji. «Overview and Exploratory Analyses of CICIDS 2017 Intrusion Detection Dataset». En: *Journal of Systems Engineering and Information Technology (JOSEIT)* 2.2 (2023), págs. 45-52. DOI: [10.29207/joseit.v2i2.5411](https://doi.org/10.29207/joseit.v2i2.5411). URL: <https://jurnal.iaii.or.id/index.php/JOSEIT/article/view/5411> (vid. pág. 40).
- [63] Atilla Özgür y Hamit Erdem. «A Review of KDD99 Dataset Usage in Intrusion Detection and Machine Learning Between 2010 and 2015». En: *PeerJ Preprints* 4 (2016). Preprint, e1954v1. DOI: [10.7287/peerj.preprints.1954v1](https://doi.org/10.7287/peerj.preprints.1954v1). URL: <https://peerj.com/preprints/1954/> (vid. pág. 38).
- [64] Gregory Palmer, Chris Parry, Daniel J. B. Harrold y Chris Willis. «Deep Reinforcement Learning for Autonomous Cyber Defence: A Survey». En: (2023). arXiv: [2310.07745](https://arxiv.org/abs/2310.07745) [cs.CR] (vid. pág. 50).

- [65] Adrian Pekar y Richard Jozsa. «Evaluating ML-Based Anomaly Detection Across Datasets of Varied Integrity: A Case Study». En: *Computer Networks* 251 (2024), pág. 110617. DOI: [10.1016/j.comnet.2024.110617](https://doi.org/10.1016/j.comnet.2024.110617). URL: <https://doi.org/10.1016/j.comnet.2024.110617> (vid. pág. 41).
- [66] Kezhou Ren, Jinshu Zhu, Tao Yuan, Sheng Wen y Frank Jiang. «ID-RDRL: A Deep Reinforcement Learning-Based Feature Selection Intrusion Detection Model». En: *Scientific Reports* 12 (2022), pág. 15370. DOI: [10.1038/s41598-022-19366-3](https://doi.org/10.1038/s41598-022-19366-3) (vid. pág. 49).
- [67] Markus Ring, Sarah Wunderlich, Deniz Scheuring, Dieter Landes y Andreas Hotho. «A Survey of Network-Based Intrusion Detection Data Sets». En: *Computers & Security* 86 (2019), págs. 147-167. DOI: [10.1016/j.cose.2019.06.005](https://doi.org/10.1016/j.cose.2019.06.005). arXiv: [1903.02460](https://arxiv.org/abs/1903.02460) (vid. págs. 3, 21, 34, 38, 39, 44, 58, 59, 97).
- [68] María Rodríguez, Álvaro Alesanco, Lorena Mehavilla y José García. «Evaluation of Machine Learning Techniques for Traffic Flow-Based Intrusion Detection». En: *Sensors* 22.23 (2022), pág. 9326. DOI: [10.3390/s22239326](https://doi.org/10.3390/s22239326). URL: <https://www.mdpi.com/1424-8220/22/23/9326> (vid. pág. 43).
- [69] Arnaud Rosay, Eloïse Cheval, Florent Carlier y Pascal Leroux. «Network Intrusion Detection: A Comprehensive Analysis of CIC-IDS2017». En: *Proceedings of the 8th International Conference on Information Systems Security and Privacy (ICISSP)*. 2022, págs. 25-36. DOI: [10.5220/0010774000003120](https://doi.org/10.5220/0010774000003120) (vid. págs. 41, 54).
- [70] Takaya Saito y Marc Rehmsmeier. «The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets». En: *PLOS ONE* 10.3 (2015), e0118432. DOI: [10.1371/journal.pone.0118432](https://doi.org/10.1371/journal.pone.0118432) (vid. págs. 56, 99).

- [71] Hamed T. Sanusi. «Network Intrusion Detection Using Deep Reinforcement Learning». Tesis de maestría. Statesboro, Georgia, USA: Georgia Southern University, 2023. URL: <https://digitalcommons.georgiasouthern.edu/etd/2676/> (vid. pág. 50).
- [72] Mohanad Sarhan, Siamak Layeghy y Marius Portmann. «Evaluating Standard Feature Sets Towards Increased Generalisability and Explainability of ML-Based Network Intrusion Detection». En: (2021). Proposes standardised NetFlow-aligned feature mapping across CIC and UNSW-NB15 datasets. arXiv: [2104.07183 \[cs.CR\]](https://arxiv.org/abs/2104.07183) (vid. págs. 37, 44, 68).
- [73] Habeeb Mohammed Sayeeduddin y T. Ranga Babu. «Network Intrusion Detection System: A Survey on Artificial Intelligence-Based Techniques». En: *Expert Systems* (2022). DOI: [10.1111/exsy.13066](https://doi.org/10.1111/exsy.13066) (vid. pág. 44).
- [74] Karen A. Scarfone y Peter M. Mell. *Guide to Intrusion Detection and Prevention Systems (IDPS)*. Inf. téc. Special Publication 800-94. National Institute of Standards y Technology, 2007. URL: <https://csrc.nist.gov/pubs/sp/800/94/final> (vid. págs. 1, 32, 33, 57).
- [75] Tom Schaul, John Quan, Ioannis Antonoglou y David Silver. «Prioritized Experience Replay». En: *International Conference on Learning Representations*. 2016. arXiv: [1511.05952](https://arxiv.org/abs/1511.05952) (vid. pág. 47).
- [76] Jamshed Ali Shaikh, Chengliang Wang, Muhammad Wajeeh Us Sima, Muhammad Arshad, Muhammad Owais, Dina S. M. Hassan, Reem Alkanhel y Mohammed Saleh Ali Muthanna. «A Deep Reinforcement Learning-Based Robust Intrusion Detection System for Securing IoMT Healthcare Networks». En: *Frontiers in Medicine* 12 (2025), pág. 1524286. DOI: [10.3389/fmed.2025.1524286](https://doi.org/10.3389/fmed.2025.1524286). URL: <https://www.frontiersin.org/journals/medicine/articles/10.3389/fmed.2025.1524286/full> (vid. pág. 51).

- [77] Iman Sharafaldin, Arash Habibi Lashkari y Ali A. Ghorbani. «A Detailed Analysis of the CICIDS2017 Data Set». En: *Information Systems Security and Privacy*. Vol. 977. Communications in Computer and Information Science. Springer, 2019, págs. 172-188. DOI: [10.1007/978-3-030-25109-3_9](https://doi.org/10.1007/978-3-030-25109-3_9) (vid. pág. 40).
- [78] Iman Sharafaldin, Arash Habibi Lashkari y Ali A. Ghorbani. «Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization». En: *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. 2018, págs. 108-116. DOI: [10.5220/0006639801080116](https://doi.org/10.5220/0006639801080116). URL: <https://www.unb.ca/cic/datasets/ids-2017.html> (vid. págs. 7, 36, 39, 40, 89, 136, 164).
- [79] Robin Sommer y Vern Paxson. «Outside the Closed World: On Using Machine Learning for Network Intrusion Detection». En: *2010 IEEE Symposium on Security and Privacy (S&P)*. 2010, págs. 305-316. DOI: [10.1109/SP.2010.25](https://doi.org/10.1109/SP.2010.25) (vid. págs. 34, 53, 133, 137).
- [80] Anna Sperotto, Gregor Schaffrath, Ramin Sadre, Cristian Morariu, Aiko Pras y Burkhard Stiller. «An Overview of IP Flow-Based Intrusion Detection». En: *IEEE Communications Surveys & Tutorials* 12.3 (2010), págs. 343-356. DOI: [10.1109/SURV.2010.032210.00054](https://doi.org/10.1109/SURV.2010.032210.00054) (vid. págs. 2, 14, 35, 36, 65, 66).
- [81] Stable-Baselines3 Contributors. *QR-DQN: Stable Baselines3 Contrib Documentation*. Software documentation; access date 2026. 2023. URL: <https://sb3-contrib.readthedocs.io/en/master/modules/qrdqn.html> (vid. págs. 7, 18, 48, 83, 87).
- [82] Maxwell Standen, Martin Lucas, David Bowman, Toby J. Richer, Junae Kim y Damian Marriott. «CybORG: A Gym for the Development of Autonomous Cyber Agents». En: *arXiv preprint arXiv:2108.09118* (2021). Preprint; also cited as an IJCAI-21 Adaptive Cyber Defense workshop paper. arXiv: [2108.09118](https://arxiv.org/abs/2108.09118) [cs.CR] (vid. pág. 50).

- [83] Richard S. Sutton y Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2.^a ed. MIT Press, 2018. URL: <http://incompleteideas.net/book/the-book-2nd.html> (vid. págs. 3, 45, 58, 79, 97, 107, 109).
- [84] Mahbod Tavallaei, Ebrahim Bagheri, Wei Lu y Ali A. Ghorbani. «A Detailed Analysis of the KDD CUP 99 Data Set». En: *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*. 2009, págs. 1-6. DOI: [10.1109/CISDA.2009.5356528](https://doi.org/10.1109/CISDA.2009.5356528) (vid. págs. 38, 39).
- [85] Ashish Thakkar y Rachna Lohiya. «A Review of the Advancement in Intrusion Detection Datasets». En: *Procedia Computer Science* 167 (2020), págs. 636-645. DOI: [10.1016/j.procs.2020.03.270](https://doi.org/10.1016/j.procs.2020.03.270) (vid. pág. 38).
- [86] Muhammad Fahad Umer, Muhammad Sher y Yaxin Bi. «Flow-Based Intrusion Detection: Techniques and Challenges». En: *Computers & Security* 70 (2017), págs. 238-254. DOI: [10.1016/j.cose.2017.05.009](https://doi.org/10.1016/j.cose.2017.05.009) (vid. págs. 2, 14, 35, 36, 65, 66).
- [87] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot y Nando de Freitas. «Dueling Network Architectures for Deep Reinforcement Learning». En: *Proceedings of the 33rd International Conference on Machine Learning (ICML)*. Vol. 48. 2016, págs. 1995-2003. URL: <https://proceedings.mlr.press/v48/wangf16.html> (vid. pág. 47).
- [88] Christopher J. C. H. Watkins y Peter Dayan. «Q-Learning». En: *Machine Learning* 8.3-4 (1992), págs. 279-292. DOI: [10.1007/BF00992698](https://doi.org/10.1007/BF00992698) (vid. pág. 46).
- [89] Jianping Xiao, Chun Long, Jing Zhao, Jinxia Wei, Anlei Hu y Guanyao Du. «A Survey on Network Intrusion Detection Based on Deep Learning». En: *Frontiers of Data and Computing* 3.3 (2021), págs. 59-74. DOI: [10.11871/jfdic.issn.2096-742X.2021.03.006](https://doi.org/10.11871/jfdic.issn.2096-742X.2021.03.006) (vid. pág. 44).

- [90] Bin Yang, Muhammad Haseeb Arshad y Qing Zhao. «Packet-Level and Flow-Level Network Intrusion Detection Based on Reinforcement Learning and Adversarial Training». En: *Algorithms* 15.12 (2022), pág. 453. DOI: [10.3390/a15120453](https://doi.org/10.3390/a15120453). URL: <https://www.mdpi.com/1999-4893/15/12/453> (vid. pág. 51).
- [91] Wanrong Yang, Alberto Acuto, Yihang Zhou y Dominik Wojtczak. «A Survey for Deep Reinforcement Learning Based Network Intrusion Detection». En: *Applied AI Letters* 7.2 (2026), e70026. DOI: [10.1002/ai12.70026](https://doi.org/10.1002/ai12.70026). arXiv: [2410.07612](https://arxiv.org/abs/2410.07612) [cs.CR] (vid. págs. 32, 49, 58, 59).
- [92] Xin Zhang et al. «Unknown Intrusion Traffic Detection Method Based on Unsupervised Learning and Open-Set Recognition». En: *Scientific Reports* 15 (2025). DOI: [10.1038/s41598-025-01084-1](https://doi.org/10.1038/s41598-025-01084-1) (vid. pág. 145).

Manual de reproducibilidad

Este anexo describe el recorrido operativo desde una copia limpia del repositorio hasta la generación de los artefactos de entrenamiento, validación e inferencia. Los comandos se ejecutan desde la raíz del repositorio. El objetivo es hacer explícitas tanto las operaciones baratas como las que requieren los CSV completos o capacidad de cómputo dedicada; no se presupone que una orden pendiente haya sido ejecutada ni se le asignan resultados.

Se emplean tres marcas de coste:

- [Ligero]** No entrena un modelo ni necesita materializar o cargar el conjunto completo. Es adecuado para una máquina local, aunque la instalación inicial de dependencias pueda requerir una descarga grande.
- [Datos]** Necesita una entrada local o LFS de gran tamaño, como los ocho CSV de CICIDS2017 o el CSV de laboratorio, y puede consumir bastante memoria, CPU y tiempo de E/S aunque no entrene una red neuronal.
- [GPU]** Prepara o utiliza el runtime CUDA, o entrena QRDQN. Debe ejecutarse en una máquina con GPU y con espacio suficiente para los datos y los artefactos.

Una tarea puede llevar más de una marca. La disponibilidad de CUDA acelera algunas inferencias, pero no convierte por sí sola una comprobación en un entrenamiento.

A.1 Obtención del repositorio y de los datos

A.1.1 Clonado y Git LFS

[**Datos**]. Son requisitos previos Git, Git LFS, Python 3.12 y, para la instalación local recomendada, uv. Tras instalar Git LFS en el sistema, una copia limpia se prepara así:

```
git lfs install
git clone https://github.com/yeaight7/TFG_CYBER_AI.git
cd TFG_CYBER_AI
git lfs pull
git lfs ls-files
```

En esta revisión, `git lfs fsck` no se utiliza como puerta global: los patrones LFS actuales también alcanzan varios modelos y escaladores históricos que se comprometieron como objetos Git ordinarios antes de esa clasificación, por lo que la orden informa de `unexpectedGitObject` aunque los CSV estén disponibles. La aceptación de los datos se basa en los ocho CSV materializados y en sus hashes documentados.

La descarga debe dejar ocho exportaciones CSV curadas bajo `datasets/CICIDS2017/`. Un fichero que todavía contiene el pequeño puntero textual de Git LFS no es un CSV utilizable. Los nombres esperados y los SHA-256 de la copia curada están inventariados en `README.md`. La integridad puede contrastarse con una de estas órdenes, según el sistema:

```
sha256sum datasets/CICIDS2017/*.csv
```

```
Get-FileHash -Algorithm SHA256 datasets/CICIDS2017/*.csv
```

A.1.2 Condiciones y procedencia de CICIDS2017

CICIDS2017 procede del Canadian Institute for Cybersecurity de la University of New Brunswick (Sharafaldin et al. 2018). Su página oficial es <https://www.unb.ca/cic/datasets/ids-2017.html>. El conjunto se ofrece para uso investigador y exige cita y atribución; la autorización para su redistribución no está expresamente concedida. Por ello, el repositorio no relicencia los CSV con AGPL-3.0 y se debe consultar la fuente oficial antes de copiarlos o publicarlos. La reproducción byte a byte de esta tesis utiliza la copia *curada* materializada por Git LFS y sus hashes documentados. Los CSV oficiales descargados de CIC/UNB son la fuente de procedencia, pero conservan columnas previas a la curación: sus bytes no coinciden con esos hashes y sustituir directamente la copia curada cambia la entrada experimental. La misma cautela se aplica a los modelos y artefactos derivados del conjunto.

NSL-KDD no es un requisito de este flujo. Sus datos y su antiguo modelo se retiraron del estado actual del repositorio; el adaptador conservado tiene valor histórico, pero no forma parte del entrenamiento MAIN ni de la Fase 2.

A.2 Instalación del entorno

A.2.1 Entorno local con uv

[Ligero]. El fichero `uv.lock` y la configuración de `pyproject.toml` fijan el entorno local. La instalación reproducible se realiza sin regenerar el lockfile:

```
uv lock --check
uv sync --all-extras --frozen
uv run python --version
```

En Windows, la fuente condicional de `torch` selecciona la rueda de CPU; en Linux selecciona el índice CUDA 13.0 definido en el proyecto. Este camino es el adecuado para desarrollo y comprobaciones locales. No debe confundirse con la reproducción directa del entorno de RunPod descrita a continuación.

A.2.2 Entorno RunPod mediante venv

[GPU]. La ejecución MAIN registró Python 3.12.11, Linux x86_64, una RTX 3090 Ti, `torch 2.12.1+cu130` y CUDA 13.0. El fichero `requirements-runpod-cu130.txt` fija la pila de entrenamiento directo para esa familia de entorno. En una instancia RunPod con Python 3.12:

```
python --version
python -m venv venv
source venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements-runpod-cu130.txt
```

El registro exacto que se debe usar para comparar versiones es `runs/cicids2017/MAIN_qrdqn_cicids2017_canonical_full_random_20260609_193655/environment.json`. Además de PyTorch, contiene las versiones efectivas de NumPy, pandas, scikit-learn, Gymnasium, Stable-Baselines3, `sb3-contrib` y joblib.

En los bloques posteriores, `python` designa el intérprete del `venv` activado en RunPod. En la ruta local con `uv`, donde no se activa manualmente ese entorno, se antepone `uv run` a cada orden que comience por `python`; el script y el resto de argumentos no cambian. Los bloques multilínea usan la barra invertida de continuación de POSIX. En PowerShell se unen en una sola línea o se sustituye cada barra de continuación por el acento grave ‘.

A.2.3 Comprobación mínima del entorno

[**Ligero**]. En local, las siguientes órdenes comprueban los imports, la dimensión canónica y la disponibilidad efectiva de CUDA sin cargar los CSV:

```
uv run python -c "import torch; print(torch.__version__,
torch.version.cuda, torch.cuda.is_available())"
uv run python -c "import stable_baselines3 as sb3,
sb3_contrib; print(sb3.__version__,
sb3_contrib.__version__)"
uv run python -c "from src.canonical_schema import
FEATURES_CANON; assert len(FEATURES_CANON) == 76; print(76,
152)"
```

Dentro del `venv` de RunPod se repiten las mismas órdenes retirando el prefijo `uv run`. En una instancia destinada a entrenar, la tercera salida de la primera orden debe ser `True`; si es `False`, no se debe iniciar MAIN hasta corregir el runtime o la selección de la GPU. La salida local puede ser `False` sin que ello indique un fallo del pipeline de CPU.

A.3 Entrenamiento canónico

A.3.1 Smoke test del perfil MAIN

[**Datos**]. Esta comprobación usa solo una muestra pequeña, pero atraviesa el cargador, el esquema canónico, la construcción del entorno, el perfil de hiperparámetros MAIN y el guardado de artefactos:

```
python src/train_rl_defender.py \
--preset fast \
--split-mode random \
```

```
--max-rows 10000 \  
--timesteps 1000 \  
--seed 42 \  
--training-profile main-experiment \  
--checkpoint-freq 0
```

Este smoke test queda por debajo de los 50 000 pasos de *learning starts* del perfil MAIN. Solo valida el recorrido y la escritura de artefactos; sus métricas no son evidencia de calidad de aprendizaje ni son comparables con MAIN.

A.3.2 Entrenamiento MAIN exacto

[GPU] + [Datos]. La orden canónica del experimento principal es:

```
python src/train_rl_defender.py \  
--preset full \  
--split-mode random \  
--timesteps 3000000 \  
--seed 42 \  
--training-profile main-experiment
```

La omisión deliberada de `--max-rows` hace que se usen todos los datos. El perfil fija, entre otros valores, la arquitectura `[1024,1024,512]`, 200 cuantiles, $\gamma = 0$, tasa de aprendizaje `5e-5`, un millón de posiciones de buffer y tres millones de pasos. El código actual utiliza la recompensa `tp=1.5`, `fp=-2.0`, `fn=-5.0` y `omission=0.0`.

Cada ejecución genera un `RUN_ID` y debe terminar con `config.json`, `metrics.json`, `environment.json`, `feature_names.json`, `scaler.joblib`, `train_percentiles.npz`, `artifact_manifest.json` y el modelo. No se debe trasladar una métrica a la memoria si no puede vincularse con ese directorio y su configuración.

A.4 Verificación del split fijo y del escalador

[Datos]. Esta comprobación no entrena. Reconstruye la partición aleatoria completa con semilla 42, comprueba los conteos, la anidación de los subconjuntos de entrenamiento, el hash del test y el ajuste del `StandardScaler` frente al artefacto MAIN:

```
python scripts/verify_fixed_test_split.py \  
  --sizes 500000 1000000 \  
  --seed 42 \  
  --run-dir \  
    runs/cicids2017/MAIN_qrdqn_cicids2017_canonical_full_  
random_20260609_193655 \  
  --check-scaler \  
    runs/cicids2017/MAIN_qrdqn_cicids2017_canonical_full_  
random_20260609_193655/scaler.joblib
```

El hash que imprime debe coincidir con el manifiesto y el valor siguientes (las dos líneas del hash se concatenan sin espacios):

```
runs/cicids2017/test_partition_reference_seed42.json  
test_set_sha256:  
cb175377f462672b3aeb3ad3dd8bff3a  
b506d8663755cb336d40f3ef9765035b
```

El éxito de esta orden acredita la reconstrucción del split y del escalador, no una repetición del entrenamiento ni de sus pesos.

Los artefactos serializados se resuelven por defecto mediante `artifact_manifest.json` y se verifican con SHA-256 antes de cargarlos. El hash acredita integridad respecto del manifiesto, pero no convierte en seguro un serializado procedente de una fuente hostil: también deben ser confiables la procedencia del manifiesto y la del repositorio. La opción `--allow-unsafe-artifacts`

desactiva esa garantía y exige rutas directas. No debe añadirse para modelos o escaladores recibidos por Internet, correo o una fuente no controlada. Solo es admisible, de forma consciente, para un artefacto local confiable que todavía no tenga manifiesto; ningún comando canónico de este anexo la necesita.

A.5 Análisis y validaciones reproducibles

A.5.1 Intervalos bootstrap

[**Ligero**]. El modo normal recupera y autoverifica los conteos de la matriz de confusión desde `config.json` y `metrics.json`, y después realiza 10 000 remuestreos estratificados. No carga el modelo, los CSV ni reentrena:

```
python scripts/bootstrap_ci.py \
  --run-dir \
    runs/cicids2017/MAIN_qrdqn_cicids2017_canonical_full_
random_20260609_193655 \
  --n-boot 10000 \
  --boot-seed 12345 \
  --out runs/validation/bootstrap_ci_seed42_recomputed.json
```

La salida recomputada se compara con el artefacto oficial y no lo sustituye. Usar directamente la ruta `runs/validation/bootstrap_ci_seed42.json` sobreescribiría evidencia rastreada y eliminaría la verificación pesada del modelo si no se añade también `--from-model`.

[**Datos**], **GPU opcional**. El modo `--from-model` es distinto: reconstruye los 566 149 ejemplos de prueba, aplica el escalador persistido, vuelve a predecir con el modelo MAIN y exige que la matriz de confusión coincida. Sigue sin reentrenar, pero ya no es una operación ligera:

```
python scripts/bootstrap_ci.py \
```

```
--run-dir \  
    runs/cicids2017/MAIN_qrdqn_cicids2017_canonical_full_  
random_20260609_193655 \  
--n-boot 10000 \  
--boot-seed 12345 \  
--from-model \  
--out runs/validation/bootstrap_ci_seed42_from_model.json
```

A.5.2 Análisis de duplicados

[Datos]. El script carece de argumentos y no entrena ningún modelo:

```
python scripts/analyze_duplicates.py
```

Carga la representación canónica sin escalar, mide duplicados exactos y la presencia de filas de test también presentes en train, y escribe `runs/validation/duplicate_analysis_seed42.json`. Es un análisis de la partición aleatoria, no una corrección automática de los datos. Como el script no admite una ruta de salida alternativa, debe ejecutarse en una copia de trabajo limpia y se debe revisar explícitamente `git diff --runs/validation/duplicate_analysis_seed42.json`. La orden sobrescribe ese fichero rastreado; si la ejecución es solo una auditoría, se conserva la salida fuera del árbol y se restaura el original en vez de reemplazar evidencia histórica sin explicar cualquier diferencia introducida por el cargador o los datos actuales.

A.5.3 Línea base Random Forest

[Datos]. La línea base tampoco acepta argumentos:

```
python src/baseline_random_forest.py
```

La orden entrena tres barridos de CPU: partición aleatoria completa, partición por día y un *leave-one-out* con el miércoles como test. Escribe configuración y métricas por barrido bajo `runs/cicids2017/baseline_random_forest_comparison/`, además del modelo de la partición aleatoria bajo `models/`. Que no requiera GPU no implica que sea una comprobación ligera: carga varias veces el conjunto completo y entrena 200 árboles por barrido.

A.5.4 Checks A, B y C con el código actual

[GPU] + [Datos]. La ejecución conjunta actual, con todos los valores relevantes explicitados, es:

```
python src/validate_checks.py \
    --run-dir \
        runs/cicids2017/MAIN_qrdqn_cicids2017_canonical_full_
random_20260609_193655 \
    --checks A B C \
    --preset full \
    --split-mode random \
    --train-csvs Monday Tuesday Wednesday \
    --test-csvs Thursday Friday \
    --timesteps-b 10000 \
    --timesteps-c 30000 \
    --seed 42
```

Check A carga el modelo MAIN verificado y predice directamente sobre X_{test} . Check B destruye la asociación con las etiquetas barajando y_{train} y entrena un modelo breve. Check C entrena otro modelo con lunes, martes y miércoles como train y jueves y viernes como test. Por tanto, la orden conjunta contiene reentrenamiento; no es una mera evaluación del modelo MAIN. La salida se guarda en un nuevo directorio `runs/validation/VAL_checks_ABC_<timestamp>/` con `config.json` y `validation_results.json`.

Los tres artefactos históricos citados en la memoria — `VAL_checks_A_20260212_235443`, `VAL_checks_B_20260212_235736` y `VAL_checks_C_20260213_004847`— siguen siendo auditables mediante sus ficheros de configuración y resultados, pero no son reproducibles byte a byte con el estado actual. Sus configuraciones registran `fp=-1.0`, mientras que el código actual usa `fp=-2.0`. Además, Check A apunta a `models/C01_qrdqn_cicids2017_canonical_full_20260212_200218.zip`, modelo ya retirado, y Check B y Check C no persistieron los modelos que entrenaron. Una nueva ejecución debe recibir un nuevo `RUN_ID` y no sustituye en silencio a esos artefactos históricos.

A.6 Inferencia MAIN de la Fase 2

[Datos], GPU opcional. La Fase 2 mantenida es inferencia offline; no instala reglas de cortafuegos ni bloquea tráfico en vivo. La orden con el modelo MAIN y el CSV local del laboratorio es:

```
python scripts/predict_real_traffic_v2.py \
  --flows pcaps/lab_capture_traffic.csv \
  --run-dir \
    runs/cicids2017/MAIN_qrdqn_cicids2017_canonical_full_
random_20260609_193655 \
  --percentiles \
    runs/cicids2017/MAIN_qrdqn_cicids2017_canonical_full_
random_20260609_193655/train_percentiles.npz \
  --clip-z 10.0 \
  --export-diagnostics
```

`pcaps/lab_capture_traffic.csv` está ignorado por Git y no aparece en un clon limpio. Debe aportarlo el operador a partir de la captura y extracción

descritas en `pcaps/README.md`; sin esa entrada local no puede repetirse la inferencia oficial de Fase 2.

`--run-dir` resuelve y verifica por manifiesto el modelo y el escalador; el fichero de percentiles indicado también debe coincidir con su hash registrado. La orden escribe un nuevo `runs/phase2/P2v2_pred_<timestamp>/` con predicciones, configuración, métricas y diagnósticos. El recorte por percentiles y por z-score es una mitigación deliberada del cambio de dominio en inferencia: el entrenamiento y el test interno de MAIN solo aplicaron `StandardScaler`. Por ello, una ejecución con estos recortes no es comparable byte a byte con las métricas del test interno. Para una ablación estrictamente alineada con el preprocesamiento de entrenamiento se omiten `--percentiles` y `--clip-z`, manteniendo el escalador.

No se activa `--include-sensitive-metadata` en la orden canónica. Si se usa para una inspección local, el fichero `predictions_sensitive_local.csv` puede contener IP, puertos, marcas de tiempo o etiquetas y no debe añadirse al control de versiones.

A.7 Experimentos diferidos con GPU

Los flujos siguientes están implementados o definidos, pero permanecen pendientes. Estos comandos documentan el procedimiento futuro; no implican que existan métricas ni autorizan a completar tablas con valores estimados. Cada ejecución válida deberá conservar su `RUN_ID`, configuración y resultados bajo `runs/`.

A.7.1 Leave-one-CSV-out de QRDQN

`[GPU] + [Datos]`. Sin `--holdout-csvs`, el script recorre los ocho CSV y entrena un modelo por fold:

```
python src/validate_leave_one_csv_out.py \
    --timesteps 30000 \
    --seed 42 \
    --predict-batch-size 8192
```

El uso de `--max-rows-per-csv` queda reservado a smoke tests y no debe presentarse como la validación completa. En el estado actual no existe un artefacto agregado completo de QRDQN que permita reportar resultados de este protocolo.

A.7.2 Escalera de tamaños con test fijo

[GPU] + [Datos]. El presupuesto se reduce solo en train mediante `--train-max-rows`; usar `--max-rows` alteraría el test y haría inválida la comparación. Los pares planificados de filas y pasos, proporcionales a los tres millones de pasos de MAIN mediante $T = \text{round}(3\,000\,000\, N/2\,264\,594)$, son:

Filas de train	Pasos QRDQN
100 000	132 474
250 000	331 185
500 000	662 370
1 000 000	1 324 741
2 000 000	2 649 482

Para cada fila de la tabla se ejecuta la orden siguiente, sustituyendo N y T por el par exacto de la misma fila:

```
python src/train_rl_defender.py \
    --preset full \
    --split-mode random \
    --train-max-rows N \
    --timesteps T \
```

```
--seed 42 \  
--training-profile main-experiment
```

Después de cada ejecución se compara `split_metadata.test_set_sha256` con el manifiesto de referencia y se conserva el escalador propio del subconjunto de train. No hay resultados comprometidos para esta escalera en el momento de redactar el anexo.

A.7.3 Estudio multisentilla del pipeline

[GPU] + [Datos]. Para caracterizar la sensibilidad conjunta del pipeline se repite la orden MAIN completa, sin cambiar ningún otro indicador, con las semillas 43, 44 y 45. En el código actual, `--seed` controla tanto la partición aleatoria como la inicialización y el muestreo del entrenamiento; por ello, este protocolo mide variabilidad combinada de split y entrenamiento, no varianza de entrenamiento sobre un test fijo. Aislar este último efecto exigiría separar la semilla de partición de la semilla del modelo, interfaz que el CLI actual no ofrece. La primera repetición es:

```
python src/train_rl_defender.py \  
--preset full \  
--split-mode random \  
--timesteps 3000000 \  
--seed 43 \  
--training-profile main-experiment
```

Se repite exactamente con `--seed 44` y `--seed 45`. Cada semilla genera un modelo y un `RUN_ID` independientes. Hasta que las tres ejecuciones terminen y sus artefactos sean revisados, la tesis solo puede declarar que MAIN es una ejecución de semilla única; no puede atribuirle una desviación entre semillas. Incluso después de ejecutar este protocolo, la dispersión deberá describirse

como variabilidad combinada y no como una estimación pura de la varianza de entrenamiento.

Entorno hardware y software

Este anexo documenta el entorno utilizado para obtener la ejecución principal **MAIN** y lo separa del entorno local de desarrollo y de la cadena de construcción de esta memoria. La distinción es necesaria porque las fuentes responden a preguntas diferentes: la confirmación del autor identifica el hardware contratado, `environment.json` registra el software observado por el proceso de entrenamiento y los manifiestos de dependencias definen instalaciones reproducibles, pero no acreditan por sí solos qué paquetes llegaron a cargarse en una ejecución concreta.

B.1 Hardware efectivamente empleado

La ejecución `MAIN_qrdqn_cicids2017_canonical_full_random_20260609_193655` se entrenó en el entorno remoto de la Tabla [B.1](#). El ordenador portátil local se reservó para desarrollo, pruebas y construcción del documento; no se utilizó para entrenar modelos.

Recurso	Especificación	Uso	Procedencia
CPU remota	AMD EPYC 9005	Entrenamiento de la ejecución MAIN	Confirmación del autor
Memoria remota	128 GB de RAM	Carga de datos y entrenamiento de MAIN	Confirmación del autor
GPU remota	NVIDIA GeForce RTX 3090 Ti, 24 GB de VRAM	Entrenamiento CUDA de MAIN	Confirmación del autor; el artefacto corrobora el modelo de GPU
Ordenador portátil local	Sin atribuirle especificaciones de entrenamiento	Desarrollo, pruebas y construcción de la memoria; ningún entrenamiento de modelos	Uso confirmado por el autor

Tabla B.1: Hardware empleado y función de cada entorno. Los datos del equipo remoto proceden de la confirmación del autor; `environment.json` corrobora de forma independiente el modelo de GPU.

No se infiere un modelo más concreto del procesador ni un número de núcleos. El artefacto de ejecución tampoco contiene la capacidad de RAM o VRAM: únicamente corrobora el nombre de la GPU. Por ello, esos datos se mantienen explícitamente como información confirmada por el autor y no como campos extraídos del artefacto.

B.2 Runtime registrado en MAIN

El fichero `environment.json` situado dentro del directorio de la ejecución MAIN constituye la fuente primaria del entorno observado durante el entrenamiento. La Tabla B.2 transcribe sus campos de plataforma y aceleración sin extenderlos a otros equipos.

Componente	Valor registrado	Campo del artefacto
Python	3.12.11, compilado con GCC 13.3.0	python_version
Plataforma	Linux 6.8.0-110-generic, x86_64, glibc 2.39	platform
Dispositivo	cuda; CUDA disponible para PyTorch	device, torch_cuda_available
GPU visible	NVIDIA GeForce RTX 3090 Ti	cuda_device_name
CUDA informada por PyTorch	13.0	torch_cuda_version
cuDNN	9.20.0 (valor entero registrado: 92000)	torch_cudnn_version

Tabla B.2: Runtime observado durante la ejecución principal. Fuente: `environment.json` de la ejecución MAIN.

El mismo artefacto registra las versiones de paquete de la Tabla B.3. Estas son versiones observadas en MAIN, no una reconstrucción a partir del estado actual del entorno local.

Paquete	Versión	Paquete	Versión
torch	2.12.1+cu130	numpy	2.4.6
pandas	3.0.3	scikit-learn	1.9.0
gymnasium	1.2.3	stable-baselines3	2.8.0
sb3-contrib	2.8.0	joblib	1.5.3

Tabla B.3: Versiones exactas de los paquetes registrados por `environment.json` para la ejecución MAIN.

B.3 Contratos de dependencias

Los manifiestos del repositorio convergen en el núcleo de versiones de la Tabla B.3, con la salvedad de que la variante CUDA de `torch` se expresa de forma distinta según la ruta de instalación. Sus diferencias se resumen en la Tabla B.4.

Contrato	Ámbito	Contenido diferenciador
<code>requirements-runpod-cu130.txt</code>	Runtime mínimo de entrenamiento y evaluación en RunPod	Fija <code>torch==2.12.1+cu130</code> y el índice CUDA 13.0. Añade <code>tensorboard==2.20.0</code> y <code>matplotlib==3.10.9</code> . Omite de forma deliberada los extras <code>dev</code> y <code>tune</code> ; para ajustar hiperparámetros debe añadirse <code>optuna==4.9.0</code> .
<code>requirements.txt</code>	Instalación directa genérica o local	Fija <code>torch==2.12.1</code> sin sufijo de plataforma. Incluye el mismo núcleo, TensorBoard, Matplotlib y <code>optuna==4.9.0</code> ; no incorpora las herramientas de desarrollo <code>pytest</code> y <code>ruff</code> .
<code>pyproject.toml</code>	Proyecto y resolución con <code>uv</code>	Exige Python <code>>=3.12,<3.13</code> . El núcleo usa <code>stable-baselines3[extra]==2.8.0</code> ; el extra <code>dev</code> fija <code>pytest==9.0.3</code> y <code>ruff==0.15.12</code> , y <code>tune</code> fija <code>optuna==4.9.0</code> . <code>tool.uv.sources</code> dirige <code>torch</code> al índice <code>cu130</code> en Linux y al índice CPU fuera de Linux.

Tabla B.4: Contratos de dependencias y diferencias entre el runtime GPU, la instalación local, el desarrollo y el ajuste de hiperparámetros. Fuente: los tres manifiestos indicados.

Por tanto, `environment.json` acredita las versiones observadas de PyTorch, NumPy, pandas, scikit-learn, Gymnasium, Stable-Baselines3, sb3-contrib y joblib. Las versiones de TensorBoard, Matplotlib, Optuna, pytest y Ruff pertenecen a los contratos de instalación correspondientes; el artefacto de MAIN no las registra y no se presentan aquí como paquetes observados durante esa ejecución.

B.4 Entorno de construcción de la memoria

La cadena de construcción del documento es independiente del runtime Linux/CUDA del modelo. Según el tracker de mejora de la memoria, el documento se construye en Windows con la pila de la Tabla B.5. Ninguno de estos componentes forma parte del entrenamiento ni de la evaluación del agente.

Componente	Versión	Función
Sistema anfitrión	Windows	Ejecución de la cadena de construcción documental
Distribución TeX	TeX Live 2025	Motor y paquetes LaTeX de la memoria
latexmk	4.87	Orquestación de las pasadas de compilación
biber	2.21	Procesamiento de la bibliografía

Tabla B.5: Pila de construcción de la memoria. Fuente: tracker de mejora de la memoria; entorno distinto del runtime empleado para MAIN.

Esquema canónico completo

Este anexo fija la enumeración completa del contrato de entrada descrito de forma agrupada en el Capítulo 4. El orden procede de `FEATURES_CANON`, definido en `src/canonical_schema.py`. Para una característica canónica de índice i , con $1 \leq i \leq 76$, su valor x_i ocupa la posición i del vector de observación y su indicador m_i ocupa la posición $76 + i$. Por tanto, las posiciones 1–76 contienen los valores canónicos y las posiciones 77–152 contienen, en el mismo orden, los indicadores cuyos nombres persistidos siguen el patrón `m_<identificador_canónico>`.

En CICIDS2017, la máscara registra la presencia de la columna fuente tras la limpieza previa al mapeo: los valores no finitos ya se han sustituido. Por tanto, 1 indica que la columna mapeada existe en ese punto del pipeline, no que preserve ausencias fila a fila. Como las 76 columnas tienen correspondencia, en los datos nativos de entrenamiento las posiciones 77–152 valen siempre 1. Ante otra entrada sin columna mapeada, el valor se imputa con cero y su indicador permanece en 0.

La Tabla C.1 conserva el orden y muestra las dos correspondencias mantenidas por el proyecto, ambas completas para los 76 identificadores. En Flowmeter.py, la presencia efectiva depende de las columnas de cada CSV. La tabla no añade unidades ni descripciones ajenas a las fuentes.

Tabla C.1: Esquema canónico completo y correspondencias de columnas fuente. Fuente: elaboración propia a partir de `src/canonical_schema.py` y `scripts/predict_real_traffic_v2.py`.

Índice	Identificador canónico	Columna fuente CICIDS2017	Columna fuente Flowmeter- py	Grupo lógico
1	<code>flow_duration</code>	Flow Duration	<code>flow_duration</code>	Estadísticas generales del flujo
2	<code>total_fwd_packets</code>	Total Fwd Packets	<code>tot_fwd_pkts</code>	Estadísticas generales del flujo
3	<code>total_bwd_packets</code>	Total Backward Packets	<code>tot_bwd_pkts</code>	Estadísticas generales del flujo
4	<code>total_length_of_fwd_packets</code>	Total Length of Fwd Packets	<code>totlen_fwd_pkts</code>	Estadísticas generales del flujo
5	<code>total_length_of_bwd_packets</code>	Total Length of Bwd Packets	<code>totlen_bwd_pkts</code>	Estadísticas generales del flujo
6	<code>fwd_packet_length_max</code>	Fwd Packet Length Max	<code>fwd_pkt_len_max</code>	Tamaño de paquetes (hacia adelante)
7	<code>fwd_packet_length_min</code>	Fwd Packet Length Min	<code>fwd_pkt_len_min</code>	Tamaño de paquetes (hacia adelante)
8	<code>fwd_packet_length_mean</code>	Fwd Packet Length Mean	<code>fwd_pkt_len_mean</code>	Tamaño de paquetes (hacia adelante)

Continúa en la página siguiente

Tabla C.1 (continuación)

Índice	Identificador canónico	Columna fuente CICIDS2017	Columna fuente Flowmeter- py	Grupo lógico
9	fwd_packet_length_std	Fwd Packet Length Std	fwd_pkt_len_std	Tamaño de paquetes (hacia adelante)
10	bwd_packet_length_max	Bwd Packet Length Max	bwd_pkt_len_max	Tamaño de paquetes (hacia atrás)
11	bwd_packet_length_min	Bwd Packet Length Min	bwd_pkt_len_min	Tamaño de paquetes (hacia atrás)
12	bwd_packet_length_mean	Bwd Packet Length Mean	bwd_pkt_len_mean	Tamaño de paquetes (hacia atrás)
13	bwd_packet_length_std	Bwd Packet Length Std	bwd_pkt_len_std	Tamaño de paquetes (hacia atrás)
14	flow_bytes_per_s	Flow Bytes/s	flow_byts_s	Tasas de flujo
15	flow_packets_per_s	Flow Packets/s	flow_pkts_s	Tasas de flujo
16	flow_iat_mean	Flow IAT Mean	flow_iat_mean	Tiempo entre llegadas (flujo)
17	flow_iat_std	Flow IAT Std	flow_iat_std	Tiempo entre llegadas (flujo)
18	flow_iat_max	Flow IAT Max	flow_iat_max	Tiempo entre llegadas (flujo)
19	flow_iat_min	Flow IAT Min	flow_iat_min	Tiempo entre llegadas (flujo)

Continúa en la página siguiente

Tabla C.1 (continuación)

Índice	Identificador canónico	Columna fuente CICIDS2017	Columna fuente Flowmeter- py	Grupo lógico
20	fwd_iat_total	Fwd IAT Total	fwd_iat_tot	Tiempo entre llegadas (hacia adelante)
21	fwd_iat_mean	Fwd IAT Mean	fwd_iat_mean	Tiempo entre llegadas (hacia adelante)
22	fwd_iat_std	Fwd IAT Std	fwd_iat_std	Tiempo entre llegadas (hacia adelante)
23	fwd_iat_max	Fwd IAT Max	fwd_iat_max	Tiempo entre llegadas (hacia adelante)
24	fwd_iat_min	Fwd IAT Min	fwd_iat_min	Tiempo entre llegadas (hacia adelante)
25	bwd_iat_total	Bwd IAT Total	bwd_iat_tot	Tiempo entre llegadas (hacia atrás)
26	bwd_iat_mean	Bwd IAT Mean	bwd_iat_mean	Tiempo entre llegadas (hacia atrás)
27	bwd_iat_std	Bwd IAT Std	bwd_iat_std	Tiempo entre llegadas (hacia atrás)

Continúa en la página siguiente

Tabla C.1 (continuación)

Índice	Identificador canónico	Columna fuente CICIDS2017	Columna fuente Flowmeter- py	Grupo lógico
28	bwd_iat_max	Bwd IAT Max	bwd_iat_max	Tiempo entre llegadas (hacia atrás)
29	bwd_iat_min	Bwd IAT Min	bwd_iat_min	Tiempo entre llegadas (hacia atrás)
30	fwd_psh_flags	Fwd PSH Flags	fwd_psh_flags	Indicadores TCP
31	bwd_psh_flags	Bwd PSH Flags	bwd_psh_flags	Indicadores TCP
32	fwd_urg_flags	Fwd URG Flags	fwd_urg_flags	Indicadores TCP
33	bwd_urg_flags	Bwd URG Flags	bwd_urg_flags	Indicadores TCP
34	fin_flag_count	FIN Flag Count	fin_flag_cnt	Indicadores TCP
35	syn_flag_count	SYN Flag Count	syn_flag_cnt	Indicadores TCP
36	rst_flag_count	RST Flag Count	rst_flag_cnt	Indicadores TCP
37	psh_flag_count	PSH Flag Count	psh_flag_cnt	Indicadores TCP
38	ack_flag_count	ACK Flag Count	ack_flag_cnt	Indicadores TCP
39	urg_flag_count	URG Flag Count	urg_flag_cnt	Indicadores TCP
40	cwe_flag_count	CWE Flag Count	cwr_flag_count	Indicadores TCP
41	ece_flag_count	ECE Flag Count	ece_flag_cnt	Indicadores TCP

Continúa en la página siguiente

Tabla C.1 (continuación)

Índice	Identificador canónico	Columna fuente CICIDS2017	Columna fuente Flowmeter- py	Grupo lógico
42	fwd_header_length	Fwd Header Length	fwd_header_len	Longitud de cabecera
43	bwd_header_length	Bwd Header Length	bwd_header_len	Longitud de cabecera
44	fwd_packets_per_s	Fwd Packets/s	fwd_pkts_s	Paquetes por segundo (por di- rección)
45	bwd_packets_per_s	Bwd Packets/s	bwd_pkts_s	Paquetes por segundo (por di- rección)
46	min_packet_length	Min Packet Length	pkt_len_min	Longitud de paquete (global)
47	max_packet_length	Max Packet Length	pkt_len_max	Longitud de paquete (global)
48	packet_length_mean	Packet Length Mean	pkt_len_mean	Longitud de paquete (global)
49	packet_length_std	Packet Length Std	pkt_len_std	Longitud de paquete (global)
50	packet_length_variance	Packet Length Variance	pkt_len_var	Longitud de paquete (global)
51	down_up_ratio	Down/Up Ratio	down_up_ratio	Ratios y estadísticas derivadas
52	average_packet_size	Average Packet Size	pkt_size_avg	Ratios y estadísticas derivadas
53	avg_fwd_segment_size	Avg Fwd Segment Size	fwd_seg_size_avg	Ratios y estadísticas derivadas
54	avg_bwd_segment_size	Avg Bwd Segment Size	bwd_seg_size_avg	Ratios y estadísticas derivadas
55	fwd_avg_bytes_per_bulk	Fwd Avg Bytes/Bulk	fwd_byts_b_avg	Ráfagas (hacia adelante)

Continúa en la página siguiente

Tabla C.1 (continuación)

Índice	Identificador canónico	Columna fuente CICIDS2017	Columna fuente Flowmeter- py	Grupo lógico
56	fwd_avg_packets_per_bulk	Fwd Avg Packets/Bulk	fwd_pkts_b_avg	Ráfagas (hacia adelante)
57	fwd_avg_bulk_rate	Fwd Avg Bulk Rate	fwd_blk_rate_avg	Ráfagas (hacia adelante)
58	bwd_avg_bytes_per_bulk	Bwd Avg Bytes/Bulk	bwd_byts_b_avg	Ráfagas (hacia atrás)
59	bwd_avg_packets_per_bulk	Bwd Avg Packets/Bulk	bwd_pkts_b_avg	Ráfagas (hacia atrás)
60	bwd_avg_bulk_rate	Bwd Avg Bulk Rate	bwd_blk_rate_avg	Ráfagas (hacia atrás)
61	subflow_fwd_packets	Subflow Fwd Packets	subflow_fwd_pkts	Subflujos
62	subflow_fwd_bytes	Subflow Fwd Bytes	subflow_fwd_byts	Subflujos
63	subflow_bwd_packets	Subflow Bwd Packets	subflow_bwd_pkts	Subflujos
64	subflow_bwd_bytes	Subflow Bwd Bytes	subflow_bwd_byts	Subflujos
65	init_win_bytes_forward	Init_Win_bytes_forward	init_fwd_win_byts	Ventana TCP
66	init_win_bytes_backward	Init_Win_bytes_backward	init_bwd_win_byts	Ventana TCP
67	act_data_pkt_fwd	act_data_pkt_fwd	fwd_act_data_pkts	Datos activos
68	min_seg_size_forward	min_seg_size_forward	fwd_seg_size_min	Datos activos

Continúa en la página siguiente

Tabla C.1 (continuación)

Índice	Identificador canónico	Columna fuente CICIDS2017	Columna fuente Flowmeter- py	Grupo lógico
69	active_mean	Active Mean	active_mean	Tiempos activos/inactivos
70	active_std	Active Std	active_std	Tiempos activos/inactivos
71	active_max	Active Max	active_max	Tiempos activos/inactivos
72	active_min	Active Min	active_min	Tiempos activos/inactivos
73	idle_mean	Idle Mean	idle_mean	Tiempos activos/inactivos
74	idle_std	Idle Std	idle_std	Tiempos activos/inactivos
75	idle_max	Idle Max	idle_max	Tiempos activos/inactivos
76	idle_min	Idle Min	idle_min	Tiempos activos/inactivos

Catálogo de artefactos

Este anexo amplía el inventario resumido de la Tabla 5.2. Su objetivo no es repetir las métricas del Capítulo 6, sino identificar qué fichero permite auditar cada resultado, dónde se conserva y si forma parte del historial versionado. El catálogo distingue las ejecuciones oficiales citadas, los análisis auxiliares que sustentan figuras o comprobaciones y dos ejecuciones de Fase 2 mantenidas únicamente para documentar la sensibilidad del pipeline a la configuración.

D.1 Convenciones de lectura

Se considera *rastreado* un artefacto devuelto por `git ls-files` y, por tanto, recuperable desde el commit de la memoria. Un fichero *local* existe en el entorno de trabajo, pero está excluido por `.gitignore`; su existencia no puede presuponerse en otro clon. La indicación *histórico auditable* significa que se conservan la configuración y los resultados originales, aunque el código actual no reproduzca necesariamente la ejecución byte a byte.

Los tipos de fichero tienen funciones diferenciadas:

- `config.json` registra, según el tipo de ejecución, los parámetros y, cuando procede, la partición, la semilla y la procedencia de las entradas;

- `metrics.json` o `validation_results.json` contiene la salida cuantitativa de una ejecución;
- `environment.json` fija el entorno de software observado durante MAIN;
- `scaler.joblib`, `train_percentiles.npz` y `feature_names.json` conservan el estado del preprocesamiento;
- `artifact_manifest.json` vincula los artefactos confiables mediante rutas y resúmenes SHA-256;
- los ficheros de predicciones permiten auditar salidas por flujo sin convertir una muestra rastreada en sustituto del conjunto completo local.

Tabla D.1: Catálogo de artefactos que respaldan los resultados de la memoria. Las rutas y el estado de seguimiento proceden del árbol Git de la revisión documentada.

Elemento	Ruta y artefactos principales	Estado y uso en la memoria
Entrenamiento principal y partición de referencia		
MAIN QRDQN	<code>runs/cicids2017/MAIN_qrdqn_cicids2017_canonical_full_random_20260609_193655/:</code> <code>artifact_manifest.json</code>, <code>config.json</code>, <code>environment.json</code>, <code>feature_names.json</code>, <code>metrics.json</code>, <code>scaler.joblib</code>, <code>train_percentiles.npz</code> y escalares TensorBoard exportados. Modelo: <code>models/MAIN_qrdqn_cicids2017_canonical_full_random_20260609_193655.zip</code>.	Oficial y rastreado. Respaldan el entrenamiento principal, la evaluación aleatoria fija, el preprocesamiento reutilizado y las curvas de entrenamiento. El evento TensorBoard bruto es local; las exportaciones CSV son la fuente durable.

Continúa en la página siguiente

Continuación de la Tabla D.1

Elemento	Ruta y artefactos principales	Estado y uso en la memoria
Referencia de prueba, semilla 42	runs/cicids2017/test_partition_reference_seed42.json.	Rastreado. Fija conteos y hashes de la partición usada para comprobar que los futuros presupuestos de entrenamiento conservan el mismo conjunto de prueba.
Comprobaciones y análisis de validación		
VAL A	runs/validation/VAL_checks_A_20260212_235443/: config.json y validation_results.json.	Oficial citado, rastreado e histórico auditable. Verifica predicción directa; su configuración apunta a un modelo C01 ya retirado del árbol actual.
VAL B	runs/validation/VAL_checks_B_20260212_235736/: config.json y validation_results.json.	Oficial citado, rastreado e histórico auditable. Registra el control con etiquetas barajadas; el modelo temporal reentrenado no se persistió.
VAL C	runs/validation/VAL_checks_C_20260213_004847/: config.json y validation_results.json.	Oficial citado, rastreado e histórico auditable. Respalda la partición por día y corresponde a una red proxy, no a los pesos MAIN; el modelo temporal no se persistió.

Continúa en la página siguiente

Continuación de la Tabla D.1

Elemento	Ruta y artefactos principales	Estado y uso en la memoria
Bootstrap MAIN	runs/validation/bootstrap_ci_seed42.json.	Rastreado. Contiene 10 000 remuestreos estratificados, los conteos de la matriz de confusión y la verificación independiente del modelo MAIN sobre la partición reproducida.
Duplicados	runs/validation/duplicate_analysis_seed42.json.	Rastreado. Cuantifica duplicados exactos y coincidencias entre entrenamiento y prueba para la partición aleatoria de semilla 42.

Línea base Random Forest

RF, partición aleatoria	runs/cicids2017/baseline_random_forest_comparison/rf_cicids2017_canonical_20260628_024735__random_split/: config.json y metrics.json. Modelo de esta partición: models/rf_cicids2017_canonical_20260628_024735.joblib.	Oficial y rastreado. Comparación directa con MAIN sobre la misma partición aleatoria fija; el modelo global conservado corresponde a este barrido.
RF, partición por día	runs/cicids2017/baseline_random_forest_comparison/rf_cicids2017_canonical_20260628_024735__day_split/: config.json y metrics.json.	Oficial y rastreado. Comparación con Check C sobre entrenamiento lunes-miércoles y prueba jueves-viernes.

Continúa en la página siguiente

Continuación de la Tabla D.1

Elemento	Ruta y artefactos principales	Estado y uso en la memoria
RF, miércoles re-tenido	runs/cicids2017/baseline_random_forest_comparison/rf_cicids2017_canonical_20260628_024735_leave_one_out_wednesday/: config.json y metrics.json.	Oficial y rastreado. Resultado exclusivo de RF: no existe todavía un artefacto QRDQN equivalente.
Inferencia offline de Fase 2		
P2v2 inicial	runs/phase2/P2v2_pred_20260224_004121/: config.json, metrics.json, diagnostics.json y predictions.csv.	Rastreado como contexto histórico. Documenta una tasa de bloqueo extrema bajo una configuración anterior; no respalda el resultado oficial de Fase 2.
P2v2 posterior	runs/phase2/P2v2_pred_20260408_230318/: config.json, metrics.json y predictions.csv.	Rastreado como contexto histórico. Su comportamiento distinto evidencia sensibilidad a la configuración y obliga a citar cada RUN_ID.
P2v2 MAIN	runs/phase2/P2v2_pred_20260610_161231_MAIN/: config.json, metrics.json, diagnostics.json y predictions_head_10000.csv. El fichero completo predictions.csv.gz es local.	Oficial. Configuración, métricas, diagnósticos y muestra de 10 000 predicciones rastreados; predicciones completas ignoradas por la regla *.gz. La evidencia se limita al laboratorio cerrado generado por el operador.

D.2 Proveniencia de la entrada de Fase 2

El fichero `config.json` de `P2v2_pred_20260610_161231_MAIN` conserva la ruta histórica `pcaps/synthetic_real_traffic.csv`. Después de la ejecución, la entrada se renombró localmente a `pcaps/lab_capture_traffic.csv` para evitar una descripción incorrecta: son paquetes reales capturados en un laboratorio doméstico cerrado, cuyo tráfico y etiquetas de intención fueron generados por el operador, no filas de características sintetizadas algorítmicamente. La ruta de la configuración no se reescribe porque forma parte de la procedencia inmutable de aquella ejecución. El CSV de entrada y el fichero completo de predicciones permanecen ignorados por su tamaño y por la sensibilidad potencial de los metadatos; la muestra rastreada sirve para auditoría, no para reconstruir el conjunto completo.

D.3 Elementos excluidos del catálogo oficial

Las sondas C01–C03, los intentos MAIN rápidos y las demás ejecuciones antiguas de Fase 2 se conservan bajo `runs/archive/` para trazabilidad histórica, pero no respaldan las cifras oficiales. Del mismo modo, `runs/cicids2017/baseline_random_forest_comparison/results_rf.txt` corresponde a un prototipo superado y no se utiliza como evidencia. Los experimentos QRDQN *leave-one-CSV-out*, la escalera de tamaños de entrenamiento y el estudio multisemilla no aparecen como ejecuciones porque siguen pendientes de GPU: existen el código o el protocolo, pero no artefactos comprometidos ni métricas que catalogar.